

Advanced Machine Learning Subsidiary Notes

Lecture 1: When Machine Learning Works

Adam Prügel-Bennett

January 30, 2024

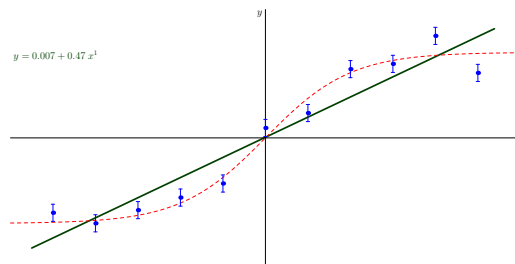
1 Keywords

- When ML Works, Bias Variance

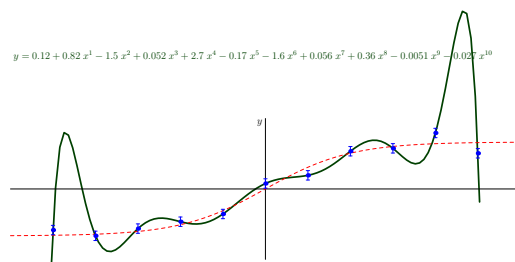
2 Main Points

2.1 Generalisation

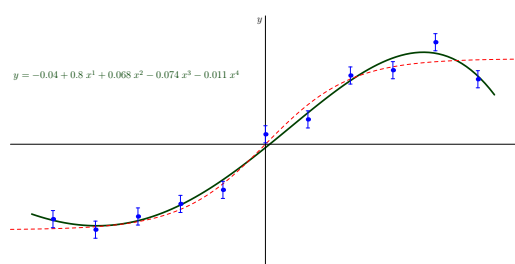
- We train our learning machines on a finite data set
- But we use our learning machines on unseen data
- If we have a too simple machine we might not be able to fit the training data and are unlikely to do well on unseen data



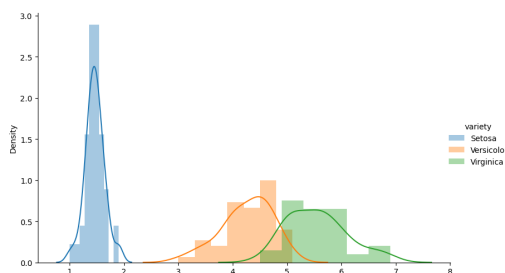
- If we have a too complicated machine we might be able to fit the training data almost perfectly, but we might have learnt a too complex rule that doesn't fit the test set



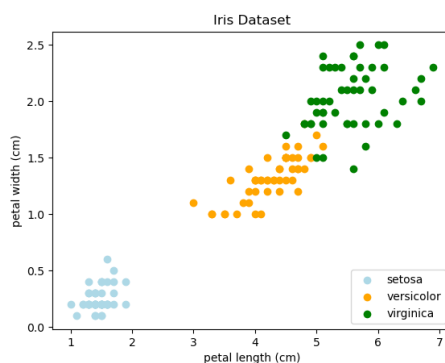
- Often there is a good compromise so that the learning machine learns a simple rule that fits the training data quite well but isn't too complicated



- We can think of the features and labels coming from an underlying distribution $\mu(x, y)$. For example, in the famous iris dataset we have four features (petal/sepal length/width). We imagine there is some probability density function for each feature. The petal length might be distributed as shown



- The features are actually jointly distributed (in 4 "dimensions"). E.g. the petal length and petal width are quite correlated



- An assumption we make (which has to be approximately correct for machine learning to have a chance of working) is that the testing data (where we are going to use the machine) is similar to the training data. Mathematically we can model this by assuming they both come from the same underlying distribution, $\mu(x, y)$

2.2 Bias-Variance Dilemmas

- We assume that we are trying to learn some function $f(x)$ where x are feature vectors
- Our task is to learn a function $\hat{f}(x|\mathcal{D})$ based on a training set $\mathcal{D}$
- We consider a scenario where we draw different training datasets $\mathcal{D}$ from a distribution of training examples $p(x)$
- Each training set contains m independent examples
- We start from the definition of the *mean machine*

$$\hat{f}_m(x) = \mathbb{E}_{\mathcal{D}} [\hat{f}(x|\mathcal{D})]$$

- The mean machine makes a prediction by averaging the results of machines trained on all possible learning datasets (clearly this is a thought experiment and not something practical)
- Now the **bias** is equal to generalisation performance of mean machine

$$B = \sum_{x \in \mathcal{X}} p(x) \left(\hat{f}_m(x) - f(x) \right)^2$$

- We consider the expected generalisation loss for a randomly drawn dataset
 - For any particular dataset we might do better or worse than this expected generalisation loss

$$\begin{aligned}
 \bar{L}_G &\stackrel{(1)}{=} \mathbb{E}_{\mathcal{D}}[L_G(\mathcal{D})] \stackrel{(2)}{=} \mathbb{E}_{\mathcal{D}} \left[\sum_{\mathbf{x} \in \mathcal{X}} p(\mathbf{x}) \left(\hat{f}(\mathbf{x}|\mathcal{D}) - f(\mathbf{x}) \right)^2 \right] \\
 &\stackrel{(3)}{=} \sum_{\mathbf{x} \in \mathcal{X}} p(\mathbf{x}) \mathbb{E}_{\mathcal{D}} \left[\left(\hat{f}(\mathbf{x}|\mathcal{D}) - f(\mathbf{x}) \right)^2 \right] \\
 &\stackrel{(4)}{=} \sum_{\mathbf{x} \in \mathcal{X}} p(\mathbf{x}) \mathbb{E}_{\mathcal{D}} \left[\left(\left(\hat{f}(\mathbf{x}|\mathcal{D}) - \hat{f}_m(\mathbf{x}) \right) + \left(\hat{f}_m(\mathbf{x}) - f(\mathbf{x}) \right) \right)^2 \right] \\
 &\stackrel{(5)}{=} \sum_{\mathbf{x} \in \mathcal{X}} p(\mathbf{x}) \left(\mathbb{E}_{\mathcal{D}} \left[\left(\hat{f}(\mathbf{x}|\mathcal{D}) - \hat{f}_m(\mathbf{x}) \right)^2 + \left(\hat{f}_m(\mathbf{x}) - f(\mathbf{x}) \right)^2 \right] \right. \\
 &\quad \left. + 2 \mathbb{E}_{\mathcal{D}} \left[\left(\hat{f}(\mathbf{x}|\mathcal{D}) - \hat{f}_m(\mathbf{x}) \right) \left(\hat{f}_m(\mathbf{x}) - f(\mathbf{x}) \right) \right] \right)
 \end{aligned}$$

- (1) This is the definition of the expected generalisation loss, $\bar{L}_G$
 - (2) The generalisation loss is the squared difference between the prediction of the learning machine, $\hat{f}(\mathbf{x}|\mathcal{D})$, and the true function, $f(\mathbf{x})$, averaged over all possible input feature vectors, $\mathbf{x}$, weighted by the probability of the input, $p(\mathbf{x})$
 - (3) We exchange the sum and expectation
 - (4) We add and subtract the prediction of the mean machine
 - (5) We expand out the sum
- The cross term cancels

$$\begin{aligned}
 C &= \mathbb{E}_{\mathcal{D}} \left[\left(\hat{f}(\mathbf{x}|\mathcal{D}) - \hat{f}_m(\mathbf{x}) \right) \left(\hat{f}_m(\mathbf{x}) - f(\mathbf{x}) \right) \right] \\
 &= \left(\mathbb{E}_{\mathcal{D}} \left[\hat{f}(\mathbf{x}|\mathcal{D}) \right] - \hat{f}_m(\mathbf{x}) \right) \left(\hat{f}_m(\mathbf{x}) - f(\mathbf{x}) \right) \\
 &= \left(\hat{f}_m(\mathbf{x}) - \hat{f}_m(\mathbf{x}) \right) \left(\hat{f}_m(\mathbf{x}) - f(\mathbf{x}) \right) = 0
 \end{aligned}$$

- Note we use the following properties of expectations
 - (1) $\mathbb{E}[A + B] = \mathbb{E}[A] + \mathbb{E}[B]$
 - (2) $\mathbb{E}[cA] = c\mathbb{E}[A]$ where c doesn't depend on the random variable you are averaging over
 - (3) $\mathbb{E}[1] = 1$
- We are left with

$$\begin{aligned}
 \bar{L}_G &= \sum_{\mathbf{x} \in \mathcal{X}} p(\mathbf{x}) \mathbb{E}_{\mathcal{D}} \left[\left(\hat{f}(\mathbf{x}|\mathcal{D}) - \hat{f}_m(\mathbf{x}) \right)^2 + \left(\hat{f}_m(\mathbf{x}) - f(\mathbf{x}) \right)^2 \right] \\
 &= \sum_{\mathbf{x} \in \mathcal{X}} p(\mathbf{x}) \mathbb{E}_{\mathcal{D}} \left[\left(\hat{f}(\mathbf{x}|\mathcal{D}) - \hat{f}_m(\mathbf{x}) \right)^2 \right] + \sum_{\mathbf{x} \in \mathcal{X}} p(\mathbf{x}) \left(\hat{f}_m(\mathbf{x}) - f(\mathbf{x}) \right)^2
 \end{aligned}$$

- Where we used the fact that the last term doesn't depend on the dataset
- The last term is equal to the bias, defined earlier as the generalisation performance of the mean machine
- The first term is known as the **variance**

$$V = \sum_{\mathbf{x} \in \mathcal{X}} p(\mathbf{x}) \mathbb{E}_{\mathcal{D}} \left[\left(\hat{f}(\mathbf{x}|\mathcal{D}) - \hat{f}_m(\mathbf{x}) \right)^2 \right]$$

- It measure how a single learning machine differs from the mean machine
- We therefore have $\bar{L}_G = B + V$ or

$$\text{Expected Generalisation Loss} = \text{Bias} + \text{Variance}$$

- The **Bias-Variance Dilemma** is that
 - Simple machine are likely to have high bias
 - * because any single machine can't represent the data well the mean machine won't be accurate
 - * this is true of the curve fitting example, but it is not true of decision trees where the average of many decision trees can learn a far more complex division boundary than a single machine
 - Complex machines are likely to have high variance
 - * Complex machine are likely to be sensitive to the training data whereas simpler machines (because of their lack of flexibility) aren't as sensitive
- A lot of this course will be looking at machines that cleverly resolve this dilemma

3 Experiments

Download the Jupyter Notebook

- This computes the training and generalisation loss as well as the bias and variance for arbitrary functions (at least approximately)
- We can do this because it is a 1-D function
- See if you can understand the code

3.1 Questions

- What is the effect of increasing the number of training points?
- What is the effect of using a more complex function, E.g. $e^{-x} \sin(x)$?

Advanced Machine Learning Subsidiary Notes

Lecture 2: Over-Fitting

Adam Prügel-Bennett

January 28, 2021

1 Keywords

- Overfitting, regularisation, feature selection

2 Main Points

- Over-fitting
 - Over-fitting is when we learn **spurious rules** that explain the training set
 - If we use an infinitely flexible machine then we can't generalise at all
- Controlling Complexity
 - More training example improves generalisation by allowing us to eliminate spurious rules
 - To achieve good generalisation performance machine must match the underlying structure of the problem
 - By preprocessing or careful feature engineering you can sometimes make the learning task easier
 - Deep learning works because
 - * It uses a lot of training data
 - * They use convolutions that
 - have many fewer parameter to learn than fully connected layers
 - respect translation invariance
 - find local features (at different scales through the network)
 - We usually don't know the structure of the inputs (they are too high dimensional)
 - * Try different machines to see what fits
 - * Fine-tune the models (adjusting hyper-parameters)
 - * Feature Engineering very often help
 - Feature selection
 - Using PCA or clustering
 - Normalise your input features
 - * Need a validation set to do this
 - * Beware you are very likely to over-estimate your generalisation error if you use your test set to select your model
 - * Can use K -fold cross-validation to get a better estimate of generalisation error
- Regularisation
 - Adding regularisation terms can help choose a simpler model

- L_2 regularisers punish large weights making the fitting function smoother
- L_1 regularisers can do automatic feature selection
- Can do subtle forms of regularisation such as choosing a maximum margin solution

3 Exercises

3.1 Estimating the error in the mean

- Consider a binary classification problem
- Suppose we measure the accuracy on a validation set of size $n = 10\,000$
- If we correctly predict 8 000 of the validation set what is
 1. an estimate of the mean accuracy?
 2. the accuracy of this estimate?
- State you assumptions
- What would the answer be if $n = 100$ and we got 80 correct on the validation set?
- Answer at the end

4 Experiments

4.1 Overfitting Game

- Write your own simulation (see lecture notes)
- Generate n random numbers $X_i \sim U(0, 3)$ (you can try different distributions)
- Add some noise to generate $Y_i = X_i + Z_i$ where $Z_i \sim \mathcal{N}(0, 1)$
- Choose i^* with the largest value of Y_i
- Compute $\Delta = Z_{i^*} = Y_{i^*} - X_{i^*}$
- Repeat many times and plot a histogram

4.2 Cross-validation

- Try using cross validation

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn import datasets
from sklearn import svm
```

```
X, y = datasets.load_iris(return_X_y=True)
X.shape, y.shape
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.4, random_state=0)
```

```
X_train.shape, y_train.shape
```

```
X_test.shape, y_test.shape
```

```
clf = svm.SVC(kernel='linear', C=1).fit(X_train, y_train)
clf.score(X_test, y_test)
```

4.3 Project

- There are a lot of practical tips that you can use in your project
 - Try them out and see what works and what doesn't
 - Many modifications won't make much difference, but occasionally one will make a lot of difference
 - Be very wary of estimating your generalisation performance
 - * You are measuring on a finite validation set so you will have errors
 - * Once you've used your validation set for selecting models, you're likely to over-estimate your generalisation error

5 Answers

5.1 Estimating the error in the mean

- We assume that the examples in the validation set are independent so the number of successes, k , will be binomially distributed

$$\text{Bin}(k|n, p) = \binom{n}{k} p^k (1-p)^{n-k}$$

- We don't know the success probability p , but an unbiased estimate of it is the number of observed success divided by n , that is 0.8
- For a binomial the variance in k is given by $np(1-p)$ or 1600 if $p = 0.8$ and $n = 10\,000$
- The typical size of the fluctuations in k would be $\sqrt{np(1-p)} = 40$
- Our estimate of p would therefore typically have fluctuations of size $40/n = 0.004$
- Thus our estimated success rate is 0.8 ± 0.004 or 80% with an error of about half a percent
- This is small because we used a large validation set. If $n = 100$ and we got 80 successes then we would have an error of 0.8 ± 0.04 so we have an error of 4%.
- Note that by choosing the best of 10 machines we might expect to over-estimate the performance by around 5% in this case

Advanced Machine Learning Subsidiary Notes

Lecture 4: Symmetry

Adam Prügel-Bennett

January 29, 2024

1 Keywords

- Inductive Bias, Symmetry, Invariance, Group theory

2 Main Points

2.1 Inductive Bias

- Learning machines are not universal approximators, but rather have a propensity to learn a specific set of rules
- This is known as their *inductive bias*
- We want to choose the learning machine so the inductive bias reflects the properties of the problem
- In almost all problems we use some smoothness property to select slowly varying functions

2.2 Invariances

- For many problems there are symmetries that you want your learning machine to respect
- For example in image classification we consider the class of an object to be invariant to its position in an image
- Learning machines whose outputs are invariant to translations have a much smaller set of functions to learn and therefore are much less likely to overfit
- CNNs are equivariant in that if we translate the input the the output will be translated in the same way
- Building classifiers using layers of CNN builds in a lot of translational invariance
- Ideally we want a classifier system to also be scale and rotation invariant, although we don't know how to this well
- Sometimes we use pyramid systems where we rescale the image and run the same operations on all scales
- However, in my datasets the foreground object is rescaled to fill most of the image with smaller object treated as the background so scale invariance is not necessarily required
- Often we try to learn invariance by augmenting the training set by artificially transforming the image (e.g. by reflecting, cropping, shearing and rescaling an image)
- Data augmentation usually gives a very significant improvement in performance
- We can sometimes achieve invariance by suitably normalising our data. Again when done right this can give a significant improvement in performance

2.3 Group Theory

- The mathematical language of symmetry is group theory
- You should know group theory exists, but I'm not going to examine you on details of group theory
- This considers a set of objects (we can think of these as transformations)
- Together with a binary operation we can think of as composition
- That is if $a, b \in \mathcal{G}$ are two transformations the $a \cdot b$ can be interpreted as applying transformation b followed by transformation a
- For the set, G , and binary operation, "." to form a group they must satisfy four conditions
 1. Closure: if $a, b \in \mathcal{G}$ the $a \cdot b \in \mathcal{G}$
 2. Associativity: for all $a, b, c \in \mathcal{G}$, we have $(a \cdot b) \cdot c = a \cdot (b \cdot c)$
 3. Existence of an identity element where $e \cdot a = a$
 4. Every element, $a \in \mathcal{G}$ has an inverse such that $a^{-1}a = e$
- (Group theory doesn't really care if the elements of the group are transformations, just that the axioms are satisfied)
- An example is the cyclic group C_2
 - We can think of this as tossing a coin with two actions
 - * flip the coin: f
 - * leave the coin alone: e
 - These form a group as
 1. Closure: $f \cdot e = f$, $f \cdot f = e$, $e \cdot e = e$ and $e \cdot f = f$ (if you don't believe this get out a coin and try. The first identity is don't do anything then flip the coin is the same as just flipping the coin)
 2. Associativity; this follows automatically from the operation being a composition (we will see this in innerProduct lecture where we look at matrix multiplication). You can also check rather boringly, e.g, $(e \cdot f) \cdot f = f \cdot f = e$ while $e \cdot (f \cdot f) = e \cdot e = e$ thus $(e \cdot f) \cdot f = e \cdot (f \cdot f)$. You can try this for all triples
 3. e is clearly the identity
 4. $e^{-1} = e$ and $f^{-1} = f$
 - The cyclic group C_3 corresponds to the rotations of an equilateral triangle maps the triangle to itself. While C_n are the rotations that keep an n sided polygon with identical sides onto itself
 - The *four group* is a second group of size 4 (besides C_4) that can be thought of as describing the flipping around a vertical and horizontal axis a rectangle, or rotating it by 180 degree around its centre of mass. We can describe the group in terms of a *multiplication table* where the first row and first column corresponds to the elements a_i and the i^{th} row and j^{th} column corresponds to the multiplication $a_j \cdot a_i$

	e	v	h	r
e	e	v	h	r
v	v	e	r	h
h	h	r	e	v
r	r	h	v	e

- * This is an "Abelian group" meaning that for any elements a, b we have $a \cdot b = b \cdot a$

- The group of permutations of n objects forms a group of size $n!$ called the symmetric group S_n
 - * This is a group as the composition of a permutation is a permutation
 - * There is an identity (doing nothing)
 - * For every element there is an inverse permutation (which is a permutation)
 - * For $n > 2$ (where $S_2 = C_2$) the group is non-Abelian meaning that there are permutations p_i and p_j where $p_1 \neq p_2$
 - * Despite this composing permutations is associative
 - * Permutations have a lot of structure and S_n is very well studied
- There exists groups with an infinite number of elements. The set of integers with the operation of addition form a group with identity, 0, and the inverse of n is $-n$
 - * You can think of these describing an infinite set of discrete shifts
 - * You first started studying this group when you first learn to count
- The set of rational numbers under addition also forms a group
- The reals under addition form a group describing any one dimensional translation
- The set of reals excluding 0 forms a group under multiplication with an identity 1 and where the inverse of x is $1/x$
 - * These can be thought of as describing scale transformations
- Groups can be interpreted in different ways with their elements having different names (e.g.)one, two, three, $\dots$ or un, deux, trois, $\dots$). The permutation of two elements has the same structure (is isomorphic to) flipping a coin. What counts is the structure of the multiplication time (up to re-ordering the elements)

2.3.1 Lie Groups

- When we consider continuous transformation in high dimensional space then we find a set of very interesting groups known as Lie groups
- These describe things like general translational invariance, rotational invariance, relativistic invariance
- The set on transformations can be represented as matrices, with the composition operator being matrix multiplication
- Different groups correspond to different sets of matrices.
- A well known group is the set of $n \times n$ orthogonal matrices $O(n)$
- By definition of orthogonal matrices $\mathbf{M}^T \mathbf{M} = \mathbf{M} \mathbf{M}^T = \mathbf{I}$
- These correspond to rotations and reflections
- Note that, in general, $\det(\mathbf{M}^T) = \det(\mathbf{M})$ and $\det(\mathbf{A} \mathbf{B}) = \det(\mathbf{A}) \times \det \mathbf{B}$
- As a consequence $\det(\mathbf{M}^T \mathbf{M}) = \det(\mathbf{I}) = 1$ so that $\det(\mathbf{M})^2 = 1$ or $\det(\mathbf{M}) = \pm 1$
- If $\det(\mathbf{M}) = -1$ then the transformation involves a reflection
- The special orthogonal group, $SO(n)$, corresponds to a subgroup of matrices in $O(2)$ with $\det(\mathbf{M}) = 1$. These correspond to rotations only
- The group $SO(2)$ corresponds to 2-d rotations where the elements of the group are matrices

$$\begin{pmatrix} \cos(\theta) & \sin(\theta) \\ -\sin(\theta) & \cos(\theta) \end{pmatrix}$$

Advanced Machine Learning Subsidiary Notes

Lecture 5: Ensemble Methods

Adam Prügel-Bennett

January 19, 2024

1 Keywords

- Decision Trees, Averaging, Bagging

2 Main Points

2.1 Decision Trees

- Decision trees are binary tree where a set of data is partitioned into two at each node of the tree
- Rule for partition depends on a single feature
- Trees are grown greedily
- All possible rules are tried to find the one that maximise the purity of the leaves
- Gini, entropy or variance used to measure purity
- Decision trees can handle categorical or numerical data
- They can handle missing data
 - after deciding on a new rule (ignoring missing data), a rule to send the missing data right or left is selected to maximise purity
- Decision trees are useful for understanding new data sets
 - By examining the rules at the top of the tree we get to see the most important variables
- Decisions trees are often not competitive
 - they can have high bias because they can only split the data on single variables (limits their expressiveness)
 - they can also have high variance because a small change to the data set that changes a rule at the top of the tree can lead to very different predictions

2.2 Ensembling

- This is used to reduces variance by averaging many different machines
- This only works if the machines make weakly correlated classifications
- We ensemble machines by coming to a consensus
 - Vote on a class in classification
 - Average for regression

2.2.1 Estimating the Mean

- Suppose we want to estimate the mean of a distribution, $f_X(x)$ from random samples of the distribution. We assume the distribution has mean $\mu = \mathbb{E}[X]$ and variance $\sigma^2 = \mathbb{E}[(X - \mu)^2]$
- Our estimated mean is

$$\hat{\mu} = \frac{1}{n} \sum_{i=1}^n X_i$$

where $X_i \sim f_X(x)$ (i.e. the X_i are drawn at random from the distribution)

- In expectation

$$\mathbb{E}[\hat{\mu}] = \frac{1}{n} \sum_{i=1}^n \mathbb{E}[X_i] = \frac{1}{n} \sum_{i=1}^n \mu = \mu$$

- However, each $\hat{\mu}$ will differ from the true μ (some will be greater and some less than μ)
- We want to measure the variance in $\hat{\mu} - \mu$

$$\hat{\mu} - \mu = \left(\frac{1}{n} \sum_{i=1}^n X_i \right) - \mu = \frac{1}{n} \sum_{i=1}^n (X_i - \mu)$$

- Thus

$$(\hat{\mu} - \mu)^2 = \left(\frac{1}{n} \sum_{i=1}^n (X_i - \mu) \right)^2 = \left(\frac{1}{n} \sum_{i=1}^n (X_i - \mu) \right) \left(\frac{1}{n} \sum_{j=1}^n (X_j - \mu) \right)$$

- Note that i and j are just dummy indices (it doesn't matter what they are called but they are different)

- Or

$$(\hat{\mu} - \mu)^2 = \frac{1}{n^2} \sum_{i=1}^n \sum_{j=1}^n (X_i - \mu)(X_j - \mu)$$

- Now there are times when $i = j$ and times when this isn't true so we separate these two out

$$(\hat{\mu} - \mu)^2 = \frac{1}{n^2} \sum_{i=1}^n (X_i - \mu)^2 + \sum_{i=1}^n \sum_{\substack{j=1 \\ j \neq i}}^n (X_i - \mu)(X_j - \mu)$$

- Taking expectations

$$\mathbb{E}[(\hat{\mu} - \mu)^2] = \frac{1}{n^2} \sum_{i=1}^n \mathbb{E}[(X_i - \mu)^2] + \sum_{i=1}^n \sum_{\substack{j=1 \\ j \neq i}}^n \mathbb{E}[(X_i - \mu)(X_j - \mu)]$$

- where I have used $\mathbb{E}[A + B] = \mathbb{E}[A] + \mathbb{E}[B]$ (so that $\mathbb{E}[\sum_i A_i] = \sum_i \mathbb{E}[A_i]$)

- Now because X_i and X_j are by assumption independently chosen

$$\mathbb{E}[(X_i - \mu)(X_j - \mu)] = \mathbb{E}[X_i - \mu] \mathbb{E}[X_j - \mu]$$

but $\mathbb{E}[X_i - \mu] = \mathbb{E}[X_i] - \mu = \mu - \mu = 0$ so

$$\mathbb{E}[(X_i - \mu)(X_j - \mu)] = 0$$

and

$$\mathbb{E}[(\hat{\mu} - \mu)^2] = \frac{1}{n^2} \sum_{i=1}^n \mathbb{E}[(X_i - \mu)^2] = \frac{1}{n^2} \sum_{i=1}^n \sigma^2 = \frac{\sigma^2}{n}$$

2.2.2 Bagging

- Uses bootstrap aggregation
 - that is we sample the training set with replacement to obtain slightly different datasets
- Random Forest
 - Uses decision trees with bootstrap aggregation
 - But also use different random subsets of features
 - Often gives state-of-the-art performance

3 Exercises

3.1 Compute expected error in mean for correlated variables

- We look at estimating the mean by sampling n random variables

$$\hat{\mu} = \frac{1}{n} \sum_{i=1}^n X_i$$

- In expectation $\mathbb{E}[X_i] = \mu$
- Thus in expectation $\mathbb{E}[\hat{\mu}] = \mu$
- But there will typically be fluctuations from the mean which we can compute using

$$\sigma_{\hat{\mu}}^2 = \mathbb{E}[(\hat{\mu} - \mu)^2]$$

- **Compute this** using

$$\mathbb{E}[(X_i - \mu)^2] = \sigma^2$$

$$\mathbb{E}[(X_i - \mu)(X_j - \mu)] = \rho \sigma^2$$

- Answer given in lecture notes
- Note that the estimated error in the mean is $\sigma_{\hat{\mu}}$ (this is what you use when you compute error-bars)

4 Experiments

4.1 Visualise a decision tree

```
from sklearn.datasets import load_iris
from sklearn import tree
X, y = load_iris(return_X_y=True)
clf = tree.DecisionTreeClassifier()
clf = clf.fit(X, y)

tree.plot_tree(clf.fit(iris.data, iris.target))
```

Advanced Machine Learning Subsidiary Notes

Lecture 6: Boosting

Adam Prügel-Bennett

January 19, 2024

1 Keywords

- Boosting, AdaBoost, Gradient Boosting

2 Main Points

2.0.1 Boosting

- Boosting constructs a *strong learner* as a weighted sum of *weak learners*
- Adaboost
 - Used for binary decisions
 - Start with a set of weak learners, $\mathcal{W}$
 - Each weak learner $h_i(\mathbf{x})$ outputs ± 1
 - Greedily build the strong learner by adding $\alpha_t h_t(\mathbf{x})$ at iteration t
 - Uses an exponential "error" to choose the weak learner and α_t
 - Algorithm does the following
 - * Define a weight, w_t^μ , for each training example $(\mathbf{x}^\mu, y^\mu)$
 - initially these are set to 1
 - Large weight implies the training example is poorly predicted
 - * Choose the weak learner, h_t that fails only where prediction is good
 - it decides this by summing the weights of training examples where the weak learner makes an error
 - it choose the weak learner with the smallest sum
 - * Choose the parameter α_t to minimises the error
 - Need to understand derivation and resulting algorithm (this is complicated)
- Gradient Boosting
 - Used on regression problems
 - Iterative algorithm where we learn a new weak learner that minimises the residual errors
 - Uses very small decision trees for regression
- Performance of Boosting
 - Can over-fit (use early stopping)
 - Only works for very simple weak-learners (strong learners will over-fit)
 - Can give state-of-the-art performance

Advanced Machine Learning Subsidiary Notes

Lecture 5: Vector Spaces

Adam Prügel-Bennett

January 17, 2024

1 Keywords

- Vectors, vector spaces, operators

2 Main Points

2.1 Vector Spaces

- Any set of objects with addition between members of the set and scalar multiplication forms a vector space if they satisfies 8 axioms
- Most of these axioms arise naturally if addition and scale multiplication behave normally
- The only additional axiom is closure
- Normal vectors, matrices and functions all form vector spaces

2.2 Distances

- A *proper distance* or *metric* between objects in a vector space satisfies 4 conditions
 1. $d(x, y) \geq 0$ (non-negativity)
 2. $d(x, y) = 0$ if and only if $x = y$ (identity of indiscernibles)
 3. $d(x, y) = d(y, x)$ (symmetry)
 4. $d(x, y) \leq d(x, z) + d(z, y)$ (triangular inequality)
- You can define different distances for the same set of objects
- Often we use *pseudo-metrics* that breaks one or other of the conditions
- Consider a function (mapping) $f : \mathbb{R}^n \rightarrow \mathbb{R}^n$ then a useful quantification of how big a change a mapping can produce is given by the Lipschitz condition

$$d(f(x), f(y)) \leq K d(x, y)$$

This is useful if for all x and y there exists some fixed K

- Lipschitz functions are continuous (have no jumps)
- If $K < 1$ the mapping is said to be a contractive mapping

2.3 Norms

- Norms provide a measure of the size of a vector
- They satisfy three conditions
 1. $\|v\| > 0$ if $v \neq 0$ (non-negativity)
 2. $\|a v\| = |a| \|v\|$ (linearity)
 3. $\|u + v\| \leq \|u\| + \|v\|$ (triangular inequality)
- Again if not all of these conditions are true we have *pseudo-norms*
- Norms provide a metric $d(x, y) = \|x - y\|$
- We will meet norms very often in this course
- **Vector Norms**
 - There are a large number of norms for normal vectors that people use
 1. Euclidean or 2-norm: $\|v\|_2 = \sqrt{v_1^2 + v_2^2 + \dots + v_n^2}$
 2. p -norm: $\|v\|_p = (\sum_{i=1}^n |v_i|^p)^{1/p}$
 3. 1-norm: $\|v\|_1 = \sum_{i=1}^n |v_i|$
 4. ∞ -norm or max-norm: $\|v\|_\infty = \max_i |v_i|$
 - Note the 1-norm, 2-norm and ∞ -norm are all p -norms with different p
 - The 0-norm counts the number of non-zero components (it is a pseudo-norm as it is not linear)
- **Matrix Norms**
 - Matrices also have norm
 1. The Frobenius norm is $\|A\|_F = \sqrt{\sum_{i=1}^m \sum_{j=1}^n |A_{ij}|^2}$
 2. Also have 1-norm, max-norm, Hilbert-norm (the maximum absolute eigenvalue), nuclear-norm, etc.
 - Note that the determinant is not a norm because it can be negative and is not linear
 - Many of the commonly used matrix norms satisfy
$$\|AB\| \leq \|A\| \times \|B\|$$
 - This is really useful because we can quickly bound norms of products of matrices
 - Many matrix and vector norms are compatible
$$\|Mv\|_b \leq \|M\|_a \times \|v\|_b$$
 - E.g. Frobenius and Euclidean norms are compatible
 - One of the main uses of matrix norms is to understand by how much it can potentially increase the size of a vector
- **Function Norms**
 - The most common function norms are
 1. The L_2 -norm

$$\|f\|_{L_2} = \sqrt{\int_{x \in \mathcal{R}} f^2(x) dx}$$

where $\mathcal{R}$ is the region over which the function is define

2. The L_1 -norm

$$\|f\|_{L_1} = \int_{\mathbf{x} \in \mathcal{R}} |f(\mathbf{x})| \, d\mathbf{x}$$

3. The ∞ or max-norm

$$\|f\|_{\infty} = \max_{\mathbf{x} \in \mathcal{R}} f(\mathbf{x})$$

- Function norms are also used to define vector spaces
 1. The L_2 vector space is the set of functions such that all functions satisfy $\|f\|_{L_2} < \infty$
 2. The L_1 vector space is the set of functions such that all functions satisfy $\|f\|_{L_1} < \infty$
- In these vector spaces we only consider functions that measurable in the sense that $\|f\| > 0$ for any non-zero function

Advanced Machine Learning Subsidiary Notes

Lecture 8: Inner-Product Spaces

Adam Prügel-Bennett

February 9, 2024

1 Keywords

- Inner-product, operators

2 Main Points

3 Inner Products

- For some vector spaces we can define an *inner product* between pairs of vectors
- Inner products are scalars associated with two elements in a vector space
- They are generally denoted by $\langle \mathbf{x}, \mathbf{y} \rangle$
- To be an inner-product requires satisfying 5 axioms

1. $\langle \mathbf{x}, \mathbf{x} \rangle \geq 0$ for all $\mathbf{x} \in \mathcal{V}$
2. $\langle \mathbf{x}, \mathbf{x} \rangle = 0$ if and only if $\mathbf{x} = \mathbf{0}$
3. $\langle \alpha \mathbf{x}, \mathbf{y} \rangle = \alpha \langle \mathbf{x}, \mathbf{y} \rangle$
4. $\langle \mathbf{x}, \mathbf{y} + \mathbf{z} \rangle = \langle \mathbf{x}, \mathbf{y} \rangle + \langle \mathbf{x}, \mathbf{z} \rangle$
5. $\langle \mathbf{x}, \mathbf{y} \rangle = \langle \mathbf{y}, \mathbf{x} \rangle$

- The inner-product induces a norm: $\|\mathbf{x}\| = \sqrt{\langle \mathbf{x}, \mathbf{x} \rangle}$
- For normal vectors (i.e. $\mathbf{x} \in \mathbb{R}^n$) the standard inner product is the dot-product

$$\langle \mathbf{x}, \mathbf{y} \rangle = \mathbf{x}^\top \mathbf{y} = \sum_{i=1}^n x_i y_i$$

- We can define an inner product between functions

$$\langle f, g \rangle = \int_{x \in \mathcal{I}} f(x) g(x) \, dx$$

- For matrices we can define the Frobenius inner-product

$$\langle \mathbf{A}, \mathbf{B} \rangle_F = \text{Tr } \mathbf{A}^\top \mathbf{B} = \sum_{i,j} A_{ij} B_{ij}$$

which defines the Frobenius norm

$$\|\mathbf{A}\|_F = \sqrt{\langle \mathbf{A}, \mathbf{A} \rangle_F} = \sqrt{\sum_{i,j} A_{ij}^2}$$

The Frobenius norm is not a compatible norm.

- A really important inequality is the *Cauchy-Schwarz* inequality

$$\langle \mathbf{x}, \mathbf{y} \rangle^2 \leq \langle \mathbf{x}, \mathbf{x} \rangle \langle \mathbf{y}, \mathbf{y} \rangle = \|\mathbf{x}\|^2 \|\mathbf{y}\|^2$$

- Inner products allow us to define the notion of similarity

$$\begin{aligned}\langle \mathbf{x}, \mathbf{y} \rangle &= \|\mathbf{x}\| \|\mathbf{y}\| \cos(\theta) \\ \langle f(x), g(x) \rangle &= \|f(x)\| \|g(x)\| \cos(\theta)\end{aligned}$$

- $\cos(\theta)$ can be seen as a measure of the correlation between vectors (or functions)
- Because of Cauchy-Schwarz $\langle \mathbf{x}, \mathbf{y} \rangle / (\|\mathbf{x}\| \|\mathbf{y}\|)$ lies between -1 and 1 (so that we can represent this quantity by the cosine of an angle)

3.1 Coordinates or Basis Vectors

- Any set of vectors that span the entire vector space can be considered a set of basis vectors or coordinates
- If our bases are linearly independent then we have a set of non-degenerate basis function where each vector is unique
- The most convenient set of basis vectors are those where the bases are normalised and orthogonal $\langle \mathbf{b}_i, \mathbf{b}_j \rangle = \delta_{ij}$

- **Basis Functions**

- For a function space we can consider a set of basis functions
- A familiar set of functions define on the interval $[0, 2\pi]$ are the Fourier functions

$$\{1, \cos(\theta), \sin(\theta), \cos(2\theta), \sin(2\theta), \dots\}$$

- This basis set is orthogonal as for any two components $\langle b_i(\theta), b_j(\theta) \rangle = \delta_{ij}$
- There are many orthogonal polynomials that have similar properties
- Given an orthogonal set of functions $\{b_i(\mathbf{x})\}$ we can decompose a function $f(\mathbf{x})$ as a (infinite) vector $\mathbf{f}$ with components $f_i = \langle f(\mathbf{x}), b_i(\mathbf{x}) \rangle$
- This allows us to represent any function as a point in an infinite-dimensional space

3.2 Operators

- Operators transform elements of a vector space
- Consider the transformation or operator $\mathcal{T} : \mathcal{V} \rightarrow \mathcal{V}'$
- This says that $\mathcal{T}$ maps some object $\mathbf{x} \in \mathcal{V}$ to an object $\mathbf{y} = \mathcal{T}[\mathbf{x}]$ in a new vector space $\mathcal{V}'$
- **Linear Operators**

- Linear operators satisfy the two conditions
 1. $\mathcal{T}[a \mathbf{x}] = a \mathcal{T}[\mathbf{x}]$
 2. $\mathcal{T}[\mathbf{x} + \mathbf{y}] = \mathcal{T}[\mathbf{x}] + \mathcal{T}[\mathbf{y}]$
- Linear operators are really important because we can understand them
- For normal vectors the most general linear operation is

$$\mathcal{T}[\mathbf{x}] = \mathbf{M} \mathbf{x}$$

where $\mathbf{M}$ is a matrix

- For functions the most general linear operator is a kernel function

$$g(\mathbf{x}) = \mathcal{T}[f(\mathbf{x})] = \int K(\mathbf{x}, \mathbf{y}) f(\mathbf{y}) d\mathbf{y}$$

* Kernels appear in SVMs, SVRs, kernel-PCA, Gaussian Processes

- Often we are interested in operators that map vectors in a vector space to new vectors in the same vector space
 - $\mathcal{T} : \mathcal{V} \rightarrow \mathcal{V}$
 - The most general linear mapping for normal vectors that does this are square matrices

3.3 Matrices and Mappings

- Matrices, $\mathbf{M}$ are linear maps from one point $\mathbf{x}$ to another $\mathbf{y} = \mathbf{M}\mathbf{x}$
- The product of a matrix $\mathbf{C} = \mathbf{A}\mathbf{B}$ corresponds to applying the mapping $\mathbf{B}$ followed by $\mathbf{A}$
- For most matrices $\mathbf{A}\mathbf{B} \neq \mathbf{B}\mathbf{A}$ (we say matrix multiplication does not commute)
- There are pairs of matrices that do commute, but they need to share a special structure for this to happen
- Matrix multiplication is associative. That is, $(\mathbf{A}\mathbf{B})\mathbf{C} = \mathbf{A}(\mathbf{B}\mathbf{C})$

Advanced Machine Learning Subsidiary Notes

Lecture 9: Understand Mappings

Adam Prügel-Bennett

February 9, 2024

1 Keywords

- Mappings, Eigenvectors

2 Main Points

2.1 Inverse Problems

- Much of machine learning can be viewed as solving an inverse problem
- We collect data about the world by performing a series of measurements
- Our task is to infer properties of the world from the data

2.2 Over-Constrained Problems

- We can have contradictory data that our model cannot explain
- This may arise because
 - We have errors in the data
 - Our data contains insufficient information
 - Our model is too simple
- If we have more training data than free parameters this is likely to occur
- We typically solve this by introducing a loss function we minimise
- A classic example is to minimise the squared error

2.3 Under-Constrained Problems

- We can also be in a situation when many models (learning machines) explain the data
- This will typically happen when we have more free parameters than data
- Here we have to choose a particular model
- To do this requires (implicitly or explicitly) making additional assumptions
- For high-dimensional inputs we can be over-constrained in some directions and under-constrained in others

2.4 Ill-Conditioning

- Even when we are not under-constrained our inverse can be very sensitive to the data
- That is small errors can be strongly magnified
- Ill-condition leads to high variance in the bias-variance sense and hence poor generalisation

2.5 Linear Regression

- In linear regression we try to fit a linear model $y_i = x_i^T w$ (or in matrix form $y = Xw$)
- We use a squared error (so can cope with conflicting constraints)
- If we have more training examples than parameters the solution is given by the pseudo-inverse $w = (X^T X)^{-1} X^T y$
- If we have less training examples than parameters (or we are unlucky in that the training examples don't span the full space) then the problem is under-constrained and there are an infinity of solutions
- Even when we have more training examples than parameters the problem can be ill-conditioned

3 Exercises

3.1 Linear Regression

- Derive the formula for the weight vector in linear regression

4 Experiments

4.1 Eigensystems

- In either Matlab/Octave or python generate random matrices and check the matrix identities

```
X = randn(5,4) % generate a mock designer matrix with 5 inputs of length 4
M = X' * X      % compute a symmetrix matrix
[V,L] = eig(M)  % compute eigenvalues
V * L * V'      % should be identical to M
V * V'          % should be the identity matrix (up to rounding precision)
V' * V          % should be the identity matrix (up to rounding precision)
x = randn(4,1) % generate a random column matrix of length 4
y = randn(4,1) % generate another random column matrix of length 4
xp = V * x      % apply V to x
yp = V * y      % apply V to y
norm(x)         % compute Euclidean norm of x
norm(xp)        % should be the same as Euclidean nor of xp
x' * y          % compute inner product of x and y
xp' * yp'       % compute inner produce of xp and yp (should be the same as above)

Z = rand(4,5)   % consider a designer matrix where we would have more unknowns the examples
W = Z' * Z      % compute a covariance type matrix (except we don't subtract the mean
eig(W)          % compute eigenvalues (one should be 0 up to machine precision)
```

Advanced Machine Learning Subsidiary Notes

Lecture 10: Eigensystems

Adam Prügel-Bennett

February 9, 2024

1 Keywords

- Eigenvectors, Orthogonal Matrices, Eigenvector Decomposition

2 Main Points

2.1 Eigen-Systems

- We can understand ill-conditioning for linear regression by the eigen-decomposition of $\mathbf{M} = \mathbf{X}^T \mathbf{X}$
- This should be revision
- You know that an *eigenvector*, $\mathbf{v}$, satisfies $\mathbf{M} \mathbf{v} = \lambda \mathbf{v}$
- For a symmetric matrix there are n real orthogonal eigenvectors
- You can prove they are orthogonal
- **Orthogonal Matrices**
 - Putting the n eigenvectors into a matrix $\mathbf{V}$ with columns $\mathbf{v}_i$ we obtain an orthogonal matrix
 - The defining property of an orthogonal matrix is $\mathbf{V}^T \mathbf{V} = \mathbf{V} \mathbf{V}^T = \mathbf{I}$
 - They correspond to rotations (with a possible reflection)
- **Matrix Decomposition**
 - We can decompose a symmetric matrix, $\mathbf{M}$ as
$$\mathbf{M} = \mathbf{V} \mathbf{\Lambda} \mathbf{V}^T$$
 - Where $\mathbf{\Lambda}$ is a diagonal matrix of eigenvalues of $\mathbf{M}$ (i.e. $\Lambda_{ii} = \lambda_i$)
 - $\mathbf{V}$ is the orthogonal matrix made up of the eigenvectors of $\mathbf{M}$
 - We can interpret the mapping of a symmetric matrix $\mathbf{M}$ as equivalent to
 1. a rotation defined by $\mathbf{V}^T$
 2. scaling of the i^{th} component by λ_i and
 3. a rotation backwards given by $\mathbf{V}$
 - Equivalently if we work in a coordinate system defined by the eigenvectors of $\mathbf{V}$ (this forms an orthonormal basis set) then we just rescale in the directions $\mathbf{v}_i$ by λ_i
 - * A symmetric matrix just squashes or expands in different orthogonal directions (this is what the eigensystem captures)

- **Inverse Matrices**

- The inverse of a symmetric matrix $\mathbf{M}$ is given by

$$\mathbf{M} = \mathbf{V} \mathbf{\Lambda}^{-1} \mathbf{V}^T$$

- Where $\mathbf{\Lambda}^{-1}$ is a diagonal matrix with elements $\Lambda_{ii}^{-1} = 1/\lambda_i$
- This is only defined if $\mathbf{M}$ if all the eigenvalues of $\mathbf{M}$ are non-zero ($\mathbf{M}$ is said to be full rank)
- If λ_i is very small then $1/\lambda_i$ is large and in taking the inverse $\mathbf{M}^{-1}\mathbf{x}$ any component of $\mathbf{x}$ in the direction $\mathbf{v}_i$ will get magnified by $1/\lambda_i$
- For linear regression we invert $\mathbf{M} = \mathbf{X}^T \mathbf{X}$
 - * in directions where the training examples don't vary much the associated eigenvalue will be small and the inverse inherently unstable

3 Exercises

3.1 Linear Regression

- Derive the formula for the weight vector in linear regression

4 Experiments

4.1 Eigensystems

- In either Matlab/Octave or python generate random matrices and check the matrix identities

```

X = randn(5,4) % generate a mock designer matrix with 5 inputs of length 4
M = X' * X     % compute a symmetrix matrix
[V,L] = eig(M) % compute eigenvalues
V * L * V'     % should be identical to M
V * V'         % should be the identity matrix (up to rounding precision)
V' * V         % should be the identity matrix (up to rounding precision)
x = randn(4,1) % generate a random column matrix of length 4
y = randn(4,1) % generate another random column matrix of length 4
xp = V * x     % apply V to x
yp = V * y     % apply V to y
norm(x)        % compute Euclidean norm of x
norm(xp)       % should be the same as Euclidean nor of xp
x' * y         % compute inner product of x and y
xp' * yp'      % compute inner produce of xp and yp (should be the same as above)

Z = rand(4,5)  % consider a designer matrix where we would have more unknowns the examples
W = Z' * Z     % compute a covariance type matrix (except we don't subtract the mean
eig(W)         % compute eigenvalues (one should be 0 up to machine precision)

```

Advanced Machine Learning Subsidiary Notes

Lecture 11: Principal Component Analysis (PCA)

Adam Prügel-Bennett

February 9, 2024

1 Keywords

- Covariance matrices, dimensionality reduction, PCA, Duality

2 Main Points

2.1 PCA

- This is revision as you should have all seen this in foundations of ML
- The covariance matrix is defined as

$$\mathbf{C} = \frac{1}{m+1} \sum_{i=1}^m (x_i - \hat{\mu})(x_i - \hat{\mu})^\top$$

- Defining the matrix $\mathbf{X}$ as

$$\mathbf{X} = \frac{1}{\sqrt{m-1}} (x_1 - \mu, x_2 - \mu, \dots, x_m - \mu)$$

then $\mathbf{C} = \mathbf{X}\mathbf{X}^\top$

- The *principal components* are the eigenvectors of the covariance matrix with the largest eigenvalues
- We can reduce the dimensionality of the inputs by projecting into the subspace spanned by the principal components
- We can reconstruct a vector from its principal component projection

$$\hat{x} = \sum_i z_i v_i$$

- v_i are the principal components (eigenvectors of the covariance matrix with largest eigenvalues)
 - $z_i = v_i^\top x$ are the values of the new features
 - the sum is over the principal components that we
- The expected squared error reconstruction loss $\mathbb{E}[(\hat{x} - x)^2]$ is equal to the sum of the eigenvalues we ignore

2.2 Duality

- We can define the dual matrix, $\mathbf{D} = \mathbf{X}^T \mathbf{X}$ with components $D_{kl} = (\mathbf{x}_k - \boldsymbol{\mu})^T (\mathbf{x}_l - \boldsymbol{\mu})$
- If $\mathbf{u}_i$ is an eigenvector of $\mathbf{D}$ with eigenvalue λ_i then $\mathbf{v}_i = \mathbf{D} \mathbf{u}_i$ is an eigenvector of $\mathbf{C}$ with the same eigenvalue
- Matrix $\mathbf{C}$ and $\mathbf{D}$ have exactly the same non-zero eigenvalues
- If we have more features than training examples it is more efficient to work with $\mathbf{D}$ than $\mathbf{C}$
- Note in this case the training examples will not span the feature space. $\mathbf{D}$ describes the fluctuations in the space spanned by the examples

3 Exercises

3.1 Duality

- Show that if $\mathbf{u}_i$ is an eigenvector of $\mathbf{D}$ with eigenvalue λ_i then $\mathbf{v}_i = \mathbf{D} \mathbf{u}_i$ is an eigenvector of $\mathbf{C}$ with the same eigenvalue
- Answer in the lecture notes

4 Experiments

4.1 Duality

- Using Matlab/Octave or python illustrate that the dual matrix and covariance matrix have the same eigenvalues

```
X = randn(5,3)    % construct a random matrix
C = X*X'           % compute a sort of covariance matrix (haven't bothered removing mean
D = X'*X           % compute dual
[V,LC] = eig(C)    % compute eigensystem of C
[U,LD] = eig(D)    % compute eigensystem of D should have the same non-zero eigenvalues
u1 = X*U(:,1)      % left multiply and eigenvector of D by X
u1/norm(u1)        % normalise above should be the same as V(:,3) (could be V(:,4) or V(:,5))
```

Advanced Machine Learning Subsidiary Notes

Lecture 12: Singular Value Decomposition (SVD)

Adam Prügel-Bennett

February 9, 2024

1 Keywords

- Singular Valued Decomposition, SVD, general linear maps

2 Main Points

2.1 Singular Value Decomposition

- Any $n \times m$ matrix, $\mathbf{X}$ can be decomposed as $\mathbf{X} = \mathbf{U} \mathbf{S} \mathbf{V}^T$
 - $\mathbf{U}$ is an $n \times n$ orthogonal matrix
 - $\mathbf{S}$ is an $n \times m$ matrix with zeros everywhere except the diagonal where $S_{ii} = s_i \geq 0$
 - $\mathbf{V}$ is an $m \times m$ orthogonal matrix
- The values s_i are known as the *singular values* of $\mathbf{X}$
- The SVD of a symmetric matrix is just the eigen-decomposition
- **Economical SVD**
 - If $n > m$ some algorithms won't bother outputting the last $n - m$ columns of $\mathbf{U}$
 - If $m < n$ some algorithms won't bother outputting the last $m - n$ columns of $\mathbf{V}$
 - In this case it will output a square $\mathbf{S}$ matrix

2.2 General Linear Mapping

- Recall that matrices are the most general linear operators
- Since any matrix $\mathbf{M}$ can be written as $\mathbf{U} \mathbf{S} \mathbf{V}^T$ we can interpret any linear mapping as doing three operations
 - A rotation (with possibly a reflection) defined by $\mathbf{V}^T$
 - A rescaling of each coordinate by s_i
 - A rotation (with possibly a reflection) defined by $\mathbf{U}$
- **Duality**
 - Using $\mathbf{X} = \mathbf{U} \mathbf{S} \mathbf{V}^T$ then
 - * $\mathbf{C} = \mathbf{X} \mathbf{X}^T = \mathbf{U} \mathbf{S} \mathbf{S}^T \mathbf{U}$
 - * $\mathbf{D} = \mathbf{X}^T \mathbf{X} = \mathbf{V} \mathbf{S}^T \mathbf{S} \mathbf{V}$
 - $\mathbf{S} \mathbf{S}^T$ and $\mathbf{S}^T \mathbf{S}$ are diagonal elements with non-zero diagonal elements s_i^2

2.3 Ridge Regression

- Ridge regression is linear regression with an L_2 regulariser
- Adding a regulariser $\nu \|w\|^2$ the weights, w^* , that minimise the loss function are given by $w^* = (\mathbf{X}^T \mathbf{X} + \nu \mathbf{I})^{-1} \mathbf{X}^T \mathbf{y}$
- Using $\mathbf{X} = \mathbf{U} \mathbf{S} \mathbf{V}^T$ then

$$w^* = \mathbf{V} \bar{\mathbf{S}}^+ \mathbf{U}^T \mathbf{y}$$

where $\bar{\mathbf{S}}^+$ is a regularised pseudo-inverse of $\mathbf{S}$ given by

$$\bar{\mathbf{S}}^+ = (\mathbf{S}^T \mathbf{S} + \nu \mathbf{I})^{-1} \mathbf{S}$$

- If $\nu = 0$ this is equal to the pseudo-inverse of $\mathbf{S}$
- $\bar{\mathbf{S}}^+$ is an $n \times m$ matrix which is zero everywhere except on the diagonal, where $\bar{S}_{ii}^+ = \frac{s_i}{s_i^2 + \nu}$
 - Note if $s_i = 0$ linear regression has an infinity of solutions and the pseudo-inverse of $\mathbf{X}$ does not exist (setting $\nu = 0$ we get $S_{ii}^+ = 1/s_i$ which is not defined when $s_i = 0$)
 - In the regularised case $\bar{S}_{ii}^+ = 0$ (we have selected one of the solutions that minimise the squared error)
 - If $s_i \ll \nu$ then without the regularisation term the inverse is very ill-conditioned while with the regularisation term $\bar{S}_{ii}^+$ will be small
 - If $s_i \gg \nu$ then $\bar{S}_{ii}^+ \approx \frac{1}{s_i} = S_{ii}^+$
- Adding a L_2 regulariser means that the optimum weights, w^* , will be less sensitive to the training data reducing the variance in the bias-variance dilemma

3 Exercises

3.1 Ridge regression

- Ridge regression is just linear regression with an L_2 regulariser
 1. Derive the optimal weights in ridge regression
 2. Show that using $\mathbf{X} = \mathbf{U} \mathbf{S} \mathbf{V}^T$ then $w^* = \mathbf{V} (\mathbf{S}^T \mathbf{S} + \nu \mathbf{I})^{-1} \mathbf{S} \mathbf{U} \mathbf{y}$
- See answers

4 Experiments

4.1 SVD

- Using Matlab/Octave or python have a play with svd

```
X = randn(3,4)           % construct a random matrix
[U,S,V] = svd(X)          % compute singular value decomposition
U*S*V'                    % should be the same as X
U*U'                      % should be the identity up to round error
U'*U                      % should be the identity up to round error
V*V'                      % should be the identity up to round error
V'*V                      % should be the identity up to round error
[Ue,L1] = eig(X*X')       % Ue should be the same as U up to permutation
S*S'                      % same as L1 up to permutation
```

```
[Ve,L2] = eig(X'*X) % Ve should be the same as V up to permutation
S'*S      % same as L2 up to permutation
```

```
inv(X'*X + 0.1*eye(4)) % check identity
V*inv(S'*S + 0.1*eye(4))*V' % should be the same
```

4.2 Verify Identity

- Again use Matlab/Octave or python
- For a random 4×5 matrix $\mathbf{X}$
 - Check that using $\mathbf{X} = \mathbf{U}\mathbf{S}\mathbf{V}^T$ that

$$(\mathbf{X}^T\mathbf{X} + \eta\mathbf{I})^{-1} = \mathbf{V}(\mathbf{S}^T\mathbf{S} + \nu\mathbf{I})^{-1}\mathbf{V}^T$$

holds for some random matrix using Matlab/Octave or python

- Examine $\mathbf{S}^T\mathbf{S}$, $\mathbf{S}^T\mathbf{S} + 0.1\mathbf{I}$, $(\mathbf{S}^T\mathbf{S} + 0.1\mathbf{I})^{-1}$ and $(\mathbf{S}^T\mathbf{S} + 0.1\mathbf{I})^{-1}\mathbf{S}^T$
- See if you can invert $\mathbf{X}^T\mathbf{X}$: it is singular, but due to rounding errors it might be inverted (it was a scary matrix when I tried it)

```
X = randn(4,5) % construct a random matrix
[U,S,V] = svd(X) % compute singular value decomposition
```

```
inv(X'*X + 0.1*eye(5)) % check identity
V*inv(S'*S + 0.1*eye(5))*V' % should be the same
```

```
S'*S % singular
S'*S + 0.1*eye(5) % now invertible
inv(S'*S + 0.1*eye(5))
inv(S'*S + 0.1*eye(5))*S' % 4x5 diagonal matrix

inv(X'*X) % shouldn't be able to do this
```

5 Answers

5.1 Ridge regression

1. It is straightforward to show

$$\mathbf{w}^* = (\mathbf{X}^T\mathbf{X} + \nu\mathbf{I})^{-1}\mathbf{X}^{-1}\mathbf{y}$$

2. The only hard part is to show is that

$$(\mathbf{X}^T\mathbf{X} + \nu\mathbf{I})^{-1} = \mathbf{V}(\mathbf{S}^T\mathbf{S} + \nu\mathbf{I})^{-1}\mathbf{V}^T$$

- It is easy to show that $\mathbf{X}^T\mathbf{X} = \mathbf{V}\mathbf{S}^T\mathbf{S}\mathbf{V}^T$
- But we also have $\mathbf{I} = \mathbf{V}\mathbf{V}^T$ as $\mathbf{V}$ is an orthogonal matrix
- Thus $\mathbf{M} = \mathbf{X}^T\mathbf{X} + \nu\mathbf{I} = \mathbf{V}(\mathbf{S}^T\mathbf{S} + \nu\mathbf{I})\mathbf{V}^T = \mathbf{V}\mathbf{W}\mathbf{V}^T$ where $\mathbf{W} = \mathbf{S}^T\mathbf{S} + \nu\mathbf{I}$
- But $(\mathbf{A}\mathbf{B}\mathbf{C})^{-1} = \mathbf{C}^{-1}\mathbf{B}^{-1}\mathbf{A}^{-1}$ (which we can verify by multiplying $\mathbf{C}^{-1}\mathbf{B}^{-1}\mathbf{A}^{-1}$ on either the left or right by $\mathbf{A}\mathbf{B}\mathbf{C}$)
- Thus $\mathbf{M}^{-1} = (\mathbf{V}\mathbf{W}\mathbf{V}^T)^{-1} = (\mathbf{V})^{T-1}\mathbf{W}^{-1}\mathbf{V}^{-1} = \mathbf{V}\mathbf{W}\mathbf{V}^T$ where we use $\mathbf{V}^{-1} = \mathbf{V}^T$ as $\mathbf{V}$ is an orthogonal matrix

Advanced Machine Learning Subsidiary Notes

Lecture 13: Optimisation

Adam Prügel-Bennett

February 9, 2024

1 Keywords

- Gradient descent, quadratic minima, differing length scales

2 Main Points

2.1 Optimisation

- Once you've designed your learning machine and chosen your loss function the rest is optimisation
- A very general method is to iteratively reduce your loss function
- In high dimensions the gradient of the loss function points in the direction of maximum increasing loss
- We still have the problem of determining the step size

- **Newton's Method**

- Uses the Hessian, $\mathbf{H}$, with elements

$$H_{ij} = \frac{\partial^2 L(\mathbf{w})}{\partial w_i \partial w_j}$$

- Assuming we are in a quadratic minimum the optima will be given by

$$\mathbf{w}^* = \mathbf{w} - \mathbf{H}^{-1} \nabla L(\mathbf{w})$$

- If we are not in a quadratic minimum, but sufficiently close we will converge to the minima *quadratically*

- * quadratically means that if we start with an error ϵ the error will be ϵ^2 after one iteration ϵ^4 after two iterations, etc.

- If we are a long way from the minimum we might go anywhere
- In very high dimensions it is not practical

- **Quasi-Newton Methods**

- There exists a host of methods that approximate the Hessian
- By ensuring the approximation is positive definite it means we move in directions that are positively correlated with the gradient
- Methods include *conjugate gradient* and *Levenberg-Marquardt*
- Levenberg-Marquardt is used for least squares problems

- These are usually preferred over Newton's methods as they are computationally cheaper
- **Gradient Descent**
 - The alternative to (quasi-)Newton methods is to just follow the negative gradient

$$w^{(t+1)} = w^{(t)} - r \nabla L(w^{(t)})$$

- If the step size, r , is too large we can diverge from quadratic minimum
- Need to tune the step size to the curvature of the problem
- In high dimension our maximum step size is limited by the direction with the greatest curvature (the largest eigenvalue of the Hessian)

3 Exercises

3.1 Divergence

- Assume a loss $L(w) = \frac{1}{2}w^2$ (we are in 1-dimension)
- If we update $w^{(t+1)} = w^{(t)} - r \nabla L(w^{(t)})$
 1. Compute the optimum step size
 2. What is $r_{\max}$ such that we no longer converge if $r \geq r_{\max}$
 3. If $r > r_{\max}$ calculate how $w^{(t)}$ grows with time
- Answer given below

3.2 Quadratic Optima

- Consider a 2-d loss function $L(w) = w_1^2/2 + w_2^2 - w_1 w_2$
 1. Compute the gradient
 2. Compute the Hessian
 3. Compute the eigenvalues of the Hessian
 4. Plot the contour lines of $L(w)$

4 Answers

4.1 Divergence

- The update equation is

$$w^{(t+1)} = (1 - r) w^{(t)}$$
 1. The optimum step size is $r = 1$ (the minimum is at $w = 0$)
 2. $r_{\max} = 2$
 3. $w^{(t)} = (1 - r)^t w^{(0)}$

4.2 Quadratic Optima

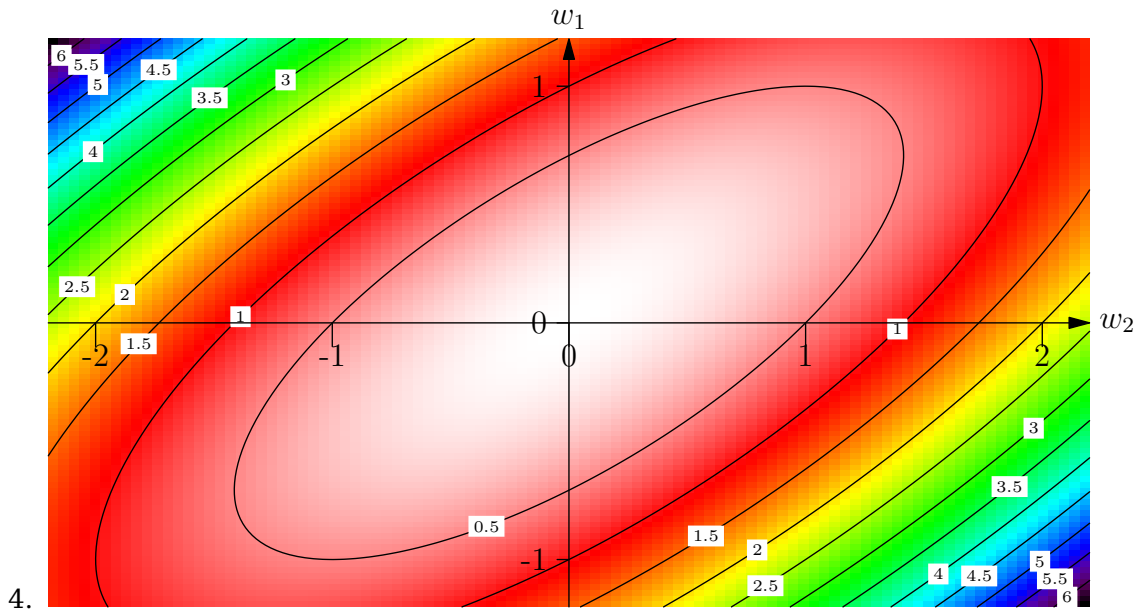
1.

$$\nabla L(\mathbf{w}) = \begin{pmatrix} w_1 - w_2 \\ 2w_2 - w_1 \end{pmatrix}$$

2.

$$\mathbf{H} = \begin{pmatrix} 1 & -1 \\ -1 & 2 \end{pmatrix}$$

3. Let $T = \text{tr } \mathbf{H} = 3$ and $D = \det(\mathbf{H}) = 1$ then $\lambda = \frac{1}{2} \left(T \pm \sqrt{T^2 - 4D} \right) = (3 \pm \sqrt{5})/2 = \{0.382, 2.618\}$



Advanced Machine Learning Subsidiary Notes

Lecture 14: Stochastic Gradient Descent

Adam Prügel-Bennett

February 9, 2024

1 Keywords

- SGD, momentum, step size, ADAM

2 Main Points

2.1 Stochastic Gradient Descent

- One can estimate the gradient from a mini-batch $\mathcal{B} \subset \mathcal{D}$

$$\nabla L_{\mathcal{B}}(\mathbf{w}) = \nabla \sum_{(\mathbf{x}, y) \in \mathcal{B}} L(\mathbf{x}, y | \mathbf{w})$$

where $L(\mathbf{x}, y | \mathbf{w})$ is the loss for example $(\mathbf{x}, y)$ with weights $\mathbf{w}$

- If $|\mathcal{B}| \ll |\mathcal{D}|$ this is massively faster than computing the full gradient
- This allows us to make relatively small steps very quickly
- By making lots of steps we average out the random errors
- **Comparison to 2nd order methods**
 - Newton and Quasi-Newton methods converge faster
 - But you only care when you are close to a minimum
 - Away from a minimum 2nd order methods can lead you astray
 - When using ReLUs the Loss landscape does not have a continuous first derivative so second order methods might not work
 - We want to minimise the generalisation error so reaching the minimum of the training error is perhaps not that important
 - In high dimensions 2nd order methods are impractical
- Automatic differentiation allow us to compute gradients for complicated loss functions for free (this is often a game changer)
- However, you still need to decide on the step size and you can diverge

2.2 Momentum

- By using "momentum" we remember our earlier movements
 - Allows us to take large steps in directions with small curvature
 - Cancels zig-zagging in directions with high curvature

- Introduce a "velocity"

$$\begin{aligned} \mathbf{v}^{(t+1)} &= (1 - \gamma) \mathbf{v}^{(t)} - \gamma r \nabla L_{\mathcal{B}}(\mathbf{w}^{(t)}) \\ \mathbf{w}^{(t+1)} &= \mathbf{x}^{(t)} + \mathbf{v}^{(t+1)} \end{aligned}$$

- γ might be small 0.1
- r is the usual step size

2.3 Adaptive Methods

- The difficulty of high dimensional optimisation is there are different curvatures
 - Where there is high curvature we want to make small steps
 - Where there is low curvature we want to make large steps
- In adaptive methods we change our step size for each variables
- We could measure the curvature in different directions

$$\frac{\partial^2 L(\mathbf{w})}{\partial w_i^2}$$

but most adaptive algorithms don't do this

- **AdaDelta**

- AdaDelta is an algorithm that estimates the curvature by computing a running mean squared gradient

$$S_i^{g(t+1)} = (1 - \gamma) S_i^g(t) + \gamma \left(\frac{\partial L_{\mathcal{B}}(\mathbf{w}^{(t)})}{w_i^{(t)}} \right)^2$$

- * This is a running average (it slowly forgets the past)
- We also compute a running average of the squared weight

$$S_i^{w(t+1)} = (1 - \gamma) S_i^w(t) + \gamma (w_i^{(t)})^2$$

- It then updates each weight according to

$$w_i^{(t+1)} = w_i^{(t)} - \eta \sqrt{\frac{S_i^w(t+1) + \epsilon}{S_i^g(t+1) + \epsilon}} \frac{\partial L_{\mathcal{B}}(\mathbf{w}^{(t)})}{\partial w_i^{(t)}}$$

- This ensures invariance in two ways
 - * If we multiply our weights by a factor we get the same relative change
 - * If we multiply our gradients by a factor we get the same change

- **ADAM**

- AdaDelta doesn't use momentum
- Adaptive Moment Estimation (ADAM) does both adaptive step-size per feature and it uses momentum
- It computes a running average momentum and squared gradient

$$M_i^{(t+1)} = (1 - \beta) M_i^{(t)} + \beta \frac{\partial L_{\mathcal{B}}(\mathbf{w}^{(t)})}{\partial w_i^{(t)}}$$

$$S_i^{(t+1)} = (1 - \gamma) S_i^{(t)} + \gamma \left(\frac{\partial L_{\mathcal{B}}(\mathbf{w}^{(t)})}{\partial w_i^{(t)}} \right)^2$$

- Running averages suffer from time-lag (it takes time for them to build-up)
- In ADAM we remove the time lag

$$\hat{M}_i^{(t+1)} = \frac{M_i^{(t+1)}}{1 - (1 - \beta)^t} \qquad \hat{S}_i^{(t+1)} = \frac{S^{(t+1)}}{1 - (1 - \gamma)^t}$$

- We then update the weights

$$w_i^{(t+1)} = w_i^{(t)} - \frac{\eta}{\sqrt{\hat{S}_i^{(t+1)} + \epsilon}} \hat{M}_i^{(t+1)}$$

- ADAM and its variants are very successful: often giving state-of-the-art performance

- **Covariance**

- The adaptive schemes works independently on each coordinate
- Covariance properties of vectors
 - * If we act on vectors using standard operations
 - scalar multiplication
 - addition
 - matrix multiplication
 then the results are invariant of the coordinate system we use
 - * In particular they will be translation and rotation invariant
 - * When we do elementwise multiplication this invariance is lost
 - * More generally this is true for tensors
 - * In machine learning although we call multi-dimensional arrays tensors we usually apply elementwise operations rather than proper tensor operations (we loose invariance to coordinate transformations)
- Because the adaptive schemes are elementwise they are not invariant to rotation
- If e_i is the direction of increasing weight w_i the if two weights interact we could have high curvature in a direction $e_i + e_j$ and low curvature in a direction $e_i - e_j$. We cannot adapt the weights individually to equalise the curvature.

2.4 Loss Landscapes

- In modern machine learning we are often perform minimisation of the loss function in a massive search space
- Unless the search space has a simple structure (e.g. is convex) there are likely to be many local optima
- There is no algorithm that is guaranteed to find the global minimum
- In such large spaces we might never get near to a minimum
- **Symmetries**
 - The loss landscape will typically have many symmetries
 - If we permute the nodes of an MLP or feature maps of a CNN we get the same solution
 - There may also be continuous symmetries
 - Most directions might not change the loss at all
 - Symmetries complicated the loss landscape
 - * If you have two local minima there will be a saddle-point in between
 - Adding skip connections removes permutation symmetries which seems to make optimisation simpler

3 Exercises

3.1 Removing Lag

- Consider a running average

$$a^{(t+1)} = (1 - \gamma) a^{(t)} + \gamma x^{(t)}$$

- Assume $x^{(t)} = x$ (i.e. constant)

- Calculate $a^{(t)}$ if $a^{(0)} = 0$ as a sum
- Using the fact that the sum of a geometric series can be written as

$$\sum_{i=0}^{t-1} z^i = 1 + z + \dots + z^{t-1} = \frac{1 - z^t}{1 - z}$$

write $a^{(t)}$ in closed form

- Compute the correction to the running mean so that the corrected running mean equals x for all t

3.2 Gradient Descent in a Quadratic Minimum

- Consider minimising a quadratic function

$$f(\mathbf{x}) = \frac{1}{2}(\mathbf{x} - \mathbf{x}^*)^\top \mathbf{Q}(\mathbf{x} - \mathbf{x}^*)$$

- Using gradient descent

$$\mathbf{x}(t+1) = \mathbf{x}(t) - r \nabla f(\mathbf{x})$$

- Using the definition of $f(\mathbf{x})$ write down the update equation
- Subtract $\mathbf{x}^*$ from both sides of the equation and write down an update equation for $\mathbf{x}(t) - \mathbf{x}^*$
- Write a closed form solution for $\mathbf{x}(t) - \mathbf{x}^*$
- Using the eigen-decomposition $\mathbf{Q} = \mathbf{V} \mathbf{\Lambda} \mathbf{V}^\top$ rewrite the closed form solution
- Defining $\mathbf{u}(t) = \mathbf{V}^\top(\mathbf{x}(t) - \mathbf{x}^*)$ write the update equation for the components $u_i(t)$ and explain what happens when $\lambda_i > 2/r$

4 Experiments

4.1 Gradient Descent

- Write a Matlab/Octave or python programme
- Compute a random 5×4 matrix $\mathbf{X}$
- Let $\mathbf{M} = \mathbf{X}^\top \mathbf{X}$
- Consider minimising $f(\mathbf{w}) = \frac{1}{2} \mathbf{w}^\top \mathbf{M} \mathbf{w}$

- Find the Hessian of $f(\mathbf{w})$
- Compute the eigenvalues of the Hessian
- Compute the gradient of $f(\mathbf{x})$
- From a random starting point $\mathbf{x}^{(0)}$ follow the negative gradient

$$\mathbf{x}^{(t+1)} = \mathbf{x}^{(t)} - r \nabla f(\mathbf{x}^{(t)})$$

5. For what value of r do you converge?
6. Repeat this using momentum

$$\begin{aligned} \mathbf{v}^{(t+1)} &= (1 - \gamma) \mathbf{v}^{(t)} - \gamma r \nabla f(\mathbf{x}^{(t)}) \\ \mathbf{x}^{(t+1)} &= \mathbf{x}^{(t)} + \mathbf{v}^{(t+1)} \end{aligned}$$

Using $\gamma = 0.1$ and $r = 1$

```
X = rand(5,4)
M = X'*X           % This is the Hessian
eig(M)             % Eigenvalues of momentum

w = rand(4,1)      % x0
r = 0.01
for t = 1:10
    f = w'*M*w/2    % current function value
    w = w - r*M*w;  % gradient is M*w
endfor             % I use octave

%%% Experiment with different values of r
for r = 0.05:0.05:0.5
    w = rand(4,1);
    for t = 1:100
        w = w - r*M*w;
    endfor
    [r, w'*M*w/2]    % function value after 100 iterations
endfor

%%% Using Momentum
w = rand(4,1);
v = zeros(4,1);
f = []
gamma = 0.1
for t = 1:100
    v = (1-gamma)*v - gamma*M*w;
    w = w + v;
    f(end+1) = w'*M*w/2;
endfor
plot(1:100,f)
```

5 Solutions

5.1 Removing Lag

1. Writing $a^{(t)}$ as a sum

$$\begin{aligned} a^{(1)} &= (1 - \gamma) a^{(0)} + \gamma x = \gamma x \\ a^{(2)} &= (1 - \gamma) a^{(1)} + \gamma x = (1 - \gamma) \gamma x + \gamma x \\ a^{(3)} &= (1 - \gamma) a^{(2)} + \gamma x = (1 - \gamma)^2 \gamma x + (1 - \gamma) \gamma x + \gamma x \\ a^{(t)} &= \gamma x \sum_{i=0}^{t-1} (1 - \gamma)^i \end{aligned}$$

2. • Geometric series

– As an aside we can prove the identity just multiply the geometric series by $1 - z$

$$(1 - z)(1 + z + \dots + z^{t-1}) = (1 + z + \dots + z^{t-1}) - (z + z^2 + \dots + z^t) = 1 - z^t$$

– Dividing both sides by $(1 - z)$ we obtain our identity

• Applying the identity to $a^{(t)}$ we find

$$a^{(t)} = \gamma x \frac{1 - (1 - \gamma)^t}{1 - (1 - \gamma)} = x (1 - (1 - \gamma)^t)$$

Note that as $t \rightarrow \infty$ then $a^{(t)}$ approaches x

3. Dividing through by $1 - (1 - \gamma)^t$ i.e.

$$\bar{a}^{(t)} = \frac{a^{(t)}}{1 - (1 - \gamma)^t}$$

we lose the lag

5.2 Gradient Descent in a Quadratic Minimum

1. $\mathbf{x}(t + 1) = \mathbf{x}(t) - r \mathbf{Q}(\mathbf{x}(t) - \mathbf{x}^*)$

2. $\mathbf{x}(t + 1) - \mathbf{x}^* = (\mathbf{I} - r \mathbf{Q})(\mathbf{x}(t) - \mathbf{x}^*)$

3. $\mathbf{x}(t) - \mathbf{x}^* = (\mathbf{I} - r \mathbf{Q})^t(\mathbf{x}(0) - \mathbf{x}^*)$

4. $\mathbf{x}(t) - \mathbf{x}^* = \mathbf{V}(\mathbf{I} - r \mathbf{\Lambda})^t \mathbf{V}^T(\mathbf{x}(0) - \mathbf{x}^*)$ see note below

5. $\mathbf{u}(t) = (\mathbf{I} - r \mathbf{\Lambda})^t \mathbf{u}(0)$ or $u_i(t) = (1 - r \lambda_i)^t u_i(0)$. If $\lambda_i > 2/r$ then $u_i(t)$ diverges exponentially fast. That is any component in the direction of $\mathbf{v}_i$ away from the optimum will diverge if the curvature (i.e. second derivative) in that direction exceeds $2/r$, where r is the step size.

Note that $\mathbf{I} - r \mathbf{Q} = \mathbf{V}(\mathbf{I} - r \mathbf{\Lambda}) \mathbf{V}^T$ so

$$(\mathbf{I} - r \mathbf{Q})^2 = \mathbf{V}(\mathbf{I} - r \mathbf{\Lambda}) \mathbf{V}^T \mathbf{V}(\mathbf{I} - r \mathbf{\Lambda}) \mathbf{V}^T$$

but $\mathbf{V}^T \mathbf{V} = \mathbf{I}$ as they are orthogonal matrix thus

$$(\mathbf{I} - r \mathbf{Q})^2 = \mathbf{V}(\mathbf{I} - r \mathbf{\Lambda})^2 \mathbf{V}^T$$

and similarly

$$(\mathbf{I} - r \mathbf{Q})^t = \mathbf{V}(\mathbf{I} - r \mathbf{\Lambda})^t \mathbf{V}^T.$$

Advanced Machine Learning Subsidiary Notes

Lecture 15: Constrained Optimisation

Adam Prügel-Bennett

February 9, 2024

1 Keywords

- Lagrangians, Inequalities, KKT, Linear Programming,

2 Main Points

2.1 Equality Constraints

- If we want to minimise $f(\mathbf{x})$ subject to the constraint $g(\mathbf{x}) = 0$ this is equivalent to solving the problem

$$\min_{\mathbf{x}} \max_{\alpha} \mathcal{L}(\mathbf{x}, \alpha)$$

where $\mathcal{L}(\mathbf{x}, \alpha)$ is a *Lagrangian* given by

$$\mathcal{L}(\mathbf{x}, \alpha) = f(\mathbf{x}) - \alpha g(\mathbf{x})$$

- α is a Lagrange multiplier that is determined by the joint optimisation problem
- Note that we seek a saddle-point
 - We minimise with respect to $\mathbf{x}$ and maximise with respect to α
 - We can't escape this
 - * if we multiply α by -1 we just changing the directions of the α axis but still have a saddle-point
 - * if we multiply both terms by -1 then we would end up minimising with respect to α , but maximising with respect to $\mathbf{x}$
- The solution to our problem must satisfy

$$\nabla \mathcal{L}(\mathbf{x}, \alpha) = \nabla f(\mathbf{x}) - \alpha \nabla g(\mathbf{x}) = 0, \quad \frac{\partial \mathcal{L}(\mathbf{x}, \alpha)}{\partial \alpha} = g(\mathbf{x}) = 0$$

- The second equation ensures that we sit on the constraint
 - The first equation says that the gradient of $f(\mathbf{x})$ must be perpendicular to the constraint
 - This is necessary for the solution to be a (local) minimum (i.e. we can not get an improvement by moving along the constraint)
 - There can be multiple solutions: these equations are satisfied for any local minima on the constraint
- If we have multiple equality constraints we just use multiple Lagrange multipliers

2.2 Inequality constraints

- If we are minimising $f(x)$ subject to an inequality constraint $g(x) \geq 0$ then one of two things can happen
 1. Either we have a (local) minimum of $f(x)$ that satisfies the constraint or
 2. We have a local minimum on the constraint
- We can therefore solve this problem by again using a Lagrangian

$$\mathcal{L}(x, \alpha) = f(x) - \alpha g(x)$$

with the additional constraint $\alpha \geq 0$

- If the minimum lies in a region that satisfies the constraint then we just set $\alpha = 0$ and minimise $f(x)$
- If the solution lies on the constraint we again have $\nabla f(x) = \alpha \nabla g(x)$, but now $\alpha > 0$ which means that not only is there no improving direction along the constraint, but also the negative-gradient of $f(x)$ points in the direction where $g(x)$ becomes smaller, i.e. in the region that violates the constraint
 - * note if $\alpha < 0$ we could find a better solution moving away from the constraint into the feasible region
- *Karush-Kuhn-Tucker (KKT) Conditions*
 - * For inequality constraints we require either
 1. $\alpha = 0$ and there is an unconstrained minimum in the regions $g(x) \geq 0$ or
 2. $\alpha > 0$ and the solution lies on $g(x) = 0$
- If we have multiple inequality constraints we just introduce a Lagrange multiply for each constraint with $\alpha \geq 0$

2.3 Duality

- Our problem of solving $f(x)$ subject to some constraints is known as the *primal problem*
- If our problem is sufficiently simple we can sometimes find a solution $x^*(\alpha)$ that satisfies

$$\nabla \mathcal{L}(x^*(\alpha), \alpha) = 0$$

- This leaves us with the *dual problem*

$$\max_{\alpha} \mathcal{L}(x^*(\alpha), \alpha)$$

possible with constraints on α (e.g. $\alpha_i \geq 0$) that arise from KKT conditions

- **Linear Programming**

- In linear programming we are trying to find a value of x that minimises a linear objective function $c^T x$ subject to linear constraints $Mx = b$ (and/or $Mx \geq b$)
- We can write this as a Lagrange problem

$$\mathcal{L} = c^T x - \alpha^T (Mx - b)$$

(subject to constraints $\alpha \geq 0$ if we have inequality constraints in the primal problem)

- We can rearrange the Lagrangian as

$$\mathcal{L} = \alpha^T b + (c^T - \alpha^T M) x$$

- Using the identity $\mathbf{a}^\top \mathbf{b} = \mathbf{b}^\top \mathbf{a}$ we can rewrite this as

$$\mathcal{L} = \mathbf{b}^\top \boldsymbol{\alpha} - \mathbf{x}^\top (\mathbf{M}^\top \boldsymbol{\alpha} - \mathbf{c})$$

- But we can now treat $\mathbf{x}$ as a Lagrange multiplier so we get the *dual problem*

$$\max_{\boldsymbol{\alpha}} \mathbf{b}^\top \boldsymbol{\alpha}$$

subject to the constraint

$$\mathbf{M}^\top \boldsymbol{\alpha} = \mathbf{c}$$

- If the original constraints were inequality constraints the $\alpha \geq 0$
- The dimensionality of the dual problem can sometimes be much lower than that of the primal problem making it easier to solve

• Quadratic Program

- In a quadratic program we have to minimise a quadratic loss $\mathbf{x}^\top \mathbf{Q} \mathbf{x}$ subject to linear constraints $\mathbf{M} \mathbf{x} = \mathbf{b}$ (or $\mathbf{M} \mathbf{x} \geq \mathbf{b}$)
- For there to be a unique minimum $\mathbf{Q}$ must be positive definite (which is sometimes written $\mathbf{Q} \succ 0$)
- We can write a Lagrangian

$$\mathcal{L}(\mathbf{x}, \boldsymbol{\alpha}) = \mathbf{x}^\top \mathbf{Q} \mathbf{x} - \boldsymbol{\alpha}^\top (\mathbf{M} \mathbf{x} - \mathbf{b})$$

- The solution is given by $\max_{\boldsymbol{\alpha}} \min_{\mathbf{x}} \mathcal{L}(\mathbf{x}, \boldsymbol{\alpha})$
- If the constraints are inequality constraints then $\alpha_i \geq 0$
- The minimum with respect to $\mathbf{x}$ is given by

$$\nabla_{\mathbf{x}} \mathcal{L}(\mathbf{x}, \boldsymbol{\alpha}) = 2 \mathbf{Q} \mathbf{x} + \mathbf{M}^\top \boldsymbol{\alpha} = 0$$

- So that $\mathbf{x}^* = \frac{1}{2} \mathbf{Q}^{-1} \mathbf{M}^\top \boldsymbol{\alpha}$
- Substituting this back into the Lagrangian we get the dual problem

$$\max_{\boldsymbol{\alpha}} -\frac{1}{4} \boldsymbol{\alpha}^\top \mathbf{M} \mathbf{Q}^{-1} \mathbf{M}^\top \boldsymbol{\alpha} + \boldsymbol{\alpha}^\top \mathbf{b}$$

with $\alpha_i \geq 0$ if we started with inequality constraints

- * note in the derivation that we end up with two terms proportional to $\boldsymbol{\alpha}^\top \mathbf{M} \mathbf{Q}^{-1} \mathbf{M}^\top \boldsymbol{\alpha}$ one partially cancelling the other

3 Exercises

3.1 Quadratic with a linear constraint

- Consider minimising $f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^\top \mathbf{Q} \mathbf{x}$ subject to the constraint $\mathbf{a}^\top \mathbf{x} = b$
 1. Write a Lagrangian for this problem
 2. Find the minimum of the Lagrangian with respect to $\mathbf{x}$
 3. Write down and solve the dual problem
 4. Hence write down a solution to the primal problem
- See answers, but also experiments

3.2 Saddle Point

- Strangely (for me at least) the optimum of a constrained optimisation problem is given by the saddle-point of the Lagrangian
- Consider the problems of minimising $x^2/2$ subject to the constraint $x = 1$
 1. Write down the Lagrangian
 2. Calculate the Hessian matrix (matrix of second derivatives)
 3. Compute the eigenvalues of the Hessian (show that they have different signs everywhere so there are no maxima or minima)
- See answers

4 Experiments

4.1 Quadratic with a linear constraint

- Let X be a 10×5 random matrix with elements drawn from $\mathcal{N}(0, 1)$
- Let $\mathbf{Q} = \mathbf{X}^T \mathbf{X}$
 - Check that this is positive definite
- Let $f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^T \mathbf{Q} \mathbf{x}$
- Let $\mathbf{a}$ be a random vector with 5 elements drawn from $\mathcal{N}(0, 1)$
- We want to minimise $f(\mathbf{x})$ subject to the constraint $\mathbf{a}^T \mathbf{x} = 1$
- Work out the Lagrangian, $L(\mathbf{x}, \alpha)$ for this system
- Write an iterative gradient solver that
 1. Makes steps $\mathbf{x} \leftarrow \mathbf{x} - r \nabla L(\mathbf{x}, \alpha)$
 2. Makes steps $\alpha \leftarrow \alpha + r \frac{\partial L(\mathbf{x}, \alpha)}{\partial \alpha}$
- Note you will have to tune the learning step r
- Compare the solution you find by running your algorithm until convergence with the exact result (see exercise and/or answer)

```
function [x,alpha] = optimise(Q,a,r)
    x = rand(5,1); % initialise x
    alpha = 0; % initialise alpha
    for t=1:1000
        x = x - r*(Q*x-alpha*a); % x = x - r \grad L
        alpha = alpha + r*(-(a'*x-1)); % alpha = alpha + r dL/d\alpha
    endfor
endfunction

[x,alpha] = optimise(Q,a,0.1)

alphaTheory = 1/(a'*Q*a)
xTheory = inv(Q)*a*alphaTheory
```

5 Answers

5.1 Quadratic with a linear constraint

1. The Lagrangian is given by

$$\mathbf{L}(x, \alpha) = \frac{1}{2} x^T \mathbf{Q} x - \alpha (\mathbf{a}^T x - b)$$

2. Minimising with respect to x we get

$$\nabla \mathbf{L}(x, \alpha) = \mathbf{Q} x + \alpha \mathbf{a} = 0$$

$$\text{or } x = \alpha \mathbf{Q}^{-1} \mathbf{a}$$

3. Thus the dual problem is

$$\max_{\alpha} -\frac{1}{2} \alpha^2 \mathbf{a}^T \mathbf{Q}^{-1} \mathbf{a} + \alpha b$$

- The solution to the dual problem is

$$\alpha = \frac{b}{\mathbf{a}^T \mathbf{Q}^{-1} \mathbf{a}}$$

4. Thus the solution to the primal problem is

$$x = \frac{b \mathbf{Q}^{-1} \mathbf{a}}{\mathbf{a}^T \mathbf{Q}^{-1} \mathbf{a}}$$

- Note that in most quadratic programming problems we are dealing with many inequality constraints so solving the dual problem in closed form isn't necessarily easy

5.2 Saddle Point

- Just do it

1. The Lagrangian is given by

$$\mathcal{L} = \frac{x^2}{2} - \alpha (x - 1)$$

2. The Hessian is given by

$$\mathbf{H} = \begin{pmatrix} 1 & -1 \\ -1 & 0 \end{pmatrix}$$

3. The traces $T = 1$ and the determinant $D = -1$ So that

$$\lambda = \frac{T \pm \sqrt{T^2 - 4D}}{2} = \frac{1 \pm 5}{2} = \{1.618, -0.618\}$$

If you prefer you can compute the eigenvalues numerically

- Note that whatever we do the determinant will be negative leading to a negative eigenvalue (the determinant is equal to the product of eigenvalues). This would be true if we were maximising $-x^2/2$. You can change the constraints or the objective function, but you will still get eigenvalues of different signs.

Advanced Machine Learning Subsidiary Notes

Lecture 16: Convexity

Adam Prügel-Bennett

February 9, 2024

1 Keywords

- Convex sets, convex functions, Jensen's inequality

2 Main Points

2.1 Convex Sets

- We are familiar geometrically with convex regions
- To define convexity we need to define an intermediate point $z = a x + (1 - a) y$
 - we requires $a \in [0, 1]$ (i.e. x is in the interval between 0 and 1) for z to be between x and y
 - to define convexity we only need to have addition and scalar multiplication
- We can the define convexity in a very general way: a set $\mathcal{S}$ is convex if for every pair of points $x, y \in \mathcal{S}$ and for every possible $a \in [0, 1]$ then $z = a x + (1 - a) y \in \mathcal{S}$
- We can apply convexity to sets of complicated objects
- For example the set of positive semi-definite matrices forms a convex set
 - This follows from the fact the sum of two positive semi-definite matrices is also positive semi-definite and if we multiply a positive semi-definite matrix by a positive number then the matrix is still positive semi-definite

2.2 Convex Functions

- We can define a function, $f(x)$, to be a *convex function* if for all pairs of points in the domain of the function and all $a \in [0, 1]$

$$f(a x + (1 - a) y) \leq a f(x) + (1 - a) f(y)$$

- This means that the function sits on or below the linear chord connecting any two points in the domain of the function
- The epigraph of a function is the area that lies on or above the functions
 - The epigraph of a convex function forms a convex region
 - If the epigraph of a function forms a convex region then the function is convex
- We can define *convex-down* or *concave* functions by inverting the constraint

$$f(a x + (1 - a) y) \geq a f(x) + (1 - a) f(y)$$

- for clarity I will sometimes refer to "convex" functions as *convex-up* functions
- convex-down functions have similar (but opposite) properties to convex up functions
- A function where for every pair of points and for any a such that $0 < a < 1$ (i.e. a lies strictly between 0 and 1) then if

$$f(ax + (1-a)y) < af(x) + (1-a)f(y)$$

then function is said to be *strictly convex*

- Linear functions $f(x) = ax + c$ are both convex-up and convex-down functions
 - For a function to be a strictly convex function it cannot have a linear section
- Convex functions lie on or above their tangent plane
 - The tangent plane to a function $f(x)$ at a point x_0 is the plane orthogonal to the gradient, $\nabla f(x_0)$ that goes through the point x_0
- A necessary and sufficient condition for a function to be convex is that its second derivative is non-negative or for multi-dimensional functions the Hessian is positive semi-definite
 - If the second derivative is positive (i.e. always greater than 0) or the Hessian is positive definite then the function is strictly positive
- Examples

- Convex-up Functions

- * $f(x) = x^2$ is strictly convex since $f''(x) = 2 > 0$
- * $f(x) = x^{-2}$ is strictly convex since $f''(x) = 2x^{-4} > 0$
- * $f(x) = x^4$ is convex since $f''(x) = 12x^2 \geq 0$
- * $f(x) = e^{cx}$ is strictly convex for all c as $f''(x) = c^2 e^{cx} > 0$
- * $f(x) = \|x\|^2$ is strictly convex since $\mathbf{H}(x) = \mathbf{I} \succ 0$
- * $f(x) = |x|$ is convex since for $a \in [0, 1]$ we have $|ax + (1-a)y| \leq a|x| + (1-a)|y|$ with equality only when $xy \geq 0$

- Convex-down Functions

1. $f(x) = -x^2$ is strictly convex-down since $f''(x) = -2 < 0$
 2. $f(x) = \sqrt{x}$ (for $x > 0$) is strictly convex-down since $f''(x) = -x^{-3/2}/4 < 0$
 3. $f(x) = \log(x)$ is strictly convex-down since $f''(x) = -1/x^2 < 0$
- A function $f(x)$ that is constrained to a convex domain ($x \in \mathcal{S}$, where $\mathcal{S}$ is a convex set) is convex in that domain if for all pairs $x, y \in \mathcal{S}$ and all $a \in [0, 1]$ we have

$$f(ax + (1-a)y) \leq af(x) + (1-a)f(y)$$

- This is just a more precise definition of a convex function
- Note that by limiting the domain of a function some non-convex functions may be convex over that domain
 - * e.g. $\cos(x)$ is convex in the interval $[-\pi/2, 3\pi/3]$
- A convex function constrained to lie in a convex set will still be convex
- Any combination of linear constraints will form a convex set
- Therefore convex functions restricted to satisfy linear constraints will be convex
- The minimum of a convex function will form a convex set
 - There can be no local minima

- For a strictly convex function the minimum will be unique
- The sum of convex functions will be convex
- **Linear regression**
 - The loss function of linear regression is convex

$$L(\mathbf{w}) = \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2$$

- * The Hessian is $\mathbf{X}^T\mathbf{X}$ which is positive semi-definite which is a sufficient condition for $L(\mathbf{w})$ to be convex
- Both the L_2 regulariser and the L_1 regulariser are convex
- The L_2 regulariser is strictly convex so there will be a unique solution
- Many machine learning algorithms are chosen because they involve minimising a convex function leading to a unique minimum

2.3 Jensen's Inequality

- For any convex-up function, if $\mathbf{x}$ is a random variable then

$$\mathbb{E}[f(\mathbf{x})] \geq f(\mathbb{E}[\mathbf{x}])$$

- $\mathbb{E}[\dots]$ denotes the expectation

- For any convex-down function

$$\mathbb{E}[f(\mathbf{x})] \leq f(\mathbb{E}[\mathbf{x}])$$

- These are known as *Jensen's Inequality*

- **Proof**

- We can prove this starting from the fact that $f(\mathbf{x})$ lies above the tangent plane at any point

$$f(\mathbf{x}) \geq f(\mathbf{x}^*) + (\mathbf{x} - \mathbf{x}^*)^T \nabla f(\mathbf{x}^*)$$

- This has to be true at the point $\mathbf{x}^* = \mathbb{E}[\mathbf{x}]$

$$f(\mathbf{x}) \geq f(\mathbb{E}[\mathbf{x}]) + (\mathbf{x} - \mathbb{E}[\mathbf{x}])^T \nabla f(\mathbb{E}[\mathbf{x}])$$

- Taking expectations of both sides of the equation

$$\mathbb{E}[f(\mathbf{x})] \geq f(\mathbb{E}[\mathbf{x}]) + (\mathbb{E}[\mathbf{x}] - \mathbb{E}[\mathbf{x}])^T \nabla f(\mathbb{E}[\mathbf{x}]) = f(\mathbb{E}[\mathbf{x}]) \quad \square$$

- Using Jensen's Inequality

- Consider the strictly convex function $f(\mathbf{x}) = x^2$ by Jensen's inequality

$$\mathbb{E}[x^2] \geq \mathbb{E}[x]^2$$

$$\text{* or } \mathbb{E}[x^2] - \mathbb{E}[x]^2 \geq 0$$

- the left-hand side is the variance so we see variances are non-negative
- because $f(\mathbf{x}) = x^2$ is strictly convex we only get equality where $\mathbf{x}$ doesn't vary at all
- Consider the Kullback-Liebler (KL) divergence defined for discrete probability probability distributions defined over the same range as

$$\mathcal{KL}(f\|g) = - \sum_i f_i \log\left(\frac{g_i}{f_i}\right)$$

- * This is often used to measure how different distribution are from each other
- * Note if $g_i = f_i$ then $\mathcal{KL}(f\|g) = 0$ since $\log(1) = 0$
- * Now we can use Jensen's inequality to show that $\mathcal{KL}(f\|g) \geq 0$

$$\begin{aligned}\mathcal{KL}(f\|g) &= -\sum_i f_i \log\left(\frac{g_i}{f_i}\right) = -\mathbb{E}_f\left[\log\left(\frac{g_i}{f_i}\right)\right] \\ &\geq -\log\left(\mathbb{E}_f\left[\frac{g_i}{f_i}\right]\right) \\ &= -\log\left(\sum_i f_i \frac{g_i}{f_i}\right) = -\log\left(\sum_i g_i\right) = -\log(1) = 0\end{aligned}$$

- Here we are assuming we have random variable that take values $X_i = g_i/f_i$ that occur with probability f_i
- The KL-divergence is therefore equal to $\mathbb{E}[-\log(X_i)]$
- Since $-\log(x)$ is convex up we have by Jensen's inequality that the KL-divergences is greater than or equal to $-\log(\mathbb{E}[X_i]) = -\log(\sum_i f_i X_i)$
- But $X_i = g_i/f_i$ so the KL-divergence is greater than $-\log(\sum_i g_i)$
- But g_i is a probability so $\sum_i g_i = 1$ giving us our result
- * This is known as the Gibbs' inequality after the mathematical physicist, J. Willard Gibbs, (founder of modern statistical mechanics) who first proved it
- * We often use KL-divergences when we want to choose the parameters of one probability distribution so that it approximates a second probability distribution

3 Exercises

3.1 Positive quadrant

- Prove that the set of vectors with non-negative elements form a convex set

3.2 Inverse of Convex Functions

1. Use the chain rule to compute the second derivative of $f(g(x))$
2. If $g(x) = f^{-1}(x)$ show that the second derivative of $f(g(x))$ vanishes
3. Use these results to derive an identity for the second derivative of $f^{-1}(x)$
4. Derive a condition for $f^{-1}(x)$ to be a convex-down function given that $f(x)$ is convex-up
5. Use this to show
 - (a) $\sqrt{x}$ is a convex-down function
 - (b) $\log(x)$ is a convex-down function

3.3 Cumulant Generating Function

- Here is something a bit harder (which you don't need to know)
- The cumulant generating function of a probability distribution $p(x)$ is defined as

$$G(\lambda) = \log\left(\mathbb{E}\left[e^{\lambda x}\right]\right)$$

- the expectation is over the random variable x drawn from $p(x)$

- It is called the cumulant generating function because if we take the n^{th} derivative and set λ to zero we obtain the n^{th} cumulant (i.e. $\kappa_n = G^{(n)}(0)$)
- The first cumulant is the mean, the second the variance while the third and fourth are proportional to the skewness and kurtosis
- Cumulant generating functions appear a lot when you work with probabilities, but go beyond this course
- Nevertheless let's show they are convex
 1. Find the second derivative
 2. Show that if $p(x)$ is a probability distribution then $q(x) = p(x) e^{\lambda x} / \mathbb{E}[e^{\lambda x}]$ is also a probability distribution
 3. Hence show that the cumulant generating function is convex
- See answers

4 Answers

4.1 Positive quadrant

- Let $\mathcal{P}$ be the set of vectors with non-negative elements
- If $x \in \mathcal{P}$ then if $c \geq 0$ we have $v = cx \in \mathcal{P}$ since each element of v will be non-negative (i.e. $v_i = cx_i \geq 0$)
- Also for any two vectors $x, y \in \mathcal{P}$ clearly $w = x + y \in \mathcal{P}$ since $w_i = x_i + y_i$
- Thus for any two vectors $x, y \in \mathcal{P}$ and any $a \in \{0, 1\}$ the vector $z = ax + (1 - a)y$ will be in $\mathcal{P}$

4.2 Inverse of Convex Functions

1. Taking derivatives

$$\frac{d^2 f(g(x))}{dx^2} = \frac{df'(g(x))g'(x)}{dx} = f''(g(x))(g'(x))^2 + f'(g(x))g''(x)$$

2. If $g(x) = f^{-1}(x)$ then $f(g(x)) = x$ and the second derivative vanishes
3. Using 1. and 2. we find (writing $f^{-1}(x)$ as $g(x)$)

$$g''(x) = -\frac{f''(g(x))(g'(x))^2}{f'(g(x))}$$

4. If $f(x)$ is convex then $f''(y) \geq 0$ for any y (including $y = f^{-1}(x)$) also $(g'(x))^2 \geq 0$ so for the inverse of $f(x)$ to be convex down we require $f'(f^{-1}(x)) > 0$
5. Use this to show

- (a) Let $f(x) = x^2$, so that $f''(x) = 2 > 0$ and $f'(y) = y$ which is non-negative if $y \geq 0$, but $f^{-1}(x) = \sqrt{x} > 0$ so $f'(f^{-1}(x)) \geq 0$ and consequently $\sqrt{x}$ is convex-down
- (b) Let $f(x) = \exp(x)$, so that $f''(x) = \exp(x) > 0$. But $f'(y) = \exp(y) > 0$ for all y so $f'(f^{-1}(x)) > 0$ which is sufficient to show $f^{-1}(x) = \log(x)$ is a convex-down function

4.3 Cumulant Generating Function

1. If $G(\lambda) = \log(\mathbb{E}[e^{\lambda x}])$ then

$$G'(\lambda) = \frac{\mathbb{E}[x e^{\lambda x}]}{\mathbb{E}[e^{\lambda x}]}$$

and

$$G''(\lambda) = \frac{\mathbb{E}[x^2 e^{\lambda x}]}{\mathbb{E}[e^{\lambda x}]} - \frac{\mathbb{E}[x e^{\lambda x}]^2}{\mathbb{E}[e^{\lambda x}]^2}$$

2.
 - Now if $p(x)$ is a probability distribution it will be non-negative for all x and sum or integrate to 1
 - But then $q(x) = p(x) e^{\lambda x} / \mathbb{E}[e^{\lambda x}]$ will be non-negative as $e^{\lambda x} > 0$ and $\mathbb{E}[e^{\lambda x}] > 0$ (the expectation of positive quantities will be positive)

- But

$$\int q(x) dx = \frac{1}{\mathbb{E}[e^{\lambda x}]} \int p(x) e^{\lambda x} dx = \frac{\mathbb{E}[e^{\lambda x}]}{\mathbb{E}[e^{\lambda x}]} = 1$$

- So $q(x)$ is non-negative and normalised so is a well defined probability distribution

3. Using the result of 1. and 2

$$G''(\lambda) = \frac{\mathbb{E}_p[x^2 e^{\lambda x}]}{\mathbb{E}_p[e^{\lambda x}]} - \frac{\mathbb{E}_p[x e^{\lambda x}]^2}{\mathbb{E}_p[e^{\lambda x}]^2} = \mathbb{E}_g[x^2] - \mathbb{E}_g[x]^2 \geq 0$$

- since variances are non-negative

Advanced Machine Learning Subsidiary Notes

Lecture 17: Support Vector Machines

Adam Prügel-Bennett

February 9, 2024

1 Keywords

- Support Vector Machines, maximum margins

2 Main Points

2.1 Overview

- Support vector machines are one of the most successful machine learning techniques when dealing with small data sets
 - They perform binary classification (you need many SVMs to do multi-class classification)
 - There are support vector machines for regression SVR (but I'm not sure their performance dominates other techniques)
- They are basically a perceptron that regularises itself by choosing the maximum margin separating plane
- Of course perceptrons only work with linear separable data
 - SVMs overcome this limitations in two ways
 1. It often projects the data into a high dimensional extended feature space making it more likely that the data is linear separable
 - * In the extended feature space the features will usually be non-linear functions of the original features, so we can separate data in the extended feature space that isn't necessarily linear separable in the original feature space
 2. It tolerates some errors using slack variables
 - * We need to tune a hyper-parameter (usually called C) to decide how much slack we want to tolerate
- SVMs can work with a huge number of features by solving the problem in the dual space
 - The dual problem depends on the number of constraints
 - There is one constraint for each pattern which states that the pattern should be on or outside a margin
 - The dual problem only depends on the inner (dot) product between vectors in the (extended) feature space
 - As positive semi-definite kernels can be represented as inner product between vectors in an extended feature space (which are the eigen-functions of the kernel) we often never need to explicitly compute the vectors in the extended feature space

2.2 Maximum Margins

- We consider data $\mathcal{D} = \{(\mathbf{x}_k, y_k) | k = 1, 2, \dots, m\}$ with $\mathbf{x}_k \in \mathbb{R}^p$ and $y_k \in \{-1, 1\}$
- We define a dividing hyper-plane by its orthogonal vector $\mathbf{w}$ and a threshold b saying how far from the origin it is
- The gap between the hyper-plane and a data point $(\mathbf{x}_k, y_k)$ is given by

$$d_k = y_k \left(\frac{\mathbf{w}^\top \mathbf{x}_k}{\|\mathbf{w}\|} - b \right)$$

- Note that y_k ensures the positive class is on one side of the dividing plane and the negative class on the other
- We want to constrain these gaps to be greater than or equal to a margin Δ

$$y_k \left(\frac{\mathbf{w}^\top \mathbf{x}_k}{\|\mathbf{w}\|} - b \right) \geq \Delta$$

- We can divide through by the margin size Δ giving

$$y_k (\bar{\mathbf{w}}^\top \mathbf{x}_k - \bar{b}) \geq 1$$

- where $\bar{\mathbf{w}} = \mathbf{w}/(\Delta \|\mathbf{w}\|)$ and $\bar{b} = b/\Delta$
- We note that

$$\frac{1}{2} \|\bar{\mathbf{w}}\|^2 = \frac{1}{2\Delta^2}$$

- thus if we minimise $\|\bar{\mathbf{w}}\|^2/2$ subject to the constraint this is equivalent to maximising Δ
 - the factor of a half is just for convenience
 - we minimise $\|\bar{\mathbf{w}}\|^2$ rather than $\|\bar{\mathbf{w}}\|$ again for convenience
 - we will just drop the bar notation (i.e. we talk about $\mathbf{w}$ and b rather than $\bar{\mathbf{w}}$ and $\bar{b}$): after all what's in a name? (More seriously it has the same optimum whatever we call it)

- **Soft Margins**

- So far we have constrained all the data points to lie outside the margin (this is known as *hard margins*)
 - We can instead allow some data points to lie within the margin (or even be misclassified)
 - * This is known as a *soft margin SVM*
 - We do this by introducing slack variables s_k so that our constraints become

$$y_k \left(\frac{\mathbf{w}^\top \mathbf{x}_k}{\|\mathbf{w}\|} - b \right) \geq 1 - s_k$$

(I've dropped the bars for simplicity)

- We punish the slack variables by adding $C \sum_{k=1}^m s_k$ to the objective function
 - We also constrain the slack variables so that $s_k \geq 0$ (so we don't get a reward if $s_k < 0$)
- Optimisation problem

- Our optimisation problem becomes

$$\min_{\mathbf{w}, b, \mathbf{s}} \quad \frac{\|\mathbf{w}\|^2}{2} + C \sum_{k=1}^m s_k$$

subject to

$$y_k \left(\frac{\mathbf{w}^\top \mathbf{x}_k}{\|\mathbf{w}\|} - b \right) \geq 1 - s_k \quad \text{and} \quad s_k \geq 0$$

for $k = 1, 2, \dots, m$

- We can write this as a Lagrange problem

$$\min_{\mathbf{w}, b, \mathbf{s}} \max_{\alpha, \beta} \quad \mathcal{L}(\mathbf{w}, b, \mathbf{s}, \alpha, \beta)$$

where

$$\mathcal{L}(\mathbf{w}, b, \mathbf{s}, \alpha, \beta) = \frac{\|\mathbf{w}\|^2}{2} + C \sum_{k=1}^m s_k - \sum_{k=1}^m \alpha_k (y_k (\mathbf{w}^\top \mathbf{x}_k - b) - 1 + s_k) - \sum_{k=1}^m \beta_k s_k$$

- * α_k and β_k are Lagrange multipliers
 - α_k ensures the margin condition
 - β_k ensures the non-negativity of the slack variables
- * As they enforce inequality constraints we require they are non-negative

2.3 Extended Feature Space

- To help linear separability we can map our inputs to a high-dimensional feature space

$$\mathbf{x} \rightarrow \phi(\mathbf{x})$$

- Note in the slides I wrote this as $\vec{\phi}(\mathbf{x})$ to make it clear that this is a different feature space (I won't do it here as it is a slightly ugly way of writing vectors)
- Note this is a function of the original feature vector meaning that if I change $\mathbf{x}$ I will get a different vector $\phi(\mathbf{x})$
- Nevertheless $\phi(\mathbf{x})$ is just a p' dimensional vector
- We call the space of $\phi(\mathbf{x})$ the *extended feature space*
- We can define this extended feature mapping explicitly (e.g. we might decide that $\phi_1(\mathbf{x}) = x_1^2$, $\phi_2(\mathbf{x}) = x_1 x_2$, etc.)
- More often this feature mapping is defined implicitly through a kernel function (more of that later)
- We usually choose the dimensionality, p' , of the extended feature space to be much larger than that of the original feature space
- Usually we would be scared of this because it can lead to over-fitting
- However, because of choosing the maximal-margin hyper-plane this regularises the problem so strongly that we get good generalisation despite working in an enormous dimensional space
- The Lagrange problem in this extended feature space is

$$\mathcal{L}(\mathbf{w}, b, \mathbf{s}, \alpha, \beta) = \frac{\|\mathbf{w}\|^2}{2} + C \sum_{k=1}^m s_k - \sum_{k=1}^m \alpha_k (y_k (\mathbf{w}^\top \phi(\mathbf{x}_k) - b) - 1 + s_k) - \sum_{k=1}^m \beta_k s_k$$

- Note that the weight vector, $\mathbf{w}$, is now in the extended weight space

2.4 Dual Problem

- To obtain the dual problem we compute the minimum of the Lagrangian with respect to w , b and s

- Weight minimisation

$$\nabla_w \mathcal{L} = w - \sum_{k=1}^m \alpha_k y_k \phi(x_k) = 0$$

$$* \text{ So that } w = \sum_{k=1}^m \alpha_k y_k \phi(x_k)$$

- Threshold minimisation

$$\frac{\partial \mathcal{L}}{\partial b} = - \sum_{k=1}^m \alpha_k y_k = 0$$

- * This gives us a global constraint on the Lagrange multipliers α_k

- Slack variable minimisation

$$\frac{\partial \mathcal{L}}{\partial s_k} = C - \alpha_k - \beta_k = 0$$

$$* \text{ Thus } \alpha_k = C - \beta_k$$

- * When substituting back into the Lagrangian all the terms proportional to s_k cancel

- * But $\beta_k \geq 0$ so this implies $\alpha_k \leq C$ (recall that $\alpha_k \geq 0$ from the KKT condition)

- Substituting the optimal values of w , b and S into the Lagrangian we obtain the dual problem

$$\max_{\alpha} -\frac{1}{2} \sum_{k,l=0}^m \alpha_k \alpha_l y_k y_l \phi^T(x_k) \phi(x_l) + \sum_{k=0}^m \alpha_k$$

subject to the conditions

$$\sum_{k=1}^m \alpha_k y_k = 0, \quad \forall_k 0 \leq \alpha_k \leq C$$

- Note that a large number of terms vanished (you really need to go through this carefully to see that this happens)
- Amazingly adding slack variables is equivalent to preventing α_k becoming larger than C
- the dual problem is another quadratic programming problem but involves the m Lagrange multipliers α_k rather than the p' weights
- importantly the dual problem depends only on $\phi(x_k)$ through the inner product $\phi^T(x_k) \phi(x_l)$

2.5 Working in the extended feature space

- If we work in the original feature space then the dual problem depends on the inner (dot) product $x_k^T x_l$
 - this is known as a linear SVM
 - quite often a linear SVM gives the best performance
 - sometimes it will pay to solve the primal problem rather than dual problem
 - however we can work with very high dimensional feature vectors
 - * this is useful when we work with text where our input vectors are naturally high dimensional
 - * In this case we would want to solve the dual problem

- We can explicitly define our extended feature vectors
 - this has the disadvantage that they could be expensive to compute, but for some applications there may be smart ways of doing this
- **Kernel Trick**
 - There is however a really nice trick
 - Any positive definite kernel, $K(\mathbf{x}, \mathbf{y})$, can be decomposed as

$$K(\mathbf{x}, \mathbf{y}) = \phi^\top(\mathbf{x}) \phi(\mathbf{y})$$

(we will talk much more about this in the next lecture)

- obviously the functions $\phi_i(\mathbf{x})$ that form the elements of the vector $\phi(\mathbf{x})$ depend on the kernel we choose
- However, as the dual problem only depends on the inner product $\phi^\top(\mathbf{x}) \phi(\mathbf{y})$ if we choose these so that $\phi^\top(\mathbf{x}) \phi(\mathbf{y}) = K(\mathbf{x}, \mathbf{y})$ then we never have to explicitly compute $\phi(\mathbf{x})$
- Our dual problem in this case is

$$\max_{\alpha} -\frac{1}{2} \sum_{k,l=0}^m \alpha_k \alpha_l y_k y_l K(\mathbf{x}_k, \mathbf{x}_l) + \sum_{k=0}^m \alpha_k$$

- In using an SVM we have to determine which side of the dividing plane a new data point, $\mathbf{x}$, would lie

- * Our prediction would be

$$\text{sgn}(\mathbf{w}^\top \phi(\mathbf{x}) - b)$$

- the function $\text{sgn}(\cdot)$ outputs ± 1 depending on the sign of the argument

- * But our optimal weight vector is given by $\mathbf{w} = \sum_{k=1}^m \alpha_k y_k \phi(\mathbf{x}_k)$

- * Thus our prediction will be

$$\text{sgn}\left(\sum_{k=1}^m \alpha_k y_k \phi^\top(\mathbf{x}_k) \phi(\mathbf{x}) - b\right) = \text{sgn}\left(\sum_{k=1}^m \alpha_k y_k K(\mathbf{x}_k, \mathbf{x}) - b\right)$$

- * Thus we don't need to compute the weight or the extended feature vectors

2.6 Practical Consideration

- To get SVMs to work in practice requires a lot of tuning
- Firstly you need to normalise the inputs
- You might need to **balance** your data set
 - SVMs perform poorly on problems where the training set has many more examples of one class than the other
 - You can balance the data set by ignoring examples from the large class or
 - Increasing the size of minority class by duplicating examples
 - Sometimes you can use data augmentation on the minority class
 - This is only a problem where the data set is strongly unbalanced
- You need to choose C (this can vary by orders of magnitudes)

- Typically found by exhaustive search start at 2^{-5} and doubling until you reach 2^{15}
- You decide on what to use by testing on a validation set
- You also have to choose the right kernel
 - This could be no kernel (linear SVM)
 - A polynomial kernel (you have to try different degrees)
 - A radial basis kernel
 - Sometimes you use a special designed kernel for the problem
 - Kernels also come with hyper-parameters (often called γ) which you also have to tune
- It is a lot of work but for many problems (particularly with small training set) it often leads to state-of-the-art performance
- **Multi-Class Classification**
 - Natively SVMs only perform binary classification
 - If you have multiple classes you need multiple SVMs
 - People use two strategies
 - * *One-versus-all*: for each class train a separate SVM using all other classes as negative examples (but see note on balancing)
 - * *All-pairs*: Train a set of SVM between all pairs of classes (this is obviously expensive when you have large classes)
 - You then have to get your SVMs to vote in some way to determine the true class

3 Experiments

3.1 SVM in Practice

- SVMs are hard to code (they need a quadratic programming solver which are complicate)
- However, there a lots of good implementations out there so have a go
- Stolen from the web...

```
#Import scikit-learn dataset library
from sklearn import datasets

#Load dataset
cancer = datasets.load_breast_cancer()
# print the names of the 13 features
print("Features: ", cancer.feature_names)

# print the label type of cancer('malignant' 'benign')
print("Labels: ", cancer.target_names)

# print data(feature)shape
cancer.data.shape

# Import train_test_split function
from sklearn.model_selection import train_test_split

# Split dataset into training set and test set
X_train, X_test, y_train, y_test =
```

```

train_test_split(cancer.data, cancer.target, test_size=0.3,random_state=109)
                # 70% training and 30% test

#Import svm model
from sklearn import svm

#Create a svm Classifier
clf = svm.SVC(kernel='linear') # Linear Kernel

#Train the model using the training sets
clf.fit(X_train, y_train)

#Predict the response for test dataset
y_pred = clf.predict(X_test)

#Import scikit-learn metrics module for accuracy calculation
from sklearn import metrics

# Model Accuracy: how often is the classifier correct?
print("Accuracy:",metrics.accuracy_score(y_test, y_pred))

# Model Precision: what percentage of positive tuples are labeled as such?
print("Precision:",metrics.precision_score(y_test, y_pred))

# Model Recall: what percentage of positive tuples are labelled as such?
print("Recall:",metrics.recall_score(y_test, y_pred))

```

Advanced Machine Learning Subsidiary Notes

Lecture 18: Kernel Trick

Adam Prügel-Bennett

February 9, 2024

1 Keywords

- The Kernel Trick, SVMs, Regression

2 Main Points

2.1 Kernels

- SVM kernels are symmetric functions of two variable that can be factorised as an inner-product

$$K(\mathbf{x}, \mathbf{y}) = \phi(\mathbf{x})^\top \phi(\mathbf{y}) = \sum_{i=1}^{p'} \phi_i(\mathbf{x}) \phi_i(\mathbf{y})$$

- $\phi(\mathbf{x})$ are vectors whose elements, $\phi_i(\mathbf{x})$, are real-valued functions of the features $\mathbf{x}$ (every different feature will correspond to a different vector $\phi(\mathbf{x})$)
- p' is the dimensionality of the extended feature space which might be infinite
- An immediate consequence of this is that the vectors are positive semi-definite
 - * This follows because for any function $f(\mathbf{x})$ the quadratic form is non-negative

$$\begin{aligned} \iint f(\mathbf{x}) K(\mathbf{x}, \mathbf{y}) f(\mathbf{y}) d\mathbf{x} d\mathbf{y} &= \iint f(\mathbf{x}) \phi(\mathbf{x})^\top \phi(\mathbf{y}) f(\mathbf{y}) d\mathbf{x} d\mathbf{y} \\ &= \sum_{i=1}^{p'} \left(\int f(\mathbf{x}) \phi_i(\mathbf{x}) d\mathbf{x} \right)^2 \geq 0 \end{aligned}$$

- **Eigenfunctions and Mercer's Theorem**

- Kernel functions play the same role for functions as matrices do for normal vectors
 - * that is they describe general linear transformations
 - * for a function $f(\mathbf{x})$ the argument $\mathbf{x}$ can be seen as an index just like i is the index of element v_i of a vector $\mathbf{v}$
 - * we will consider only symmetric kernels (that is, where $K(\mathbf{x}, \mathbf{y}) = K(\mathbf{y}, \mathbf{x})$)
 - * these play a similar role as symmetric matrices
- Eigensystems for Kernels
 - * $\psi(\mathbf{y})$ is said to be an eigenfunction of a kernel functions if

$$\int K(\mathbf{x}, \mathbf{y}) \psi(\mathbf{y}) d\mathbf{y} = \lambda \psi(\mathbf{x})$$

- * In an analogy to the eigen-decomposition of a symmetric matrix we can define the eigen-decomposition of a symmetric kernel function

$$K(\mathbf{x}, \mathbf{y}) = \sum_i \lambda_i \psi_i(\mathbf{x}) \psi_i(\mathbf{y})$$

- * This is known as **Mercer's Theorem**
- * We proved the decomposition for matrices
 - the difficult part in the proof is that you need the eigenvectors to span the vector space
 - this is intuitively obvious if there are n orthogonal eigenvectors in an n -dimensional space
 - it is harder in functions spaces and you need to define the vector space you are modelling (e.g. L_2)
 - if you assume that the set of eigenvectors span the function space then the rest of the proof is the same as for matrices
 - don't worry if you don't understand this it is enough to remember Mercer's Theorem
- * Mercer's Theorem holds for any symmetric kernel function (it does not have to be positive semi-definite)
- * But if $K(\mathbf{x}, \mathbf{y})$ are positive semi-definite then there exist real functions $\phi_i(\mathbf{x}) = \sqrt{\lambda_i} \psi_i(\mathbf{x})$ such that

$$K(\mathbf{x}, \mathbf{y}) = \sum_i \lambda_i \psi_i(\mathbf{x}) \psi_i(\mathbf{y}) = \sum_i \phi_i(\mathbf{x}) \phi_i(\mathbf{y})$$

- if $K(\mathbf{x}, \mathbf{y})$ was not positive semi-definite then some of the eigenvalues would be negative and the functions $\psi_i(\mathbf{x})$ would not be real-valued

2.2 SVM Kernels

- SVM Kernels are positive semi-definite symmetric functions
 - There are four necessary and sufficient conditions that hold for any positive semi-definite kernel
 1. All their eigenvalues are non-negative (i.e. either zero or positive)
 2. They can be decomposed as

$$K(\mathbf{x}, \mathbf{y}) = \phi(\mathbf{x})^\top \phi(\mathbf{y}) = \sum_{i=1}^{p'} \phi_i(\mathbf{x}) \phi_i(\mathbf{y})$$

where $\phi_i(\mathbf{x})$ are real-valued functions

3. Their quadratic form with any function $f(\mathbf{x})$ is non-negative
4. For any set of points $\{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n\}$ the matrix

$$\mathbf{K} = \begin{pmatrix} K(\mathbf{x}_1, \mathbf{x}_1) & K(\mathbf{x}_1, \mathbf{x}_2) & \cdots & K(\mathbf{x}_1, \mathbf{x}_n) \\ K(\mathbf{x}_2, \mathbf{x}_1) & K(\mathbf{x}_2, \mathbf{x}_2) & \cdots & K(\mathbf{x}_2, \mathbf{x}_n) \\ \vdots & \vdots & \ddots & \vdots \\ K(\mathbf{x}_n, \mathbf{x}_1) & K(\mathbf{x}_n, \mathbf{x}_2) & \cdots & K(\mathbf{x}_n, \mathbf{x}_n) \end{pmatrix}$$

is a positive semi-definite matrix

- * such matrices are known as **Gram matrices**
- * I didn't mention this in the lecture and won't use this property, but for completeness I mention it here (you won't be tested on it)
- * the proof that this is a necessary condition follows rather simply from the fact that if we define a matrix Φ with elements $\Phi_{ik} = \phi_i(\mathbf{x}_k)$ then $\mathbf{K} = \Phi^\top \Phi$ and we have seen many times any such matrix is positive semi-definite
- Recall from the previous lecture that any kernel function that allows a decomposition in terms of positive functions can be used an SVM where we can use the kernel trick
 - If we don't use positive semi-definite kernels then our "distances" (used in computing margins) are no-longer proper distances and can be negative (invalidating everything)

2.3 Constructing SVM Kernel

- Most functions of two variable won't be positive semi-definite
- Given a function of two variables it is hard to determine if it is positive-semi definite (none of the definitions are particularly easy to use)
- However we can use simple rules to build positive-semi definite (PSD) kernels from other positive semi-definite kernels

1. Our starting point is to note the inner produce $\langle \mathbf{x}, \mathbf{y} \rangle = \mathbf{x}^T \mathbf{y}$ is positive semi-definite
 - as an aside we don't necessarily need to use normal vectors as our features so long as we objects with an inner-product

2. Adding PSD kernels

if $K_1(\mathbf{x}, \mathbf{y})$ and $K_2(\mathbf{x}, \mathbf{y})$ are PSD kernels then so is $K_3(\mathbf{x}, \mathbf{y}) = K_1(\mathbf{x}, \mathbf{y}) + K_2(\mathbf{x}, \mathbf{y})$

- To prove this we can use the property that PSD have non-negative quadratic form

$$\begin{aligned} Q &= \int f(\mathbf{x}) K_3(\mathbf{x}, \mathbf{y}) f(\mathbf{y}) d\mathbf{x} d\mathbf{y} \\ &= \int f(\mathbf{x}) (K_1(\mathbf{x}, \mathbf{y}) + K_2(\mathbf{x}, \mathbf{y})) f(\mathbf{y}) d\mathbf{x} d\mathbf{y} \\ &= \int f(\mathbf{x}) K_1(\mathbf{x}, \mathbf{y}) f(\mathbf{y}) d\mathbf{x} d\mathbf{y} + \int f(\mathbf{x}) K_2(\mathbf{x}, \mathbf{y}) f(\mathbf{y}) d\mathbf{x} d\mathbf{y} \geq 0 \end{aligned}$$

3. Multiplication by a positive scalar

if $K_1(\mathbf{x}, \mathbf{y})$ is a PSD kernels and $c > 0$ then so is $K_3(\mathbf{x}, \mathbf{y}) = c K_1(\mathbf{x}, \mathbf{y})$

- We can prove this in a similar way to the last proof

4. Multiply PSD kernels

if $K_1(\mathbf{x}, \mathbf{y})$ and $K_2(\mathbf{x}, \mathbf{y})$ are PSD kernels then so is $K_3(\mathbf{x}, \mathbf{y}) = K_1(\mathbf{x}, \mathbf{y}) K_2(\mathbf{x}, \mathbf{y})$

- This is easy to prove using the decomposition of PSD to inner products

$$K_3(\mathbf{x}, \mathbf{y}) = \sum_{i,j} \phi_i^1(\mathbf{x}) \phi_i^1(\mathbf{y}) \phi_j^2(\mathbf{x}) \phi_j^2(\mathbf{y}) = \sum_{i,j} \phi_{ij}^3(\mathbf{x}) \phi_{ij}^3(\mathbf{y})$$

where $\phi_{ij}^3(\mathbf{x}) = \phi_i^1(\mathbf{x}) \phi_j^2(\mathbf{x})$

* this (double) sum we can treat as an inner-product

* it is easy to show that the quadratic form with any function $f(\mathbf{x})$ is non-negative

5. Powers of PSD kernels

if $K_1(\mathbf{x}, \mathbf{y})$ is a PSD kernels then so is $K_1^n(\mathbf{x}, \mathbf{y})$ for any natural number n

- Since the product of any two PSD kernels are PSD then the square of a PSD kernel is PSD
- But by an inductive argument this holds for any integer power

6. Exponential of PSD kernels

The exponential of a PSD kernel is also a PSD kernel

- convergent Taylor expansions allow us to approximate a function to any degree of accuracy
- often Taylor expansions aren't everywhere convergent (so we have to be careful)
- but Taylor expansions of exponentials are everywhere convergent
- further Taylor expanding an exponential of a PSD kernel involves a sum of PSD kernels

$$e^{K(\mathbf{x}, \mathbf{y})} = \sum_i \frac{1}{i!} K^i(\mathbf{x}, \mathbf{y}) = 1 + K(\mathbf{x}, \mathbf{y}) + \frac{1}{2} K^2(\mathbf{x}, \mathbf{y}) + \dots$$

* each term is a PSD kernel

- Using these properties we see that $K(\mathbf{x}, \mathbf{y}) = e^{-\gamma \|\mathbf{x} - \mathbf{y}\|^2}$ is a PSD kernel if $\gamma > 0$
 - Since
$$e^{-\gamma \|\mathbf{x} - \mathbf{y}\|^2} = e^{-\gamma \|\mathbf{x}\|^2} e^{2\gamma \mathbf{x}^\top \mathbf{y}} e^{-\gamma \|\mathbf{y}\|^2}$$
 - But $e^{-\gamma \|\mathbf{x}\|^2}$ and $e^{-\gamma \|\mathbf{y}\|^2}$ are just positive constants
 - $\mathbf{x}^\top \mathbf{y}$ is an inner product so a PSD kernel
 - Since $2\gamma > 0$ then $2\gamma \mathbf{x}^\top \mathbf{y}$ is a PSD kernel
 - But then so is $e^{2\gamma \mathbf{x}^\top \mathbf{y}}$
 - This kernel is known as the *radial basis function* or *RBF* or *Gaussian kernel*
 - It has a hyper-parameter, γ that determines the length scale in the problem (or rather inverse-length scale)
 - this is a very important kernel as it often (but certainly not always) gives good performance (if γ is appropriately chosen)
- **Non-numerical Kernels**
 - When SVMs were fashionable there was a whole industry of researchers finding clever kernels
 - When working with language or trees or graphs it paid to create bespoke kernels for these structures
 - Typically these would all be built up from inner-products
 - Using clever algorithms you can build very clever kernels functions
 - One down side of SVM kernels is they don't naturally capture prior knowledge about the problem being tackled
 - * a clever work around is to build SVMs based on other learning machines that are trained the problem
 - * an example of this is the use of *Fisher kernels* based on Fisher information

2.4 Beyond SVMs

- There are a lot of other kernel based learning machines
- Many of these use constraints
- They often involve linear operations between vectors where the optimum depends on the inner-product of vectors
 - thus we can use the kernel trick
- SVR are support vector machines for regression
 - here we try to find a dividing plane so that all points lie within a margin (the exact opposite of what we had)
 - We can introduce slack variables to allow some points to lie outside the margin
 - * the slack variables must be non-negative
 - * we can use a linear punishment s_i or quadratic punishment s_i^2
- We can also do *kernel ridge regression*

$$\min_{\mathbf{w}} \lambda \|\mathbf{w}\|^2 + \sum_i (y_i - \mathbf{w}^\top \phi(\mathbf{x}_i))^2$$

- $\|\mathbf{w}\|^2$ is a regularisation term

- The weights must lie in the space spanned by the set of extended feature vectors $\{\phi(\mathbf{x}_k) | k = 1, 2, \dots, m\}$

- Thus we can write

$$\mathbf{w} = \sum_i \alpha_i \phi(\mathbf{x}_i)$$

- * Note that here α_i are just parameters; they are not Lagrange multipliers and they can be negative

- Substituting this into the objective function for ridge regression we get a quadratic optimisation problem in α that just depends on the inner products $\phi^\top(\mathbf{x}_i) \phi(\mathbf{x}_j)$
- We can use the kernel trick

- *Kernel PCA*

- For kernel PCA we map features into an external feature space
- We then use the dual form of PCA (which we've done in an earlier lecture)
- This allows us to find non-linear manifolds where the data varies

- *Kernel Canonical Correlation Analysis*

- Canonical correlation analysis finds correlations between datasets
- The linear form is a bit naff
- But the kernel form can give nice results

- *Gaussian Processes*

- Gaussian Processes also use kernels
- They are a bit different to other kernel methods
 - * we don't think of the working in an extended feature space
 - * but they are PSD
- They are one of the most successful methods for doing regression
- We will look at them later

3 Exercises

3.1 Quadratic Kernels

- Show that the kernel function $K(\mathbf{x}, \mathbf{y}) = \phi^\top(\mathbf{x}) \phi(\mathbf{y})$, where

$$\phi(\mathbf{x}) = (x_1^2, x_2^2, x_3^2, \sqrt{2}x_1x_2, \sqrt{2}x_1x_3, \sqrt{2}x_2x_3)$$

can be written as $(\mathbf{x}^\top \mathbf{y})^2$ if $\mathbf{x}$ and $\mathbf{y}$ are vectors of length 3.

- Answer below

3.2 Kernel Ridge Regression

- Work out the details for kernel ridge regression
- Have a go at implementing kernel ridge regression on a real data set
- I'll leave you to work this out

4 Experiments

4.1 Gram Matrix

- Generate ten random vectors $(\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_{10})$ where $\mathbf{x}_k \in \mathbb{R}^5$
- Compute the Gram matrix $\mathbf{K}$ with components

$$K_{kl} = K(\mathbf{x}_k, \mathbf{x}_l) = e^{-\|\mathbf{x}_k - \mathbf{x}_l\|^2}$$

- Show that $\mathbf{K}$ is positive definite by computing its eigenvalues

```
n = 10;
X=randn(n,5);           % matrix of vectors
K = zeros(n,n);         % define holder for Gram
for i = 1:n
    x = X(i,:);
    for j = 1:n
        y = X(j,:);
        K(i,j) = exp(-norm(x-y)^2); % define elements of Gram matrix
    endfor
endfor

K           % Gram matrix
eig(K)      % Eigenvalues should all be non-negative
```

5 Answers

5.1 Quadratic Kernel

- This is just straightforward algebra

$$\begin{aligned}\phi^T(\mathbf{x})\phi(\mathbf{y}) &= x_1^2 y_1^2 + x_2^2 y_2^2 + x_3^2 y_3^2 + 2x_1 x_2 y_1 y_2 + 2x_1 x_3 y_1 y_3 + 2x_2 x_3 y_2 y_3 \\ &= (x_1 y_1 + x_2 y_2 + x_3 y_3)^2 = (\mathbf{x}^T \mathbf{y})^2\end{aligned}$$

- In the lecture notes we did the 2-d case
- Note that the more general polynomial kernel is

$$K_p(\mathbf{x}, \mathbf{y}) = (1 + \mathbf{x}^T \mathbf{y})^p$$

- this is more commonly used as it incorporates the lower dimensional polynomial kernels

Advanced Machine Learning Subsidiary Notes

Lecture 19: Wasserstein GANs

Adam Prügel-Bennett

February 9, 2024

1 Keywords

- GANs, Wasserstein distance, Duality, WGANs

2 Main Points

2.1 Generative Adversarial Networks GANs

- GANs are generative models
- They are often used for generating images or text
 - we will consider images just to be concrete but nothing really changes if we use text
- The task of the GAN is to generate images from the same distribution as those of a dataset $\mathcal{D}$
- GANs use two networks
 1. *A generator network*
 - It is fed a random vector $z \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
 - They are usually networks made with fully connected and deconvolution layers with weights w_G
 - They generate an "image" $\hat{x} = G(z, w_G)$
 - We train the weights to deceive the discriminator network
 2. *A discriminator network*
 - They receive either
 - * a random sample from $\mathcal{D}$ or
 - * an image $\hat{x}$ generated by the generator seeded with a random vector z
 - They are trained using backpropagation where the target output is
 - * 1 for the real image from $\mathcal{D}$ or
 - * 0 if the input is from the generator
 - They are usually CNN networks
- The generator weights are also learned by back-propagation through both the discriminator and generator networks where the target is for the discriminator for output 1
 - the discriminator weights aren't changed
 - this is opposite to the loss for the discriminator
 - it is fed the information about how to fool the discriminator (i.e. how to change the elements of $\hat{x}$ to maximise the output of the discriminator)

- Hopefully over time the generator produces image more like those from $\mathcal{D}$

- **Problems with GANs**

- GANs are notoriously hard to train
- The training of the discriminator and generator can become decoupled
- For example, the discriminator can become so good that any local change of $\hat{x}$ doesn't fool the discriminator
 - * but this means there is no gradient to direct the learning of the generator
- To overcome this we consider a very different approach

2.2 Wasserstein Distance

- The Big Picture

- We consider minimising the distance between the distribution of images generated by the generator (that is, the distribution of $\hat{x} = G(z, w_G)$ where $z \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$) and the distribution of real images (where we consider $\mathcal{D}$ to be a set of samples drawn from this distribution)
- How do we measure distances between probability distributions?
- One of the most common methods is to use the KL-divergence

$$\text{KL}(p||q) = \int p(\mathbf{x}) \log\left(\frac{p(\mathbf{x})}{q(\mathbf{y})}\right) d\mathbf{x}$$

- * Relatively nice to compute
- * Not a true distance (but that doesn't bother us)
- * Unfortunately it can get very large even when the probability distributions are relatively close together

- **Earth-Moving Distance**

- A very natural distance measure is the minimum distances you have to move the probability mass in one distribution $p(\mathbf{x})$ to make it identical to a second distribution $q(\mathbf{x})$
- This is also known as the *Wasserstein* distance
- Although conceptually straightforward it is a bit nasty to compute
- **Optimal Transport Policy**

- * We start from a transport policy $\gamma(\mathbf{x}, \mathbf{y})$ that tells us how much probability mass (or density) we need to move from probability distribution p at point $\mathbf{x}$ to probability distribution q at point $\mathbf{y}$
- * As we start with a distribution $p(\mathbf{x})$ we need

$$\int \gamma(\mathbf{x}, \mathbf{y}) d\mathbf{y} = p(\mathbf{x})$$

- * As we end with a distribution $q(\mathbf{x})$ we require

$$\int \gamma(\mathbf{x}, \mathbf{y}) d\mathbf{x} = q(\mathbf{y})$$

- * Note that $\gamma(\mathbf{x}, \mathbf{y})$ looks like a joint probability distribution
 - It is non-negative
 - Integrating over $\mathbf{x}$ and $\mathbf{y}$ we get 1

- * We denote the set of probability distributions that satisfy these constraints $\Lambda(p, q)$
- * The cost of a particular transport policy is

$$C(\gamma) = \iint d(\mathbf{x}, \mathbf{y}) \gamma(\mathbf{x}, \mathbf{y}) d\mathbf{x} d\mathbf{y} = \mathbb{E}_{\gamma}[d(\mathbf{x}, \mathbf{y})]$$

since $\gamma(\mathbf{x}, \mathbf{y})$ is the amount of probability mass we move and $d(\mathbf{x}, \mathbf{y})$ is the distance we move it

- * The optimal transport policy is the distribution $\gamma \in \Lambda(p, q)$ with the minimum cost
- * The cost of the optimal transport policy is the Wasserstein distance

$$W(p, q) = \min_{\gamma \in \Lambda(p, q)} \mathbb{E}_{\gamma}[d(\mathbf{x}, \mathbf{y})]$$

- * For high dimensional probability distributions finding the optimal transport policy using this definition is impractical

- Linear Programming

- * Computing the Wasserstein distance is a linear programming problem
- * We want to choose $\gamma(\mathbf{x}, \mathbf{y})$ to minimise a linear objective function $C(\gamma)$ subject to linear constraints
- * We can write this as a Lagrange problem

$$\begin{aligned} \mathcal{L} = \iint d(\mathbf{x}, \mathbf{y}) \gamma(\mathbf{x}, \mathbf{y}) d\mathbf{x} d\mathbf{y} &- \int \alpha(\mathbf{x}) \left(\int \gamma(\mathbf{x}, \mathbf{y}) d\mathbf{y} - p(\mathbf{x}) \right) d\mathbf{x} \\ &- \int \beta(\mathbf{y}) \left(\int \gamma(\mathbf{x}, \mathbf{y}) d\mathbf{x} - q(\mathbf{y}) \right) d\mathbf{y} \end{aligned}$$

subject to $\gamma(\mathbf{x}, \mathbf{y}) \geq 0$

- $\alpha(\mathbf{x})$ and $\beta(\mathbf{y})$ are Lagrange multiplier functions
- This looks strange because we are used to optimise vectors in Linear programming but here we optimise functions
- We can discretise the function and we would get a vector
- But functions form a vector space so we can define a linear programme for functions

* Dual Form

- We can rearrange the Lagrangian as

$$\mathcal{L} = \int \alpha(\mathbf{x}) p(\mathbf{x}) d\mathbf{x} + \int \beta(\mathbf{y}) q(\mathbf{y}) d\mathbf{y} - \iint \gamma(\mathbf{x}, \mathbf{y}) (\alpha(\mathbf{x}) + \beta(\mathbf{y}) - d(\mathbf{x}, \mathbf{y})) d\mathbf{x} d\mathbf{y}$$

- Now we can interpret $\gamma(\mathbf{x}, \mathbf{y})$ as a Lagrange multiplier function so that the dual problem is

$$\max_{\alpha(\mathbf{x}), \beta(\mathbf{x})} \int \alpha(\mathbf{x}) p(\mathbf{x}) d\mathbf{x} + \int \beta(\mathbf{y}) q(\mathbf{y}) d\mathbf{y}$$

subject to

$$\alpha(\mathbf{x}) + \beta(\mathbf{y}) \leq d(\mathbf{x}, \mathbf{y})$$

- note this is an inequality constraint because $\gamma(\mathbf{x}, \mathbf{y}) \geq 0$
- But this has to be true when $\mathbf{x} = \mathbf{y}$ so

$$\alpha(\mathbf{x}) + \beta(\mathbf{x}) \leq d(\mathbf{x}, \mathbf{x}) = 0$$

- Thus $\beta(\mathbf{x}) = -\alpha(\mathbf{x}) - \epsilon(\mathbf{x})$ where $\epsilon(\mathbf{x}) \geq 0$
- Our objective function becomes

$$\int \alpha(\mathbf{x}) p(\mathbf{x}) d\mathbf{x} + \int \beta(\mathbf{y}) q(\mathbf{y}) d\mathbf{y} = \int \alpha(\mathbf{x}) (p(\mathbf{x}) - q(\mathbf{x})) d\mathbf{x} - \int q(\mathbf{x}) \epsilon(\mathbf{x}) d\mathbf{x}$$

- But this is clearly maximised when $\epsilon(\mathbf{x}) = 0$ therefore $\beta(\mathbf{x}) = -\alpha(\mathbf{x})$
- The problem simplifies to

$$\max_{\alpha(\mathbf{x})} \int \alpha(\mathbf{x}) p(\mathbf{x}) d\mathbf{x} - \int \alpha(\mathbf{y}) q(\mathbf{y}) d\mathbf{y} = \max_{\alpha(\mathbf{x})} (\mathbb{E}_p[\alpha(\mathbf{x})] - \mathbb{E}_q[\alpha(\mathbf{x})])$$

subject to

$$\alpha(\mathbf{x}) - \alpha(\mathbf{y}) \leq d(\mathbf{x}, \mathbf{y})$$

- functions $\alpha(\mathbf{x})$ that satisfy this constraint are known as *Lipschitz-1 functions*
- An equivalent condition is that

$$\|\nabla_{\mathbf{x}} \alpha(\mathbf{x})\| \leq 1$$

- this is a continuity condition saying the output has to change slowly as we change the input

2.3 Wasserstein GANs

- In our Wasserstein GAN we train a generator to minimise the Wasserstein distance between the distribution of images from the generator and the true distribution
- We use mini-batches to approximate the expectations

$$\mathbb{E}_p[\alpha(\mathbf{x})] \approx \frac{1}{|\mathcal{B}|} \sum_{\mathbf{x} \in \mathcal{B}} \alpha(\mathbf{x}), \quad \mathbb{E}_q[\alpha(\mathbf{x})] \approx \frac{1}{n} \sum_{i=1}^n \alpha(G(\mathbf{z}_i, \mathbf{w}_G))$$

- We need to find the function $\alpha(\mathbf{x})$ that maximises the difference between these expectations
- We make $\alpha(\mathbf{x})$ a neural network called the *critic*
 - this plays the same role as the discriminator in a normal GAN
 - Again we make this a CNN
 - The difference is it has to be Lipschitz-1
 - This is difficult to achieve and is usually bodged (you can read the literature if you are interested)
- Wasserstein GANs claim to solve many of the problems of normal GANs
 - They are not perfect because they only approximate the Lipschitz-1
- They are for me one of the elegant solutions in machine learning of the last few years

Advanced Machine Learning Subsidiary Notes

Lecture 20: Probability

Adam Prügel-Bennett

February 9, 2024

1 Keywords

- Probability, Random variable, Expectations

2 Main Points

2.1 Basic Language

- Probability is the process of modelling a world with uncertain outcomes
- The model is not perfect, but reflects what we care about
- Mathematicians model the possible outcomes of the world they are considering in terms of a set of *elementary events*
- This set is frequently called Ω (but you are free to give it any name)
- The elementary events are *mutual exclusive* so that if $\omega_i, \omega_j \in \Omega$ then $\omega_i \cap \omega_j = \emptyset$ (this just means there is no intersection between elementary events)
- The set is exhaustive so that $\bigcup_i \omega_i = \Omega$
- This means in our model of the world every possible outcome is included in Ω
- Now we are often not interested in elementary events (they may be too small)
- Instead we consider **events** $\mathcal{E} = \bigcup_{i \in \mathcal{I}} \omega_i$ where $\mathcal{I}$ is some *index set*
- Suppose we are modelling the probability of a darts match
- Ω might represent every location where a dart might land
- $\omega_i \in \Omega$ is a particular position
- I might be interested in the probability of a bull's eye $\mathcal{B} = \bigcup_{i \in \mathcal{I}} \omega_i$
- In this particular case this is a union of an uncountable infinity of points
- In this case I might represent the elementary events by a 2-d vector x
- The event of getting a bull's eye corresponds to x being within the region that defines the bull's eye

2.2 Probability and Random Variables

- We can attribute a **probability**, $\mathbb{P}[\mathcal{E}]$, to an event, $\mathcal{E}$
- Probabilities are number between 0 and 1: $0 \leq \mathbb{P}[\mathcal{E}] \leq 1$
- 0 means the event never occurs, 1 means the event definitely will occur
- $\mathbb{P}[\mathcal{E}] + \mathbb{P}[\neg\mathcal{E}] = 1$ where $\neg\mathcal{E} = \Omega \setminus \mathcal{E}$
- That is, the probability of an event occurring and the event not occurring is 1
- In some cases we can interpret $\mathbb{P}[\mathcal{E}]$ as the expected frequency of occurrence of a repetitive trial
- E.g. If the event, $\mathcal{E}$ is rolling an (honest) dice and getting an even number then $\mathbb{P}[\mathcal{E}] = \frac{1}{2}$ can be seen as saying that on average this will happen half the time if you repeat the experiment often enough
- However how to we interpret $\mathbb{P}[\text{Pass COMP6208 exam}]$?
- Hopefully, this is something you only attempt once
- We can think of probability as encoding our informed belief that something will happen
- When our knowledge changes our probability will change
- Random variables, X , are numbers we assign to each elementary event (many events can have the same number assigned)
- That is we partition the set of outcomes Ω and assign a numbers to each partition
- E.g. for a dice

$$X = \begin{cases} 0 & \text{if } \omega \in \{1, 3, 5\} \\ 1 & \text{if } \omega \in \{2, 4, 6\} \end{cases}$$

- If we let $\mathcal{E}_i$ be the event that corresponding to the partition with value x_i then $\mathbb{P}[X = x_i] = \mathbb{P}[\mathcal{E}_i]$
- It is common practice (though not universal) to denote random variables with capital letters
- The correspond to events with uncertain outcomes (things that happen in the future)
- We distinguish the scalar that we assign to random variables by writing them in lower case, e.g. x_i
- When we write $\mathbb{P}[X]$ we can view this as short-hand for

$$(\mathbb{P}[X = x] \mid x \in \mathcal{X}) = (\mathbb{P}[X = x_1], \mathbb{P}[X = x_2], \dots, \mathbb{P}[X = x_n])$$

where $\mathcal{X}$ is the set of possible values that X can take

- That is, $\mathbb{P}[X]$, is the probability mass function which tells us the probability of the random variable X for all possible values it can take
- We differentiate between random variables (capitals) and scalars (lower case) because they behave very differently when we take expectations
- A function, $Y = g(X)$, of a random variable X is a random variable
- **Continuous random variables**

Advanced Machine Learning Subsidiary Notes

Lecture 21: Bayesian Inference

Adam Prügel-Bennett

February 9, 2024

1 Keywords

- Bayes, Conjugate Priors, Uninformative Priors

2 Main Points

** Bayesian Statistics

- Background
 - Bayesian statistics is an approach to making statistical inference
 - It was first proposed by the Rev. Thomas Bayes but published after his death in 1761
 - It was championed by the great French polymath Pierre-Simon Laplace
 - It fell out of favour at the beginning of the 20th Century due to philosophical prejudice
 - * It was deemed to be unscientific because it requires specifying a prior probability which seemed objective
 - It was championed from the 1950's by Ed Jaynes who eventually convinced most people that Bayesian inference is consistent
 - Perhaps more importantly with the raise of computers it was possible to demonstrate that Bayesian methods work
- Bayes' Rule
 - Bayes rule follows from a simple identity

$$\mathbb{P}[h_i|\mathcal{D}] = \frac{\mathbb{P}[\mathcal{D}|h_i] \mathbb{P}[h_i]}{\mathbb{P}[\mathcal{D}]}$$

- * $\mathbb{P}[h_i|\mathcal{D}]$ is the *posterior* probability of hypothesis h_i given data $\mathcal{D}$
- * $\mathbb{P}[\mathcal{D}|h_i]$ is the *likelihood* of the data given the hypothesis h_i
- * $\mathbb{P}[h_i]$ is our *prior* probability (our belief in hypothesis h_i before seeing the data)
- * $\mathbb{P}[\mathcal{D}]$ is a normalisation terms I will call the *evidence* and is just equal to

$$\mathbb{P}[\mathcal{D}] = \sum_{i=1}^n \mathbb{P}[h_i, \mathcal{D}] = \sum_{i=1}^n \mathbb{P}[\mathcal{D}|h_i] \mathbb{P}[h_i]$$

- Although this is just a mathematical identity, to use it in inference requires specifying a prior belief
 - I argued from the first lecture that for machine learning to work we need to make assumptions about the data
 - In Bayesian approach these assumptions are built into our likelihood and prior

- A nice property of Bayesian inference is that it turns the problem from an inverse problem (computing the posterior $\mathbb{P}[h_i|\mathcal{D}]$) into a forward problem of computing the likelihood $\mathbb{P}[\mathcal{D}|h_i]$
 - This solves problems of missing data (we only need to calculate the probability of the data we see)
 - It is also much more straightforward to model the forward process (from physics) rather than the inverse process
 - That said, it can still be technically very challenging
- Bayes' rule has the same form whether we are working with discrete variables (where we have probability masses) or continuous variables (where we have probability densities)

2.1 Conjugate Priors

- Technically performing Bayesian inference can be difficult because the posterior can be very ugly to work with
- But for a few of the classic likelihood functions there exist *conjugate prior distributions* such that the posterior is in the same class of distributions as the prior
 - Suppose that the likelihood of observing data $\mathbf{X}$ given some parameters θ is $p(\mathbf{X}|\theta)$ (this may be a probability mass or density depending on whether $\mathbf{X}$ is continuous or not)
 - We want to infer θ which we do through the posterior $p(\theta|\mathbf{X})$
 - If we are lucky there is a family of distributions, $\text{Conj}(\theta|\phi)$ parameterised by ϕ , that describe the parameters of the likelihood θ and satisfy

$$\text{Conj}(\theta|\phi') \propto p(\mathbf{X}|\theta) \text{Conj}(\theta|\phi')$$

- Thus if we start with a prior $\text{Conj}(\theta|\phi_0)$ then after observing data $\mathbf{X}_1$ we would end up with a posterior $\text{Conj}(\theta|\phi_1)$ where ϕ_1 depends on observation $\mathbf{X}_1$ and ϕ_0
- We can repeat this
 - * When we make a second observation $\mathbf{X}_2$ our posterior now becomes our prior (we have updated our beliefs) so now we obtain a new posterior

$$\text{Conj}(\theta|\phi_2) \propto p(\mathbf{X}_2|\theta) \text{Conj}(\theta|\phi_1)$$

- As an example let us consider a number of Bernoulli trials (we assume independent observations $X_i \in \{0, 1\}$ occurring with probability p)
 - But we don't know p
 - We assume our prior distribution is a beta distribution $\text{Beta}(p|a_0, b_0) \propto p^{a_0-1} (1-p)^{b_0-1}$
 - Our likelihood is $\mathbb{P}[X|p] = p^X (1-p)^{1-X}$
 - Using Bayes' rule our posterior is

$$f(p|X) \propto p^X (1-p)^{1-X} p^{a_0-1} (1-p)^{b_0-1} = p^{a_0+X-1} (1-p)^{b_0+1-X-1}$$

- But this has the same functional form as a beta-distribution
- It will be a beta-distribution as it has to be normalised (posteriors are always normalised)
- Thus $f(p|X) = \text{Beta}(p|a_1, b_1)$ where $a_1 = a_0 + X$ and $b_1 = b_0 + 1 - X$
- The Poisson distribution is defined as

$$\text{Pois}[N|\mu] = \frac{\mu^N e^{-\mu}}{N!}$$

- It describes the probability of observing N events where each event is assumed independent and the expected number of events is μ
- Suppose we have made observations N_1, N_2 , etc. and we are trying to infer μ
- The Poisson distribution has a conjugate distribution which is the gamma distribution

$$\text{Gamma}(\mu|a, b) = \frac{b^a \mu^{a-1} e^{-b\mu}}{\Gamma(a)}$$

- Assuming a prior $\text{Gamma}(\mu|a_0, b_0)$ and assume we make an observation N_1 then the posterior is given by

$$f(\mu|N_1) \propto \mu^{N_1} e^{-\mu} \mu^{a_0-1} e^{-b_0\mu} = \mu^{a_0+N_1-1} e^{-(b_0+1)\mu}$$

- Thus $f(\mu|N_1) = \text{Gamma}(\mu|a_1, b_1)$ where

$$* a_1 = a_0 + N_1$$

$$* b_1 = b_0 + 1$$

- Conjugate priors are the exception rather than the rule but they do occur for some classic distributions. E.g.

Likelihood	Prior
Binomial/Bernoulli	Beta
Poisson	Gamma
Multinomial	Dirchlet
Univariate Normal	Gamma-Normal
Multivariate Normal	Wishart

2.2 Uninformative Priors

- What should we do if we have no prior information
- This disturbed statisticians who felt that there might not be a subjective answer to this
- However, Ed Jaynes argued that there is a unique answer that we get by requiring invariance
- Scale parameters
 - Often we are trying to infer scale parameters
 - These would include the rate in the Poisson distribution or the standard deviation in a normal distribution
 - They are non-negative numbers that determine the scale (i.e. what units we should measure in)
 - For most problems we have some idea about this (otherwise it would be difficult to do the measurement)
 - But we may still not know the order of magnitude of what we are measuring in
 - What should we use as a prior?
 - The answer to this was given by Harold Jeffreys and is known as the *Jeffreys' prior*
 - Let's see the argument
 - If we have no idea about what scale an observable x takes then we should expect there to be equal probability to be in the intervals $[A, B]$ or the interval $[A/c, B/c]$
 - This may seem strange as these two intervals are different lengths (but non-overlapping if $c \neq 1$) but if this wasn't the case we would know something about which scale to use

- If our prior for x is $p(x)$ then we require

$$\int_A^B p(x) dx = \int_{A/c}^{B/c} p(x) dx$$

- Now we can make a change of variables
- $y = cx$ so this last integral becomes

$$\int_{A/c}^{B/c} p(x) dx = \int_A^B \frac{1}{c} p\left(\frac{y}{c}\right) dy$$

- But for this to equal the first integral for any interval $[A, B]$ we require

$$p(x) \propto \frac{1}{x}$$

- We note that if $p(x) = 1/x$ then

$$p\left(\frac{x}{c}\right) = \frac{c}{x} = c p(x)$$

as required

- The strange thing about this distribution is it is improper in that

$$\int_0^\infty \frac{1}{x} = \infty$$

- Strangely a lot of uninformative priors turn out to be improper
- However, this doesn't seem to matter in Bayesian inference: after making some observations we end up with a proper prior

• Benford's Law

- There is a strange consequence of Jeffreys prior which was first speculated upon in 1881 when Simon Newcomb noticed that logarithm tables were much more used for numbers beginning with 1 rather than 2 and 2 rather than 3, etc. Frank Benford in 1938 looked at the occurrence of numbers in data set and found that the digit of first significant figure was more likely to be 1 than 2 and more likely to be 2 than 3.
- If we assume these numbers are measured in an arbitrary scale and the occurrence of these numbers followed Jeffreys prior then the probability that real number between 1 and 10 actually took values between n and $n + 1$ is given by

$$\frac{\int_n^{n+1} \frac{1}{x} dx}{\int_1^{10} \frac{1}{x} dx} = \frac{\log(n+1) - \log(n)}{\log(10)} = \log\left(\frac{n+1}{n}\right)$$

- Note that we would get the same result if the number was between 10 and 100 and we asked the probability that it was in the range $10n$ to $10(n+1)$
- But this means the digit of the most significant figure will be n
- So numbers beginning with 1 occur more often than numbers beginning with 2, and numbers beginning with 2 occur more often than those beginning with 3, etc.
- This is weird but true (for most naturally occurring non-negative numbers)

3 Exercises

3.1 Throwing Dice

- There are three dice on the table. Two of them are normal dice, while the other dice has 6 on two faces and 1, 2, 3 and 5 on the other face. A dice is chosen at random and is thrown 10 times. On three occasions the top face is a 6. What is the probability that the dice chosen is the dishonest dice?
- See answers below

4 Experiments

4.1 Benford's Law

- Benford's law is so wacky that to convince yourself it is true you really need to test it out
- Find a set of data with features that are clearly scale parameters and have a go (you need enough data points to be convincing)
 - if you numbers are positive and could take any value then they are likely to be scale parameter

5 Answers

5.1 Throwing Dice

- We use Bayes' rule

$$P(\text{dishonest}|\text{data}) = \frac{P(\text{data}|\text{dishonest}) P(\text{dishonest})}{P(\text{data}|\text{dishonest}) P(\text{dishonest}) + P(\text{data}|\text{honest}) P(\text{honest})}$$

- Now $P(\text{dishonest}) = 1/3$ and $P(\text{honest}) = 2/3$
- The probability of k success out of n trials with a success probability p is given by the binomial distribution

$$\text{Binom}(k|n, p) = \binom{n}{k} p^k (1-p)^{n-k}$$

- The probability of 3 out of 10 rolls being a 6 is given by

$$P(\text{data}|\text{dishonest}) = \text{Binom}(3|10, \frac{1}{3}) = \binom{3}{10} \left(\frac{1}{3}\right)^3 \left(\frac{2}{3}\right)^7$$

$$P(\text{data}|\text{honest}) = \text{Binom}(3|10, \frac{1}{6}) = \binom{3}{10} \left(\frac{1}{6}\right)^3 \left(\frac{5}{6}\right)^7$$

- Thus

$$P(\text{dishonest}|\text{data}) = \frac{1}{1 + \frac{P(\text{data}|\text{honest}) P(\text{honest})}{P(\text{data}|\text{dishonest}) P(\text{dishonest})}}$$

$$= \frac{1}{1 + 2 \left(\frac{1}{2}\right)^3 \left(\frac{5}{4}\right)^7} = 0.29549$$

Advanced Machine Learning Subsidiary Notes

Lecture 22: Gaussian Processes

Adam Prügel-Bennett

February 9, 2024

1 Keywords

- Gaussian Processes, regression

2 Main Points

2.1 The Big Picture

- Gaussian processes are a machine learning method for regression
 - You can use them for classification but it is regression where they excel
- They put a prior on functions (preferring smooth functions)
- Conceptually they are easy
- They are easy to use
- But they are mathematically fiddly to understand
- Gaussian processes were first used for spatial modelling where the technique was known as *kriging*
- They are one of the most powerful methods we have of doing regression
- They are quite general purpose as the prior we use is reasonable for a large number of applications

2.2 Gaussian Processes

- Gaussian Processes are also mathematical objects
- A Gaussian Process assigns a probability to functions depending on their spatial smoothness
 - We assume that the probability of $f(\mathbf{x})$ taking a particular value is normally/Gaussian distributed in a way that depends on the value of the function at other points
- This will be governed by a *covariance* or *kernel* function $k(\mathbf{x}, \mathbf{y})$ which tells us the covariance of the prior

$$\mathbb{E}_f [(f(\mathbf{x}) - m(\mathbf{x})) (f(\mathbf{y}) - m(\mathbf{y}))] = k(\mathbf{x}, \mathbf{y})$$

- This is an expectation over all functions from our probability distribution
- $m(\mathbf{x})$ encodes a prior belief in the value of $f(\mathbf{x})$ —normally we choose this to be 0 as most often we don't have any prior knowledge of the function
- Because the covariance is quadratic it will be positive semi-definite

- The kernel function is critical to determining the type of function we will learn
- Choosing this kernel function correctly is essential to make GP work
- The values at each point $\mathbf{x}$ is normally or Gaussianly distributed

$$p(f|m, k) \propto e^{-\frac{1}{2} \int (f(\mathbf{x}) - m(\mathbf{x})) k^{-1}(\mathbf{x}, \mathbf{y}) (f(\mathbf{y}) - m(\mathbf{y})) d\mathbf{x} d\mathbf{y}}$$

- It is a little unclear exactly what this means for functions: working with function spaces one has to be very careful (e.g. to normalise this we have to sum over all functions)
- Nevertheless it is clear that we can use this for comparing the likelihood of different functions
- If you discretise the domain of $\mathbf{x}$ then

$$p(f|m, k) \propto e^{-\frac{1}{2} \sum_{i,j} (f(\mathbf{x}_i) - m(\mathbf{x}_i)) k^{-1}(\mathbf{x}_i, \mathbf{x}_j) (f(\mathbf{x}_j) - m(\mathbf{x}_j))}$$

- * the probability function is now a well-defined multi-dimensional normal distribution
- * here it is clear that the value at each point is normally distributed
- * to get back to the integral form we have to take a limit where our discrete points become every closer together (this stresses out mathematicians who point out that nasty things can happen, but for the rest of us we just go ahead)
- We can sample Gaussian processes from this distribution
 - * in low dimensions this allows us to visualise the type of functions we are likely to learn
 - * you can try this out in the experiment section

2.3 Bayesian Inference

- We can use Gaussian Processes as a prior for performing regression
- We assume we have a set of data points $\mathcal{D} = ((\mathbf{x}_i, y_i) | i = 1, \dots, m)$
- The likelihood for data is taken to be a normal distribution

$$p(\mathcal{D}|f) = \prod_{i=1}^m \mathcal{N}(y_i | f(\mathbf{x}_i), \sigma^2)$$

- We assume that $f(\mathbf{x}_i)$ is the true value of the function at $\mathbf{x}_i$, but, because of measurement error, y_i will be distributed according to a normal distribution
- σ is the accuracy of our measured data (it is another hyper-parameter we must choose)
- Note we are assuming the data points are independent of each other (which is usually a good approximation)
- We can now compute a posterior

$$p(f|\mathcal{D}, m, k, \sigma) = \frac{p(\mathcal{D}|f, \sigma) p(f|m, k)}{p(\mathcal{D}|m, k, \sigma)}$$

- I have explicitly written in the dependencies on the hyper-parameters m , k and σ
- The Gaussian Process prior is a conjugate distribution for the normal likelihood
- Computing probabilities over functions is slightly wild
- It is more useful to compute the probability at a particular point $\mathbf{x}^*$

- To compute this we marginalise over all other points
 - * This is non-trivial but doable
 - * If you have a multivariate normal distribution

$$\mathcal{N}(\mathbf{x}|\boldsymbol{\mu}, \mathbf{C}) = \frac{1}{\sqrt{|2\pi\mathbf{C}|}} e^{-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^\top \mathbf{C}(\mathbf{x}-\boldsymbol{\mu})}$$

and you integrate over x_i (marginalise it out) then

$$\int_{-\infty}^{\infty} \mathcal{N}(\mathbf{x}|\boldsymbol{\mu}, \mathbf{C}) dx_i = \mathcal{N}(\hat{\mathbf{x}}|\hat{\boldsymbol{\mu}}, \hat{\mathbf{C}})$$

where $\hat{\mathbf{x}}$ and $\hat{\boldsymbol{\mu}}$ are identical to $\mathbf{x}$ and $\boldsymbol{\mu}$ except with their i^{th} component removed and $\hat{\mathbf{C}}$ is identical to $\mathbf{C}$ except with the i^{th} row and column removed

- * Intuitively this seems very natural (if x_i can take any value it doesn't change the mean or covariance between other random variables)
- * You can prove this algebraically but it is not a trivial calculation
- * Because of this for a Gaussian Process we only care about those set of points where we have data values
- For all point $\mathbf{x}$ (except the point $\mathbf{x}^*$) we integrate over the possible values that $f(\mathbf{x})$ can take
- Where we have no prior this integral is just a constant—and will cancel with the denominator $p(\mathcal{D}|m, k, \sigma)$
- We are left with

$$p(f(\mathbf{x}^*)|\mathcal{D}) \propto \left(\prod_i \int df(\mathbf{x}_i) \right) e^{-\frac{1}{2\sigma^2} \sum_{i=1}^m (y_i - f(\mathbf{x}_i))^2} e^{-\frac{1}{2} \sum_{i,j=1}^m (f(\mathbf{x}_i) - m(\mathbf{x}_i)) k^{-1}(\mathbf{x}_i, \mathbf{x}_j) (f(\mathbf{x}_j) - m(\mathbf{x}_j))}$$

$$e^{-\sum_{j=1}^m (f(\mathbf{x}^*) - m(\mathbf{x}^*)) k^{-1}(\mathbf{x}^*, \mathbf{x}_j) (f(\mathbf{x}_j) - m(\mathbf{x}_j))}$$

$$e^{-\frac{1}{2} (f(\mathbf{x}^*) - m(\mathbf{x}^*)) k^{-1}(\mathbf{x}^*, \mathbf{x}^*) (f(\mathbf{x}^*) - m(\mathbf{x}^*))}$$

- * Writing $f^* = f(\mathbf{x}^*)$, $f_i = f(\mathbf{x}_i)$, $m_i = m(\mathbf{x}_i)$, $k_{ij}^{-1} = k^{-1}(\mathbf{x}_i, \mathbf{x}_j)$, $k_j^{*-1} = k^{-1}(\mathbf{x}^*, \mathbf{x}_j)$ and $k^{*-1} = k^{-1}(\mathbf{x}^*, \mathbf{x}^*)$ then

$$p(f^*|\mathcal{D}) \propto e^{-\frac{1}{2} k^{*-1} (f^* - m^*)^2} \left(\prod_i \int df_i \right) e^{-\frac{1}{2\sigma^2} \sum_{i=1}^m (y_i - f_i)^2 - \frac{1}{2} \sum_{i,j=1}^m (f_i - m_i) k_{ij}^{-1} (f_j - m_j)}$$

$$e^{-\sum_{j=1}^m (f^* - m^*) k_j^{*-1} (f_j - m_j)}$$

- * Doing Gaussian integrals is a pain—I don't expect you to do this
- * But if you are brave enough to try let $g_i = f_i - m_i$, $g^* = f^* - m^*$ and $\hat{y}_i = y_i - m_i$ then we can rewrite this as

$$p(f^*|\mathcal{D}) \propto e^{-\frac{1}{2} k^{*-1} g^{*2}} \left(\prod_i \int dg_i \right) e^{-\frac{1}{2\sigma^2} \sum_{i=1}^m (\hat{y}_i - g_i)^2 - \frac{1}{2} \sum_{i,j=1}^m g_i k_{ij}^{-1} g_j - g^* \sum_{j=1}^m k_i^{*-1} g_j}$$

· where we made a change of variable in the integral

- * In matrix form we can write this as

$$p(f^*|\mathcal{D}) \propto e^{-\frac{1}{2} k^{*-1} g^{*2}} \left(\prod_i \int dg_i \right) e^{(\sigma^{-2} \hat{\mathbf{y}} - g^* \mathbf{k}^{*-1})^\top \mathbf{g} - \frac{1}{2} \mathbf{g}^\top (\mathbf{k}^{-1} - \sigma^{-2} \mathbf{I}) \mathbf{g}}$$

- I've dropped a term $e^{-\frac{\|\hat{\mathbf{y}}\|^2}{2\sigma^2}}$ since this is a constant that can be absorbed into the constant of proportionality
- * To perform the integrals over g_i we *complete the square*. To make the algebra easier to follow we define $\mathbf{M} = \mathbf{k}^{-1} - \sigma^{-2}\mathbf{I}$ and $\mathbf{b} = \sigma^{-2}\hat{\mathbf{y}} - g^*\mathbf{k}^{*-1}$. We now rewrite the exponent, E , of the integral as

$$\begin{aligned} E &= (\sigma^{-2}\hat{\mathbf{y}} - g^*\mathbf{k}^{*-1})^\top \mathbf{g} - \frac{1}{2}\mathbf{g}^\top (\mathbf{k}^{-1} - \sigma^{-2}\mathbf{I}) \mathbf{g} = \mathbf{b}^\top \mathbf{g} - \frac{1}{2}\mathbf{g}^\top \mathbf{M} \mathbf{g} \\ &= -\frac{1}{2}(\mathbf{g} - \mathbf{M}^{-1}\mathbf{b})^\top \mathbf{M} (\mathbf{g} - \mathbf{M}^{-1}\mathbf{b}) + \frac{1}{2}\mathbf{b}^\top \mathbf{M}^{-1} \mathbf{b} \end{aligned}$$

- You simply have to expand out all the terms to see these are the same
- * Defining $\mathbf{h} = \mathbf{g} - \mathbf{M}^{-1}\mathbf{b}$ and substituting this back into $p(f^*|\mathcal{D})$ we get

$$p(f^*|\mathcal{D}) \propto e^{-\frac{1}{2}k^{*-1}g^{*2} + \frac{1}{2}\mathbf{b}^\top \mathbf{M}^{-1} \mathbf{b}} \left(\prod_i \int d\mathbf{h}_i \right) e^{-\frac{1}{2}\mathbf{h}^\top \mathbf{M} \mathbf{h}}$$

- * The last integral is a Gaussian integral that integrates to a constant (and can be absorbed into the constant of proportionality)
- * We are left with

$$\begin{aligned} p(f^*|\mathcal{D}) &\propto e^{-\frac{1}{2}k^{*-1}g^{*2} + \frac{1}{2}\mathbf{b}^\top \mathbf{M}^{-1} \mathbf{b}} \\ &= e^{-\frac{1}{2}k^{*-1}g^{*2} + \frac{1}{2}(\sigma^{-2}\hat{\mathbf{y}} - g^*\mathbf{k}^{*-1})^\top \mathbf{M}^{-1} (\sigma^{-2}\hat{\mathbf{y}} - g^*\mathbf{k}^{*-1})} \\ &\propto e^{-\frac{1}{2}(k^{*-1} - \mathbf{k}^{*-1\top} \mathbf{M}^{-1} \mathbf{k}^{*-1})g^{*2} - \frac{g^*}{\sigma^2} \hat{\mathbf{y}}^\top \mathbf{M}^{-1} \mathbf{k}^{*-1}} \end{aligned}$$

- where we have used $\mathbf{b} = \sigma^{-2}\hat{\mathbf{y}} - g^*\mathbf{k}^{*-1}$
- we have also thrown away terms not involving g^* (as they are constants that can be absorbed into the constant of proportionality)
- * We can again *complete the square* this time for g^*

$$p(f^*|\mathcal{D}) \propto e^{-\frac{1}{2}(k^{*-1} - \mathbf{k}^{*-1\top} \mathbf{M}^{-1} \mathbf{k}^{*-1})(g^* - \frac{\hat{\mathbf{y}}^\top \mathbf{M}^{-1} \mathbf{k}^{*-1}}{\sigma^2(k^{*-1} - \mathbf{k}^{*-1\top} \mathbf{M}^{-1} \mathbf{k}^{*-1})})^2}$$

- We've again dropped terms that don't contain g^*
- * But $g^* = f^* - m^*$ so we can write this as

$$\begin{aligned} p(f^*|\mathcal{D}) &\propto e^{-\frac{1}{2}(k^{*-1} - \mathbf{k}^{*-1\top} \mathbf{M}^{-1} \mathbf{k}^{*-1})(f^* - m^* - \frac{\hat{\mathbf{y}}^\top \mathbf{M}^{-1} \mathbf{k}^{*-1}}{\sigma^2(k^{*-1} - \mathbf{k}^{*-1\top} \mathbf{M}^{-1} \mathbf{k}^{*-1})})^2} \\ &\propto \mathcal{N}\left(f^* \middle| m^* + \frac{\hat{\mathbf{y}}^\top \mathbf{M}^{-1} \mathbf{k}^{*-1}}{\sigma^2(k^{*-1} - \mathbf{k}^{*-1\top} \mathbf{M}^{-1} \mathbf{k}^{*-1})}, k^{*-1} - \mathbf{k}^{*-1\top} \mathbf{M}^{-1} \mathbf{k}^{*-1}\right) \end{aligned}$$

- where we use the fact that we end up with a term that has the form of a normal distribution
- because the posterior term is normalise, in fact it has to be exactly equal to this normal distribution
- * Using $\mathbf{M} = \mathbf{k}^{-1} - \sigma^{-2}\mathbf{I}$

$$p(f^*|\mathcal{D}) = \mathcal{N}\left(f^* \middle| m^* + \frac{\hat{\mathbf{y}}^\top (\mathbf{k}^{-1} - \sigma^{-2}\mathbf{I})^{-1} \mathbf{k}^{*-1}}{\sigma^2(k^{*-1} - \mathbf{k}^{*-1\top} (\mathbf{k}^{-1} - \sigma^{-2}\mathbf{I})^{-1} \mathbf{k}^{*-1})}, \frac{1}{k^{*-1} - \mathbf{k}^{*-1\top} (\mathbf{k}^{-1} - \sigma^{-2}\mathbf{I})^{-1} \mathbf{k}^{*-1}}\right)$$

- * In other words the expected mean value of $f^* = f(\mathbf{x}^*)$ is

$$\mathbb{E}[f(\mathbf{x}^*)] = m(\mathbf{x}^*) + \frac{\hat{\mathbf{y}}^\top (\mathbf{k}^{-1} - \sigma^{-2}\mathbf{I})^{-1} \mathbf{k}^{*-1}}{\sigma^2(k^{*-1} - \mathbf{k}^{*-1\top} (\mathbf{k}^{-1} - \sigma^{-2}\mathbf{I})^{-1} \mathbf{k}^{*-1})}$$

(note that usually $m(\mathbf{x}^*) = 0$) and the expected variance in the value is

$$\mathbb{V}\text{ar}[f(\mathbf{x}^*)] = \frac{1}{k^{*-1} - \mathbf{k}^{*-1\top} (\mathbf{k}^{-1} - \sigma^{-2}\mathbf{I})^{-1} \mathbf{k}^{*-1}}$$

- * The result is pretty horrible because it involves inverting a matrix $\mathbf{K}$ with components $k(\mathbf{x}, \mathbf{y})$ evaluated at the set of points $\{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_m, \mathbf{x}^*\}$
- * This is usually feasible but there is a simpler form that occurs because of identities involving inverse matrices
- * This is best obtained using a second way of deriving the results
- * I went to the pain of deriving the result this way because it is just a consequence of Bayes' rule
- In the second derivation we consider joint probability of the observations $\{y_i | i = 1, 2, \dots, m\}$ and $f^* = f(\mathbf{x}^*)$
 - Even this derivation is a pain and you are **not** expected to know it
 - Now we assume $y_i = f(\mathbf{x}_i) + \epsilon_i$ where $\epsilon_i \sim \mathcal{N}(0, \sigma^2)$ (that is our observations have a normally distributed error) then

$$\mathbb{E}[y_i y_j] = \mathbb{E}[(f(\mathbf{x}_i) + \epsilon_i)(f(\mathbf{x}_j) + \epsilon_j)] = k(\mathbf{x}_i, \mathbf{x}_j) + \sigma^2 \delta_{ij}$$

- * since the observation error, ϵ_i , is assumed independent of the function value, $f(\mathbf{x}_i)$
- * here we assume $m(\mathbf{x}) = 0$ just to make life simple

- Similarly

$$\mathbb{E}[y_i f^*] = \mathbb{E}[(f(\mathbf{x}_i) + \epsilon_i)f(\mathbf{x}^*)] = k(\mathbf{x}_i, \mathbf{x}^*)$$

and

$$\mathbb{E}[(f^*)^2] = k(\mathbf{x}^*, \mathbf{x}^*)$$

- Thus

$$p(f^*, \mathbf{y}) = \mathcal{N}\left(\begin{pmatrix} \mathbf{y} \\ f^* \end{pmatrix} \middle| \mathbf{0}, \begin{pmatrix} \mathbf{K} + \sigma^2 \mathbf{I} & \mathbf{k}^* \\ \mathbf{k}^{*\top} & k^* \end{pmatrix}\right)$$

- * where $\mathbf{K}$ is a matrix with components $k(\mathbf{x}_i, \mathbf{x}_j)$, $\mathbf{k}^*$ is a vector with components $k(\mathbf{x}_i, \mathbf{x}^*)$ and $k^* = k(\mathbf{x}^*, \mathbf{x}^*)$

- We now use an identity involving matrix inverses

$$\begin{pmatrix} \mathbf{K} + \sigma^2 \mathbf{I} & \mathbf{k}^* \\ \mathbf{k}^{*\top} & k^* \end{pmatrix}^{-1} = \begin{pmatrix} \left(\mathbf{K} + \sigma^2 \mathbf{I} - \frac{\mathbf{k}^{*\top} \mathbf{k}^*}{k^*}\right)^{-1} & -\frac{(\mathbf{K} + \sigma^2 \mathbf{I})^{-1} \mathbf{k}^*}{k^* - \mathbf{k}^{*\top} (\mathbf{K} + \sigma^2 \mathbf{I})^{-1} \mathbf{k}^*} \\ -\frac{\mathbf{k}^{*\top} (\mathbf{K} + \sigma^2 \mathbf{I})^{-1}}{k^* - \mathbf{k}^{*\top} (\mathbf{K} + \sigma^2 \mathbf{I})^{-1} \mathbf{k}^*} & \frac{1}{k^* - \mathbf{k}^{*\top} (\mathbf{K} + \sigma^2 \mathbf{I})^{-1} \mathbf{k}^*} \end{pmatrix}$$

- * these identities involving inverses of matrices seem to come from nowhere—they make working with normal distributions a real pain

- * you can prove the identity by multiply the right-hand side by $\begin{pmatrix} \mathbf{K} + \sigma^2 \mathbf{I} & \mathbf{k}^* \\ \mathbf{k}^{*\top} & k^* \end{pmatrix}$ and showing you get the identity

- * if you are tempted to do this then set $\sigma^2 = 0$ and rename $\mathbf{k}^*$ and k^* so its easier to do the algebra—it is a tedious calculation (see exercises)

- Now we use $p(f^* | \mathcal{D}) = p(f^* | \mathbf{y}) = p(f^*, \mathbf{y}) / p(\mathbf{y})$
- All you need to do is collect the terms in $p(f^*, \mathbf{y})$ involving f^*

$$p(f^* | \mathcal{D}) \propto e^{-\frac{f^{*2}}{2(k^* - \mathbf{k}^{*\top} (\mathbf{K} + \sigma^2 \mathbf{I})^{-1} \mathbf{k}^*)}} + \frac{f^* \mathbf{k}^{*\top} (\mathbf{K} + \sigma^2 \mathbf{I})^{-1} \mathbf{y}}{k^* - \mathbf{k}^{*\top} (\mathbf{K} + \sigma^2 \mathbf{I})^{-1} \mathbf{k}^*}$$

- *Completing the square*

$$p(f^* | \mathcal{D}) \propto e^{-\frac{1}{2(k^* - \mathbf{k}^{*\top} (\mathbf{K} + \sigma^2 \mathbf{I})^{-1} \mathbf{k}^*)}} (f^* - \mathbf{k}^{*\top} (\mathbf{K} + \sigma^2 \mathbf{I})^{-1} \mathbf{y})^2$$

- But as $p(f^* | \mathcal{D})$ must be normalised

$$p(f(\mathbf{x}^*) | \mathcal{D}) = \mathcal{N}\left(f(\mathbf{x}^*) \middle| \mathbf{k}^{*\top} (\mathbf{K} + \sigma^2 \mathbf{I})^{-1} \mathbf{y}, k^* - \mathbf{k}^{*\top} (\mathbf{K} + \sigma^2 \mathbf{I})^{-1} \mathbf{k}^*\right)$$

- That is the mean value of f at a point $\mathbf{x}^*$ is

$$\mathbb{E}[f(\mathbf{x}^*)] = m(\mathbf{x}^*) + \mathbf{k}^{*\top}(\mathbf{K} + \sigma^2 \mathbf{I})^{-1} \mathbf{y}$$

* I've re-instated $m(\mathbf{x}^*)$ (even though it is usually taken to be zero)

- The variance is given by

$$\text{Var}[f(\mathbf{x}^*)] = k^* - \mathbf{k}^{*\top}(\mathbf{K} + \sigma^2 \mathbf{I})^{-1} \mathbf{k}^*$$

- This may look completely different from the previous expression but surprisingly it is identical (you can demonstrate this numerically)
- The advantage over the previous methods is that we only need to do one matrix inversion (and we get a slightly easier expression)
- Although the mathematics to get here was horrible the expression is extremely easy to compute
- One of the advantages of Bayesian inference is it provides an estimate of its own uncertainty (in this case $\text{Var}[f(\mathbf{x}^*)]$)

2.4 Learning Hyperparameters

• Hyperparameters in the Bayesian Framework

- As I have said many times to get machine learning to work it is vital to choose the hyperparameter properly (we will see this is true for GPs)
- In the Bayesian framework if ϕ are hyperparameters then Bayes' rules says

$$p(\mathbf{x}|\mathcal{D}, \phi) = \frac{p(\mathcal{D}|\mathbf{x}, \phi) p(\mathbf{x}|\phi)}{p(\mathcal{D}|\phi)}$$

- * this is just Bayes' rule with everything conditioned on the hyperparameters ϕ
- We can select hyperparameters by considering the **evidence**, $p(\mathcal{D}|\phi)$
 - * when the hyperparameters are viewed as different models this is known as **model selection**
 - * this is also called the evidence framework
 - * We can view the relative likelihood of one model, ϕ_1 compare to a second, ϕ_2 by examining

$$\frac{p(\mathcal{D}|\phi_1)}{p(\mathcal{D}|\phi_2)}$$

- If we want to be hyper-Bayesian then we can put a prior, $p(\phi)$, over our hyperparameters and then calculate the joint posterior for the parameters of the likelihood θ and the hyper-parameters ϕ

$$p(\theta, \phi|\mathcal{D}) = \frac{p(\mathcal{D}|\theta, \phi) p(\theta|\phi) p(\phi)}{p(\mathcal{D})}$$

- * to compute the posterior we are interested in we marginalise out the hyperparameters

$$p(\theta|\mathcal{D}) = \int p(\theta, \phi|\mathcal{D}) d\phi$$

- * Often this integral is not computable in closed form and we are forced to estimate it using Monte-Carlo techniques

• Hyperparameter for Gaussian Processes

- For Gaussian Processes this means choosing the right kernel function

- Here we are in a better position than in SVMs in that the kernel represents the covariance function
- Given data we could estimate the kernel function using

$$\mathbb{E}[y_i, y_j] = k(\mathbf{x}_i, \mathbf{x}_j) + \sigma^2 \delta_{ij}$$

- It would be an inverse problem to estimate $k(\mathbf{x}_i, \mathbf{x}_j)$
- Often we start with a guess of the form of the kernel
- A very common kernel function is the Gaussian kernel

$$k(\mathbf{x}, \mathbf{y}) = e^{-\frac{\|\mathbf{x}-\mathbf{y}\|^2}{2\ell^2}}$$

- Now ℓ is a *scale parameter* that has to be determined
- However, a great advantage of Gaussian Processes is that we can compute the evidence in closed form

$$\log(p(\mathcal{D}|\boldsymbol{\theta})) = -\frac{1}{2}\mathbf{y}^\top(\mathbf{K} + \sigma^2\mathbf{I})^{-1}\mathbf{y} - \frac{1}{2}\log(|\mathbf{K} + \sigma^2\mathbf{I}|) - \frac{m}{2}\log(2\pi)$$

- * This allows us to rapidly compare different hyperparameters (kernels and noise level)
- * We are not doing this in a fully Bayesian way (we are maximising our evidence rather than updating a distribution for the hyperparameters) therefore we could overfit
- * However, usually we have relatively few hyperparameters so overfitting is less severe

3 Exercises

3.1 Working with Gaussians

This is optional. Learning to work with Gaussians allows you to understand a lot of machine learning techniques, but it is not going to be examined.

1. Integrals involving Gaussians can be done in closed form. This means that they are heavily used in machine learning. It does, however, take a lot of practice to learn how to do these integrals. The starting point is the integral

$$I_1 = \int_{-\infty}^{\infty} e^{-x^2/2} dx = \sqrt{2\pi}.$$

This integral isn't easy. The trick is to turn this into a two dimensional integral and perform the integral in polar coordinates. Have a go.

2. Show by a change of variables

$$I_2 = \int_{-\infty}^{\infty} e^{-x^2/(2\sigma^2)} dx = \sqrt{2\pi}\sigma.$$

3. Show that

$$I_3 = \int_{-\infty}^{\infty} e^{-x^2/2+ax} \frac{dx}{\sqrt{2\pi}} = e^{a^2/2}$$

This involves completing the square and using a change of variables. It is probably the most frequently used trick working with Gaussian integrals.

4. Show that

$$I_4 = \int_{-\infty}^{\infty} \cdots \int_{-\infty}^{\infty} e^{-\frac{1}{2}\|\mathbf{x}\|^2} d\mathbf{x} = (2\pi)^{\frac{n}{2}}$$

Often we don't bother writing all the integral signs (one will do)

5. By writing $\mathbf{M} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^T$ show that

$$I_5 = \int_{-\infty}^{\infty} e^{-\frac{1}{2}\mathbf{x}^T\mathbf{M}\mathbf{x}} d\mathbf{x} = (2\pi)^{\frac{n}{2}} \frac{1}{\sqrt{|\mathbf{M}|}}$$

where $|\mathbf{M}|$ is the determinant of $\mathbf{M}$. Note that $|\mathbf{M}| = |\mathbf{\Lambda}| = \prod_i \lambda_i$.

6. Show that

$$I_6 = \int_{-\infty}^{\infty} e^{-\frac{1}{2}\mathbf{x}^T\mathbf{M}^{-1}\mathbf{x}} d\mathbf{x} = (2\pi)^{\frac{n}{2}} \sqrt{|\mathbf{M}|}$$

3.2 Working with Matrix Inverses

- Modelling using normal (aka Gaussian) distributions is very powerful because we can usually computer everything in closed form—integrals involving Gaussians are always doable
- However, there is a twist which makes it extremely painful
 - The normal distribution uses the inverse of the covariance matrix
 - The inverse of matrices are a pain to work with
- To get nice results we often consider partition matrices into four blocks—the inverse of the full matrix can be written in terms of the inverse of each block
- Let's look at a very simple example
 - Show that

$$\begin{pmatrix} \mathbf{K} & \ell \\ \ell^T & m \end{pmatrix}^{-1} = \begin{pmatrix} \left(\mathbf{K} - \frac{\ell\ell^T}{m}\right)^{-1} & -\frac{\mathbf{K}^{-1}\ell}{m - \ell^T\mathbf{K}^{-1}\ell} \\ -\frac{\ell^T\mathbf{K}^{-1}}{m - \ell^T\mathbf{K}^{-1}\ell} & \frac{1}{m - \ell^T\mathbf{K}^{-1}\ell} \end{pmatrix}$$

* $\mathbf{K}$ is a symmetric matrix, ℓ is a vector and m a scalar

* Similar identities exist when ℓ and m are matrices, but let's keep thing simple

- Now we can show this by multiply both sides by $\begin{pmatrix} \mathbf{K} & \ell \\ \ell^T & m \end{pmatrix}$ (the algebra is easier if we multiply on the right)
- This is still very fiddly to show, so break it down
 - * Show that

$$\begin{pmatrix} \mathbf{A} & \mathbf{b} \\ \mathbf{b}^T & c \end{pmatrix} \begin{pmatrix} \mathbf{K} & \ell \\ \ell^T & m \end{pmatrix} = \begin{pmatrix} \mathbf{A}\mathbf{K} + \mathbf{b}\ell^T & \mathbf{A}\ell + m\mathbf{b} \\ \mathbf{b}^T\mathbf{K} + c\ell^T & \mathbf{b}^T\ell + cm \end{pmatrix}$$

* Now you have to show that when $\mathbf{A} = \left(\mathbf{K} - \frac{\ell\ell^T}{m}\right)^{-1}$, $\mathbf{b} = -\frac{\mathbf{K}^{-1}\ell}{m - \ell^T\mathbf{K}^{-1}\ell}$ and $c = \frac{1}{m - \ell^T\mathbf{K}^{-1}\ell}$ that

1. $\mathbf{A}\mathbf{K} + \mathbf{b}\ell^T = \mathbf{I}$
2. $\mathbf{A}\ell + m\mathbf{b} = \mathbf{0}$
3. $\mathbf{b}^T\mathbf{K} + c\ell^T = \mathbf{0}^T$
4. $\mathbf{b}^T\ell + cm = 1$

* To show 1. and 2. take out $\mathbf{A}$ as a common factor using $\mathbf{I} = \mathbf{A}\mathbf{A}^{-1}$ (answer below)

4 Experiments

4.1 Generate Samples from a Gaussian Processes

- To generate a Gaussian Process, $\mathcal{GP}(0, k)$, with $m(\mathbf{x}) = 0$ and covariance

$$k(\mathbf{x}, \mathbf{y}) = e^{-\frac{\|\mathbf{x}-\mathbf{y}\|^2}{2\ell^2}}$$

a points $\{\mathbf{x}_i | i = 1, 2, \dots, n\}$

1. We use first compute the covariance matrix $\mathbf{K}$ with elements $K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$
2. Compute the *Cholesky decomposition* $\mathbf{L}$ such that $\mathbf{K} = \mathbf{L}\mathbf{L}^\top$
 - For any positive definite matrices we can always compute a Cholesky decomposition (although beware, due to numerical rounding some matrices that should be positive definite, appear not to have Cholesky decompositions—you can $\epsilon \mathbf{I}$ to your matrix to help)
 - The Cholesky decomposition is a lower diagonal matrix
 - It is used to efficiently solve linear problems involving positive definite matrices (it is more efficient and more stable than Gaussian elimination)
3. Now let $\mathbf{f} = \mathbf{L}\boldsymbol{\eta}$ where $\boldsymbol{\eta} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
 - We note that

$$\mathbb{E}[\mathbf{f}\mathbf{f}^\top] = \mathbb{E}[\mathbf{L}\boldsymbol{\eta}\boldsymbol{\eta}^\top\mathbf{L}^\top] = \mathbf{L}\mathbb{E}[\boldsymbol{\eta}\boldsymbol{\eta}^\top]\mathbf{L}^\top = \mathbf{L}\mathbf{I}\mathbf{L}^\top = \mathbf{L}\mathbf{L}^\top = \mathbf{K}$$

- Thus $\mathbf{f}$ can be viewed as samples from a Gaussian Process
 - Draw many different samples and plot them
4. Experiment with different values of ℓ

1-d Gaussian Processes

```
x = [-10:0.2:10];    % define some points
n = length(x)
l = 1.0               % define length scale
K = zeros(n,n);       % define holder
for i = 1:n
    K(i,i) = 1;
    for j = 1:i
        K(j,i) = K(i,j) = exp(-(x(i)-x(j))^2/(2*l)); % define covariance matrix
    endfor
endfor
L = chol(K+0.0001*eye(n)); % cheat to persuade octave K is positive definite
f1 = L*randn(n,1);
f2 = L*randn(n,1);
f3 = L*randn(n,1);
plot(x,f1,x,f2,x,f3)
```

2-d Gaussian Process

```
range = [-4:0.2:4];    % define some points
n = length(range)
X = zeros(n*n,2);
for i = 1:n
    for j = 1:n
        X((i-1)*n+j,1) = range(i);
```

```

X((i-1)*n+j,2) = range(j);
endfor
endfor
l = 1.0 % define length scale
N = length(X)
K = zeros(N,N);
for i = 1:N
    K(i,i) = 1;
    for j = 1:i
        K(j,i) = K(i,j) = exp(-norm(X(i,:)-X(j,:))^2/(2*l)); % define covariance matrix
    endfor
endfor

L = chol(K+0.0001*eye(N)); % cheat to persuade octave K is positive definite
f = L*randn(N,1);
fm = reshape(f,n,n);
[xx, yy] = meshgrid(range, range);
mesh(xx, yy, fm);
xlabel ("x");
ylabel ("y");
zlabel ("f(x,y)");
title ("2-d Gaussian Process");

```

5 Answers

5.1 Working with Gaussians

1. The Gaussian Integral.

- We consider the two dimensional integral

$$\begin{aligned}
 I_1^2 &= \left(\int_{-\infty}^{\infty} e^{-x^2/2} dx \right)^2 \stackrel{(a)}{=} \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} e^{-x^2/2-y^2/2} dx dy \\
 &\stackrel{(b)}{=} \int_0^{\infty} \int_0^{2\pi} r e^{-r^2} dr d\theta \stackrel{(c)}{=} \int_0^{\infty} r e^{-r^2} dr \int_0^{2\pi} d\theta = 2\pi
 \end{aligned}$$

(a) Changing the name of the dummy index from x to y

(b) Using the change of variable $x = r \cos(\theta)$ and $y = r \sin(\theta)$ so that $dx dy = r dr d\theta$

(c) Reordering the integrals and using $\int_0^{\infty} r e^{-r^2} dr = 1$ (which you can prove by a change of variables $u = r^2/2$ so that $r dr = du$)

Therefore $I_1 = \sqrt{2\pi}$.

2.

$$I_2 = \int_{-\infty}^{\infty} e^{-x^2/(2\sigma^2)} dx \stackrel{(a)}{=} \sigma \int_{-\infty}^{\infty} e^{-u^2/2} du \stackrel{(b)}{=} \sqrt{2\pi} \sigma.$$

(a) Using $u = x/\sigma$ so that $dx = \sigma du$

(b) Using I_1

3.

$$\begin{aligned}
 I_2 &= \int_{-\infty}^{\infty} e^{-x^2/2+ax} \frac{dx}{\sqrt{2\pi}} \stackrel{(a)}{=} \int_{-\infty}^{\infty} e^{-\frac{1}{2}(x-a)^2 + \frac{a^2}{2}} \frac{dx}{\sqrt{2\pi}} \\
 &\stackrel{(b)}{=} e^{a^2/2} \int_{-\infty}^{\infty} e^{-\frac{1}{2}(x-a)^2} \frac{dx}{\sqrt{2\pi}} \stackrel{(c)}{=} e^{a^2/2}
 \end{aligned}$$

- (a) Using $-x^2/2 + ax = -\frac{1}{2}(x-a)^2 + \frac{a^2}{2}$
 (b) Factoring out $e^{a^2/2}$
 (c) Making a change of variables $u = x - a$ and using integral I_1

4.

$$\begin{aligned} I_4 &= \int_{-\infty}^{\infty} \cdots \int_{-\infty}^{\infty} e^{-\frac{1}{2}\|\mathbf{x}\|^2} d\mathbf{x} \stackrel{(a)}{=} \int_{-\infty}^{\infty} \cdots \int_{-\infty}^{\infty} e^{-\frac{1}{2} \sum_{i=1}^n x_i^2} \prod_{i=1}^n dx_i \\ &\stackrel{(b)}{=} \prod_{i=1}^n \left(\int_{-\infty}^{\infty} e^{-\frac{x_i^2}{2}} dx_i \right) \stackrel{(c)}{=} (2\pi)^{\frac{n}{2}} \end{aligned}$$

- (a) Using $\|\mathbf{x}\|^2 = \sum_{i=1}^n x_i^2$
 (b) Using $\exp(\sum_i a_i) = \prod_i e^{a_i}$ and factoring out the different terms
 (c) Using integral I_1

5.

$$\begin{aligned} I_5 &= \int_{-\infty}^{\infty} e^{-\frac{1}{2}\mathbf{x}^T \mathbf{M} \mathbf{x}} d\mathbf{x} \stackrel{(a)}{=} \int_{-\infty}^{\infty} e^{-\frac{1}{2}\mathbf{x}^T \mathbf{V} \mathbf{\Lambda} \mathbf{V}^T \mathbf{x}} d\mathbf{x} \\ &\stackrel{(b)}{=} \int_{-\infty}^{\infty} e^{-\frac{1}{2}\mathbf{u}^T \mathbf{\Lambda} \mathbf{u}} d\mathbf{u} \stackrel{(c)}{=} \int_{-\infty}^{\infty} e^{-\frac{1}{2} \sum_i \lambda_i u_i^2} d\mathbf{u} \\ &\stackrel{(d)}{=} \prod_i \left(\int_{-\infty}^{\infty} e^{-\frac{1}{2} \lambda_i u_i^2} du_i \right) \stackrel{(e)}{=} \prod_i \left(\sqrt{\frac{2\pi}{\lambda_i}} \right) \stackrel{(f)}{=} (2\pi)^{n/2} \frac{1}{\sqrt{\prod_i \lambda_i}} \stackrel{(g)}{=} \frac{(2\pi)^{\frac{n}{2}}}{\sqrt{|\mathbf{M}|}} \end{aligned}$$

- (a) Using $\mathbf{M} = \mathbf{V} \mathbf{\Lambda} \mathbf{V}^T$ where
 (b) Writing $\mathbf{u} = \mathbf{V}^T \mathbf{x}$. Note that $d\mathbf{u} = |\mathbf{V}^T| d\mathbf{x}$, but $\mathbf{V}$ is an orthogonal matrix so $|\mathbf{V}^T| = 1$. So $d\mathbf{u} = d\mathbf{x}$.
 (c) $\mathbf{\Lambda}$ is a diagonal matrix with elements $\Lambda_{ii} = \lambda_i$
 (d) Rearranging
 (e) Using integral I_2 with $\sigma = 1/\sqrt{\lambda_i}$
 (f) Rearranging
 (g) Using $\prod_i \lambda_i = |\mathbf{M}|$.

6. This follows using I_5 because $|\mathbf{M}^{-1}| = \frac{1}{|\mathbf{M}|}$.

5.2 Working with Matrix Inverses

- Using $\mathbf{A} = \left(\mathbf{K} - \frac{\ell \ell^T}{m} \right)^{-1}$, $\mathbf{b} = -\frac{\mathbf{K}^{-1} \ell}{m - \ell^T \mathbf{K}^{-1} \ell}$ and $c = \frac{1}{m - \ell^T \mathbf{K}^{-1} \ell}$, we show the four identities

1.

$$\begin{aligned} \mathbf{A} \mathbf{K} + \mathbf{b} \ell^T &= \mathbf{A} (\mathbf{K} + \mathbf{A}^{-1} \mathbf{b} \ell^T) \\ &= \left(\mathbf{K} - \frac{\ell \ell^T}{m} \right)^{-1} \left(\mathbf{K} + \left(\mathbf{K} - \frac{\ell \ell^T}{m} \right) \frac{(-\mathbf{K}^{-1} \ell \ell^T)}{m - \ell^T \mathbf{K}^{-1} \ell} \right) \\ &= \left(\mathbf{K} - \frac{\ell \ell^T}{m} \right)^{-1} \left(\mathbf{K} - \frac{\ell \ell^T}{m - \ell^T \mathbf{K}^{-1} \ell} + \frac{\ell \ell^T \mathbf{K}^{-1} \ell \ell^T / m}{m - \ell^T \mathbf{K}^{-1} \ell} \right) \\ &= \left(\mathbf{K} - \frac{\ell \ell^T}{m} \right)^{-1} \left(\mathbf{K} - \frac{\ell \ell^T}{m} \frac{m - \ell^T \mathbf{K}^{-1} \ell}{m - \ell^T \mathbf{K}^{-1} \ell} \right) \\ &= \left(\mathbf{K} - \frac{\ell \ell^T}{m} \right)^{-1} \left(\mathbf{K} - \frac{\ell \ell^T}{m} \right) = \mathbf{I} \end{aligned}$$

2.

$$\begin{aligned}
\mathbf{A}\ell + m\mathbf{b} &= \mathbf{A}(\ell + m\mathbf{A}^{-1}\mathbf{b}) \\
&= \mathbf{A}\left(\ell - m\left(\mathbf{K} - \frac{\ell\ell^\top}{m}\right)\frac{\mathbf{K}^{-1}\ell}{m - \ell^\top\mathbf{K}^{-1}\ell}\right) \\
&= \mathbf{A}\left(\ell - \frac{m\ell}{m - \ell^\top\mathbf{K}^{-1}\ell}\left(1 - \frac{\ell^\top\mathbf{K}^{-1}\ell}{m}\right)\right) \\
&= \mathbf{A}(\ell - \ell) = \mathbf{0}
\end{aligned}$$

3.

$$\mathbf{b}^\top\mathbf{K} + c\ell^\top = -\frac{\ell^\top}{m - \ell^\top\mathbf{K}^{-1}\ell} + \frac{\ell^\top}{m - \ell^\top\mathbf{K}^{-1}\ell} = \mathbf{0}^\top$$

4.

$$\mathbf{b}^\top\ell + cm = -\frac{\ell^\top\mathbf{K}^{-1}\ell}{m - \ell^\top\mathbf{K}^{-1}\ell} + \frac{m}{m - \ell^\top\mathbf{K}^{-1}\ell} = 1$$

Advanced Machine Learning Subsidiary Notes

Lecture 23: Probabilistic Inference

Adam Prügel-Bennett

February 9, 2024

1 Keywords

- Hierarchical Models, Mixture of Gaussians, Expectation Maximisation

2 Main Points

2.1 Laws of Probability

- We quickly review probabilities (you should know all this)
- Probabilities and events
 - We typically associate probabilities with events
 - The probability of an event lie between 0 and 1
 - If the set of events $\mathcal{E}$ are exhaustive and mutually exclusive then

$$\sum_{A \in \mathcal{E}} \mathbb{P}[A] = 1$$

- Often we associate numbers to events
 - These are known as *random variables*
 - Conventionally random variables are denoted by capitals, while we use lower-case letters to represent the value the random variable takes
 - We associate probability distributions to random variables
 - For discrete random variables these are known as *probability mass functions* and are denoted $\mathbb{P}[X = x]$
 - When our events are continuous we often associate the outcome to a continuous random variable
 - In this case the probability of a continuous random variable taking a particular value is typically 0
 - We then look at probability densities

$$f_X(x) = \lim_{\delta x \rightarrow 0} \frac{\mathbb{P}[x \leq X \leq x + \delta x]}{\delta x}$$

- * Densities are not probabilities (they can be greater than 1 although

$$\mathbb{P}[a \leq X \leq b] = \int_a^b f_X(x) \, dx$$

is a probability and is always less than or equal to 1)

- Probabilities become interesting when we have multiple events
 - The *joint probability* of both event A and B occurring is denoted $\mathbb{P}[A, B]$
 - The probability of random variables X and Y taking values x and y is denoted by
 - * $\mathbb{P}[X = x, Y = y]$ for discrete random variables or
 - * $f_{X,Y}(x, y)$ for continuous random variables (where this is now a probability density)
 - * sometimes we write $\mathbb{P}[X, Y]$ or $f(x, y)$ when the context is clear
 - The *conditional probability* of an event A happening given and event B has happened is denoted by $\mathbb{P}[A|B]$ for discrete random variables or $f_{X|Y}(x|y)$ for continuous random variables
 - * sometimes we write $\mathbb{P}[X|Y]$ or $f(x|y)$ when the context is clear
 - * conditional probability doesn't imply any causation
 - * Note that $\mathbb{P}[X|Y]$ or $f(x|y)$ are probabilities or densities of X

$$\sum_x \mathbb{P}[X = x|Y = y] = 1 \quad \int f(x|y) dx = 1$$

- * But they are not probabilities or densities of Y
- One of the most important rules in probabilities is
 - * $\mathbb{P}[X, Y] = \mathbb{P}[X|Y] \mathbb{P}[Y] = \mathbb{P}[Y|X] \mathbb{P}[X]$
 - * $f(x, y) = f(x|y) f(y) = f(y|x) f(x)$
 - * Clearly this is where Bayes' rule comes from
- A second rule that we use all the time is

$$\sum_y \mathbb{P}[X, Y = y] = \mathbb{P}[X] \quad \int f(x, y) dy = f(x)$$

- All this generalises to more the two random variables
 - * $\mathbb{P}[X = x, Y = y|Z = z]$ is the probability that both $X = x$ and $Y = y$ given that $Z = z$
 - * $\mathbb{P}[X = x|Y = y, Z = z]$ is the probability that $X = x$ given that $Y = y$ and $Z = z$
- Random variables are *independent* of each other if $\mathbb{P}[X, Y] = \mathbb{P}[X] \mathbb{P}[Y]$
- Random variables X and Y are conditionally independent of each other given Z if

$$\mathbb{P}[X, Y|Z] = \mathbb{P}[X|Z] \mathbb{P}[Y|Z]$$

- We often consider *random vectors* $\mathbf{X} = (X_1, X_2, \dots, X_n)$ where each component is a random variable
- I am expecting to you to know this material

2.2 Probabilistic Inference

- Most probabilistic inference involves constructing a model of the underlying data generation process and using Bayes' rule or maximum likelihood to learn unknown parameters of the model
- In modelling physical processes it is often easier to specify conditional probabilities where there is some causal relationship
- **Discriminative Models**
 - Often in machine learning our goal is to learn the probability distribution $\mathbb{P}[Y|\mathbf{X}]$ where Y is our target and $\mathbf{X}$ is a data point

- We may parameterise this distribution with some parameters Θ and our task would be to learn these parameters based on training data

- **Generative Models**

- Surprisingly it is often easier to model the joint probability $\mathbb{P}[Y, \mathbf{X}]$
- This means that we model the process of both generating the targets and the feature vectors together
- These are known as *generative models* as they allow us to generate random examples
- We don't necessarily want to use them to generate random samples; it just makes the modelling process easier (although you need to get used to this as it feels counter-intuitive)
- We can use generative models to do discrimination since $\mathbb{P}[Y|\mathbf{X}] = \mathbb{P}[Y, \mathbf{X}] / \mathbb{P}[Y]$ where $\mathbb{P}[Y] = \sum_{\mathbf{X}} \mathbb{P}[Y, \mathbf{X}]$
- Examples of generative models include *Hidden Markov Models* and *Topic Models* (covered later)

- **Latent Variables**

- In building probabilistic models we often model quite complicated processes
- To do this we introduce intermediate processes described by random variables that we never observe
- These are known as **latent variable**
- Often our model will involve many different layers between the inputs $\mathbf{X}$ and targets Y : this construction is sometimes known as a *hierarchical model*

- **Difficulty of Bayes**

- Bayesian inference is difficult because for most likelihoods there is no conjugate prior and the posterior is a mess
- In this case it can be very difficult to compute the *evidence* or *marginal likelihood*

$$\mathbb{P}[\mathcal{D}] = \sum_{\Theta} \mathbb{P}[\mathcal{D}|\Theta] \mathbb{P}[\Theta]$$

or

$$f(\mathcal{D}) = \int f(\mathcal{D}|\theta) f(\theta) d\theta$$

- * this is hard when Θ takes on too many values (e.g. it might be a high dimensional vector or a continuous variable)
- One solution to this is to obtain samples from the posterior distribution (this approach uses Monte Carlo methods which are very powerful, but can be slow)
- Another approach is to seek the **maximum a-posteriori** or **MAP** solution

$$\theta_{\text{MAP}} = \underset{\theta}{\operatorname{argmax}} f(\mathcal{D}|\theta) f(\theta) = \underset{\theta}{\operatorname{argmax}} \log(f(\mathcal{D}|\theta)) + \log(f(\theta))$$

- * this is much easier than the full Bayesian approach as we don't need to compute the marginal likelihood $f(\mathcal{D})$
- * but it isn't really Bayesian (although some users will claim it is)
- * it throws away the posterior and replaces this with its mode
- * we have lost a measure of uncertainty
- We can go one step further and assume a uniform prior
 - * This leads to the **maximum likelihood estimate**
 - * This was first proposed by Ronald Fisher in the time when Bayesian inference was considered taboo
 - * Despite its strong connection to Bayesian inference it was accepted by the statistical community

2.3 Mixtures of Gaussians

- To illustrate latent variables and a simple hierarchical model we consider a classic probabilistic model known as *mixture of Gaussians*
- We consider a concrete scenario
- We suppose we are observing the decay of two types (A and B) of short-lived particles ¹
- We can measure their half lives, X_i , but we don't know the type of particle
- We have a measurement error of the half-life
- Let $Z_i \in \{0, 1\}$ equal 1 if particle i is of type A and 0 if it is of type B
- The probability distribution of the half-life measurement is therefore

$$f(X_i|Z_i, \Theta) = Z_i \mathcal{N}(X_i|\mu_A, \sigma_A^2) + (1 - Z_i) \mathcal{N}(X_i|\mu_B, \sigma_B^2)$$

- where μ_A and μ_B are the expected half-lives for particles of type A and B respectively
- σ_A and σ_B are the standard deviations in the measurements
- this says that if the i^{th} particle is of type A then the probability of X_i is $\mathcal{N}(X_i|\mu_A, \sigma_A^2)$, while if it of type B , then X_i is distributed according to $\mathcal{N}(X_i|\mu_B, \sigma_B^2)$
- We show some typical data from $m = 1\,000$ observations in Figure 1

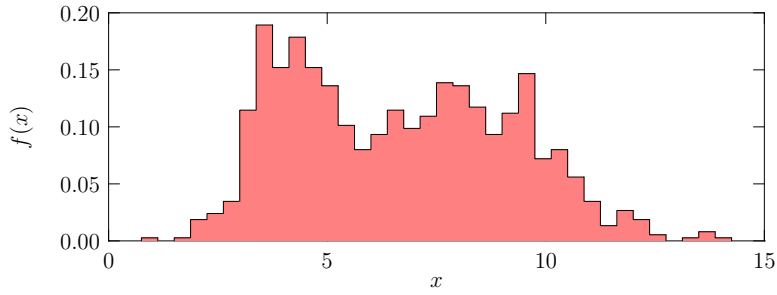


Figure 1: Example of data measuring the half-lives of two types of particles

- Our job is to infer the random variables $\Theta = (\mu_1, \mu_2, \sigma_1, \sigma_2, p)$, where $p = \mathbb{P}[Z_i = 1]$ is the probability of the particle being type A
- We can do a full Bayesian calculation, but let us just use maximum likelihood
- The maximum likelihood of the data $\mathcal{D} = \{X_i | i = 1, 2, \dots, m\}$ is

$$\begin{aligned} f(\mathcal{D}|\Theta) &\stackrel{(1)}{=} \sum_{\mathbf{Z} \in \{0,1\}^m} f(\mathcal{D}, \mathbf{Z}|\Theta) \\ &\stackrel{(2)}{=} \prod_{i=1}^m \sum_{Z_i \in \{0,1\}} f(X_i, Z_i|\Theta) \stackrel{(3)}{=} \prod_{i=1}^m \sum_{Z_i \in \{0,1\}} f(X_i|Z_i, \Theta) \mathbb{P}[Z_i] \end{aligned}$$

(1) where we marginalise out the latent variables $\mathbf{Z} = (Z_1, Z_2, \dots, Z_n)$

(2) we assume the data is independent

(3) we use the identity $f(X_i, Z_i|\Theta) = f(X_i|Z_i, \Theta) \mathbb{P}[Z_i]$

¹If you prefer, you can think of an autonomous vehicle using lidar where it detects reflections from two different, but close by, objects, A and B . We make multiple noisy measurements of the distance from the two objects.

- It is usually easier working with the log-likelihood

$$\begin{aligned}\log(f(\mathcal{D}|\Theta)) &= \sum_{i=1}^m \log(f(X_i|Z_i=1) \mathbb{P}[Z_i=1] + f(X_i|Z_i=0) \mathbb{P}[Z_i=0]) \\ &= \sum_{i=1}^m \log(p \mathcal{N}(X_i|\mu_A, \sigma_A) + (1-p) \mathcal{N}(X_i|\mu_B, \sigma_B))\end{aligned}$$

- We could do gradient descent on this, but it is an ugly expression to work with

2.4 Expectation Maximisation

- Rather than maximise the likelihood directly we can iteratively maximise the expected log-likelihood starting from some initial guess $\Theta^{(0)}$; we get an improved guess

$$\Theta^{(t+1)} = \underset{\Theta}{\operatorname{argmax}} \sum_{\mathbf{Z} \in \{0,1\}^m} \mathbb{P}[\mathbf{Z}|\mathcal{D}, \Theta^{(t)}] \log(f(\mathcal{D}, \mathbf{Z}|\Theta)) \quad (1)$$

- This is a general optimisation strategy that is regularly used when we have latent variables
- It is known as **expectation maximisation** or the **EM-algorithm**
- This looks very different to maximising the log-likelihood: it takes some effort to understand why this works
- To understand this we note

$$f(\mathcal{D}, \mathbf{Z}|\Theta) = f(\mathcal{D}|\mathbf{Z}, \Theta) \mathbb{P}[\mathbf{Z}|\Theta]$$

From which we can deduce (taking logs and rearranging)

$$\log(f(\mathcal{D}|\mathbf{Z}, \Theta)) = \log(f(\mathcal{D}, \mathbf{Z}|\Theta)) - \log(\mathbb{P}[\mathbf{Z}|\Theta])$$

- We now consider the probability distribution $\mathbb{P}[\mathbf{Z}|\mathcal{D}, \Theta^{(t)}]$, that tells us the probability that $Z_i = 1$ given X_i and the parameters $\Theta^{(t)}$ (this is different to the prior distribution $\mathbb{P}[\mathbf{Z}|\Theta^t] = p^{(t)}$)
- If we not take expectations of $\log(f(\mathcal{D}|\Theta))$ give above with respect to this distribution then

$$\begin{aligned}\log(f(\mathcal{D}|\Theta)) &= \mathbb{E}_{\mathbf{Z}|\Theta^{(t)}}[\log(f(\mathcal{D}, \mathbf{Z}|\Theta))] - \mathbb{E}_{\mathbf{Z}|\Theta^{(t)}}[\log(\mathbb{P}[\mathbf{Z}|\Theta])] \\ &= Q(\Theta|\Theta^{(t)}) + S(\Theta|\Theta^{(t)})\end{aligned}$$

- Note that the left-hand side does not involve the latent variables so when we take the expectation we get itself
- The first term on the right-hand side is

$$Q(\Theta|\Theta^{(t)}) = \mathbb{E}_{\mathbf{Z}|\Theta^{(t)}}[\log(f(\mathcal{D}, \mathbf{Z}|\Theta))] = \sum_{\mathbf{Z} \in \{0,1\}^m} \mathbb{P}[\mathbf{Z}|\mathcal{D}, \Theta^{(t)}] \log(f(\mathcal{D}|\mathbf{Z}, \Theta))$$

- This is the term we are optimising in equation (1)
- The second term is

$$S(\Theta|\Theta^{(t)}) = -\mathbb{E}_{\mathbf{Z}|\Theta^{(t)}}[\log(\mathbb{P}[\mathbf{Z}|\Theta])] = - \sum_{\mathbf{Z} \in \{0,1\}^m} \mathbb{P}[\mathbf{Z}|\mathcal{D}, \Theta^{(t)}] \log(\mathbb{P}[\mathbf{Z}|\Theta])$$

- Using the identity for the log-likelihood we can write the change in log-likelihood when we update our parameters

$$\begin{aligned}\Delta L &= \log(f(\mathcal{D}|\Theta^{(t+1)})) - \log(f(\mathcal{D}|\Theta^{(t)})) \\ &= Q(\Theta^{(t+1)}|\Theta^{(t)}) - Q(\Theta^{(t)}|\Theta^{(t)}) + S(\Theta^{(t+1)}|\Theta^{(t)}) - S(\Theta^{(t)}|\Theta^{(t)}) \\ &= Q(\Theta^{(t+1)}|\Theta^{(t)}) - Q(\Theta^{(t)}|\Theta^{(t)}) + \text{KL}\left(\mathbb{P}[Z|\Theta^{(t)}] \parallel \mathbb{P}[Z|\Theta^{(t+1)}]\right)\end{aligned}$$

– where

$$\begin{aligned}\text{KL}\left(\mathbb{P}[Z|\Theta^{(t)}] \parallel \mathbb{P}[Z|\Theta^{(t+1)}]\right) &= S(\Theta^{(t+1)}|\Theta^{(t)}) - S(\Theta^{(t)}|\Theta^{(t)}) \\ &= - \sum_{Z \in \{0,1\}^m} \mathbb{P}[Z|\mathcal{D}, \Theta^{(t)}] \log\left(\frac{\mathbb{P}[Z|\Theta^{(t+1)}]}{\mathbb{P}[Z|\Theta^{(t)}]}\right)\end{aligned}$$

– We have shown in a previous lecture that KL-divergences are non-negative

- Now in expectation maximisation we choose

$$\Theta^{(t+1)} = \underset{\Theta}{\operatorname{argmax}} Q(\Theta|\Theta^{(t)})$$

which implies $Q(\Theta^{(t+1)}|\Theta^{(t)}) \geq Q(\Theta^{(t)}|\Theta^{(t)})$

- Thus $\Delta L \geq 0$, so in each step we increase the log-likelihood
- This gives us a relative simple procedure for maximising the likelihood (we can also use this to maximise the *a posteriori* solution); we choose Θ to maximise

$$Q(\Theta|\Theta^{(t)}) = \sum_{Z \in \{0,1\}^m} \mathbb{P}[Z|\mathcal{D}, \Theta^{(t)}] \log(f(\mathcal{D}|Z, \Theta))$$

- Let us return to the problem of working out the half-life statistics of our two types of particles A and B

- Recall $f(\mathcal{D}, Z|\Theta) = \prod_{i=1}^m f(X_i|Z_i, \Theta) \mathbb{P}[Z_i]$ where

$$f(X_i, Z_i|\Theta) = p Z_i \mathcal{N}(X_i|\mu_A, \sigma_A^2) + (1-p)(1-Z_i) \mathcal{N}(X_i|\mu_B, \sigma_B^2)$$

- Let

$$\begin{aligned}p_i^{(t)} &= \mathbb{P}[Z_i = 1|X_i, \Theta^{(t)}] = \frac{p^{(t)} \mathcal{N}(X_i|\mu_A^{(t)}, \sigma_A^{2(t)})}{p^{(t)} \mathcal{N}(X_i|\mu_A^{(t)}, \sigma_A^{2(t)}) + (1-p^{(t)}) \mathcal{N}(X_i|\mu_B^{(t)}, \sigma_B^{2(t)})} \\ q_i^{(t)} &= \mathbb{P}[Z_i = 0|X_i, \Theta^{(t)}] = \frac{(1-p^{(t)}) \mathcal{N}(X_i|\mu_B^{(t)}, \sigma_B^{2(t)})}{p^{(t)} \mathcal{N}(X_i|\mu_A^{(t)}, \sigma_A^{2(t)}) + (1-p^{(t)}) \mathcal{N}(X_i|\mu_B^{(t)}, \sigma_B^{2(t)})} = 1 - p_i^{(t)}\end{aligned}$$

- Then

$$\begin{aligned}Q(\Theta|\Theta^{(t)}) &= \sum_{i=1}^m p_i^{(t)} \log(p^{(t)} \mathcal{N}(X_i|\mu_A, \sigma_A^2)) + q_i^{(t)} \log((1-p^{(t)}) \mathcal{N}(X_i|\mu_B, \sigma_B^2)) \\ &= \sum_{i=1}^m p_i^{(t)} \left(\log(p) - \frac{(X_i - \mu_A)^2}{2\sigma_A^2} - \frac{1}{2} \log(2\pi\sigma_A^2) \right) \\ &\quad + q_i^{(t)} \left(\log(1-p) - \frac{(X_i - \mu_B)^2}{2\sigma_B^2} - \frac{1}{2} \log(2\pi\sigma_B^2) \right)\end{aligned}$$

- To optimise this we just set the derivatives to 0

- Optimising with respect to p

$$\frac{\partial Q(\Theta|\Theta^{(t)})}{\partial p} = \frac{1}{p} \sum_{i=1}^m p_i^{(t)} - \frac{1}{1-p} \sum_{i=1}^m q_i^{(t)} = 0$$

solving for p

$$p^{(t+1)} = \frac{\sum_{i=1}^m p_i^{(t)}}{\sum_{i=1}^m (p_i^{(t)} + q_i^{(t)})} = \frac{1}{m} \sum_{i=1}^m p_i^{(t)}$$

- Optimising with respect to μ_A

$$\frac{\partial Q(\Theta|\Theta^{(t)})}{\partial \mu_A} = - \sum_{i=1}^m p_i^{(t)} \frac{X_i - \mu_A}{\sigma_A^2}$$

solving for μ_A (and performing a similar optimisation for μ_B)

$$\mu_A^{(t+1)} = \frac{\sum_{i=1}^m p_i^{(t)} X_i}{\sum_{i=1}^m p_i^{(t)}}, \quad \mu_B^{(t+1)} = \frac{\sum_{i=1}^m q_i^{(t)} X_i}{\sum_{i=1}^m q_i^{(t)}}$$

- Putting in the optimal value for $\mu_A^{(t)}$ and optimising with respect to σ_A^2

$$\frac{\partial Q(\Theta|\Theta^{(t)})}{\partial \sigma_A^2} = \frac{1}{2\sigma_A^4} \sum_{i=1}^m p_i^{(t)} (X_i - \mu_A^{(t)})^2 - \frac{1}{\sigma_A^2} \sum_{i=1}^m p_i^{(t)}$$

Solving for σ_A^2 (and performing a similar optimisation for σ_B^2)

$$\sigma_A^2 = \frac{\sum_{i=1}^m p_i^{(t)} (X_i - \mu_A^{(t)})^2}{\sum_{i=1}^m p_i^{(t)}}, \quad \sigma_B^2 = \frac{\sum_{i=1}^m q_i^{(t)} (X_i - \mu_B^{(t)})^2}{\sum_{i=1}^m q_i^{(t)}}$$

- These are very natural update equations
 - we make an estimate, $p_i^{(t)}$ of the probability that observation X_i is a particle of type A or B base on our current parameters
 - we then update all our parameters based on these estimates
- We are guaranteed that our EM-algorithm never decreases the likelihood (although it could reach a local rather than global optimum)
- For the data set we showed earlier (which was a random sample of size 1000 generated using $p = 0.3$, $\mu_A = 4$, $\sigma_A = 0.8$, $\mu_B = 8$ and $\sigma_B = 2$) we get the results shown in Figure 2
- The EM algorithm often leads to very natural update equations, but its convergence is often rather slow

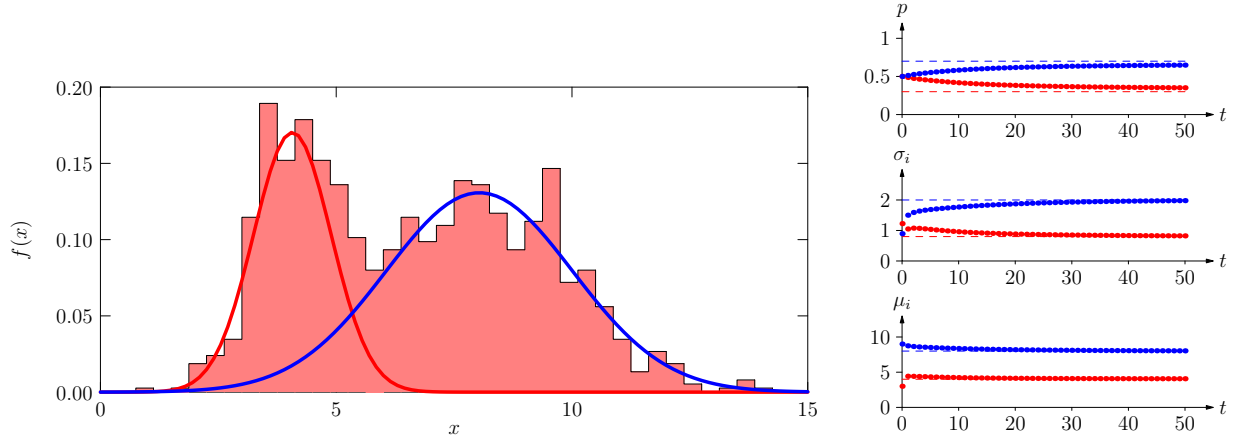


Figure 2: Example of EM algorithm to compute the statistics for the half-lives of our two particles

3 Exercises

3.1 Mysterious Disease

- We assume that we are tracking some disease
- Let $Z(t)$ be the number of people that catch the disease on day t , but this is unknown (a latent variable)
- We assume the rate of growth of the disease is

$$\mathbb{P}[Z(t+1)] = \text{Poi}\left(Z(t+1) \middle| \frac{r_0}{3} (Z(t) + Z(t-1) + Z(t-2))\right)$$

- We are assuming that someone is virulent for the first three days after catching the disease
- We assume $Z(1) = 1$ and $Z(0) = Z(-1) = 0$
- In expectation everyone with the disease will infect r_0 new people
- We observe $X(t)$ which is some proportion of new patients such that

$$\mathbb{P}[X(t) = k] = \text{Binom}(k|Z(t), p) = \binom{Z(t)}{k} p^k (1-p)^{Z(t)-k}$$

- We assume that each patient will be tested with a probability p
- We are given a time series $(X(1), X(2), \dots, X(T))$
- Build a probabilistic model to estimate p and k_0
- See answers

3.2 Experiments

3.3 Mysterious Disease

- Build a simulator of your models (assume you know p and r_0)
- Choose any language you are comfortable with
- If you are feeling very adventurous you could try to solve your model to predict p and r_0 , but be warned this is hard (you probably need to use MCMC, but you could try an EM algorithm)

```

r0 = 2
r = r0/3
p = 0.2
T = 20
Z(1) = 1;
Z(2) = poissrnd(r*Z(1));
Z(3) = poissrnd(r*(Z(1)+Z(2)));
for t = 4:T
    Z(t) = poissrnd(r*(Z(t-1)+Z(t-2)+Z(t-3)));
endfor

for t= 1:T
    X(t) = binornd(Z(t),p);
endfor

```

4 Answers

4.1 Mysterious Disease

- We want to compute $f(p, k_0 | \mathcal{D})$ where $\mathcal{D} = (X(1), X(2), \dots, X(T))$
- We have a likelihood of

$$\mathbb{P}[X(t) | Z(t), p] = \text{Binom}(X(t) | Z(t), p)$$

where

$$\mathbb{P}[Z(t+1) | r_0, Z(t), Z(t-1), Z(t-2)] = \text{Poi}\left(Z(t+1) \middle| \frac{r_0}{3} (Z(t) + Z(t-1) + Z(t-2))\right)$$

- To perform a Bayesian calculation we would have to put priors, $f(p)$ and $f(r_0)$, on p and r_0
 - A reasonable prior to use for p is $f(p) = \text{Beta}(p | a, b)$ —you could use $a = b = 0$ as an uninformative priors, but you might have some prior knowledge, e.g. $a = b = 2$ say which says $f(p) = 6p(1-p)$
 - A reasonable prior for r_0 would be $\text{Gamma}(r_0 | a, b)$ —you could use $a = b = 0$ as an uninformative prior but again you might have some prior belief, e.g. $a = 2, b = 1$
- Bayes rule tells us

$$f(p, r, \mathbf{Z} | \mathbf{X}) = \frac{\mathbb{P}[X(t) | Z(t), p] \prod_{t=1}^T \mathbb{P}[Z(t) | Z(t-1), Z(t-2), Z(t-3), r_0] f(r_0) f(p)}{\mathbb{P}[\mathbf{X}]}$$

- To get estimates of p and r_0 we have to marginalise out $\mathbf{Z}$
- Now the problem is this is rather horrible to compute (your priors are not conjugate priors in this problem and the posterior is very complicated)
- Probably the best way to do this is to use Markov Chain Monte Carlo (MCMC), but you will have to wait before I get there

Advanced Machine Learning Subsidiary Notes

Lecture 24: Generative Models

Adam Prügel-Bennett

February 9, 2024

1 Keywords

- Conditional Independence, Graphical models, LDA

2 Main Points

2.1 Overview

- We have so far considered building rather simple probabilistic models
- But what if we want to do inference on a more complicated problem, e.g.
 - We might want to write a fault diagnosis system for a car
 - Or we want to create an AI doctor
 - The model of the spread of a virus
- Here we have a vast number of random variables with complicated relationships between them
- To help design our system we can build a *graphical model* showing the causal relationships between random variables

2.2 Conditional Independence

- **Independence**

- Two random variable X and Y are independent if

$$\mathbb{P}[X, Y] = \mathbb{P}[X] \mathbb{P}[Y]$$

- Independence can significantly speed up calculations, e.g.

$$\mathbb{E}[X^2 Y] = \sum_{X,Y} X^2 Y \mathbb{P}[X, Y] = \left(\sum_X X^2 \mathbb{P}[X] \right) \left(\sum_Y Y \mathbb{P}[Y] \right)$$

- * If X and Y takes n and m values then without independence the double sum $\sum_{X,Y}$ would be over $n \times m$ possible values
- * With independence we can compute these sums independently so it just takes $m+n$ additions
- When we have large systems with many independent variables then the time saving is often the difference between calculations being feasible or infeasible
- Unfortunately in most complex systems there is likely to be some dependence between random variables

- **Conditional Independence**

- A weaker notion than full independence is *conditional independence*
- We say that X and Y are conditionally independent given Z if

$$\mathbb{P}[X, Y|Z] = \mathbb{P}[X|Z] \mathbb{P}[Y|Z]$$

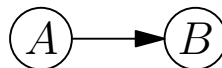
- Again this can lead to significant speed up in evaluating expectations, e.g.

$$\begin{aligned} \mathbb{E}[X^2 Y Z^3] &= \sum_{X,Y,Z} \mathbb{P}[X, Y, Z] X^2 Y Z^3 = \sum_{X,Y,Z} \mathbb{P}[X, Y|Z] \mathbb{P}[Z] X^2 Y Z^3 \\ &= \sum_Z Z^3 \left(\sum_X X^2 \mathbb{P}[X|Z] \right) \left(\sum_Y Y \mathbb{P}[Y|Z] \right) \end{aligned}$$

- * If X , Y and Z have l , m and n values respectively, then, ignoring conditional independence, this expectation would require $l \times m \times n$ additions; using conditional independence it only requires $n \times (l + m)$ additions
- Although conditional dependence doesn't imply causality, if random variables X and Y are not directly causally related they will be conditionally independent
- This is important prior information we can build into our model

2.3 Graphical Models

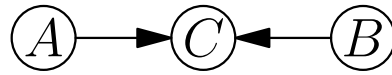
- In graphical models we represent each random variable as a node in a graph
- There are two main classes of graphical models
 - Bayesian Belief Networks
 - * use directed graphs
 - * we will spend most of our time discussing these
 - Markov Fields
 - * uses undirected graphs
 - * these are used in graphics a lot
 - * won't really discuss these
- Each causal connection we represent as a directed edge in a Bayesian Belief Network
 - if A directly influences B we represent this as



- **Cakes**

- We consider the following example
- Abi and Ben both bake cakes and like to bring them into the coffee room
- They do this randomly without consulting with each other
- Abi will bring in cakes 20% of the time: $\mathbb{P}[A = 1] = 0.2$
- Ben will bring in cakes 10% of the time: $\mathbb{P}[B = 1] = 0.1$
- 90% of the time if either Abi or Ben have put cakes in the coffee room there is some left when I enter $\mathbb{P}[C = 1|A = 1, B = 0] = \mathbb{P}[C = 1|A = 0, B = 1] = 0.9$
- If they both make cake then there is always cake left $\mathbb{P}[C = 1|A = 1, B = 1] = 1$
- If neither Abi or Ben has made cake there is still a 5% chance someone else has put cake in the coffee room $\mathbb{P}[C = 1|A = 0, B = 0] = 0.05$

- We note that $\mathbb{P}[C = 0|A, B] = 1 - \mathbb{P}[C = 1|A, B]$ as $\sum_{C \in \{0,1\}} \mathbb{P}[C|A, B] = 1$
- We can draw the causal relationships as



- This allows us to break down the joint probability as

$$\begin{aligned} \mathbb{P}[A, B, C] &\stackrel{(1)}{=} \mathbb{P}[C, B|A] \mathbb{P}[A] \\ &\stackrel{(2)}{=} \mathbb{P}[C|A, B] \mathbb{P}[B|A] \mathbb{P}[A] \stackrel{(3)}{=} \mathbb{P}[C|A, B] \mathbb{P}[B] \mathbb{P}[A] \end{aligned}$$

- (1) Using the definition of conditional probability (this is always true)
- (2) Using the definition of conditional probability again (this is always true)
- (3) Using the fact that B and A are independent (there is no arrow between them in the graphical representation) so $\mathbb{P}[B|A] = \mathbb{P}[B]$
 - * From the graphical representation we can immediately write down a simple form for this joint distribution
- We can use this decomposition to help us compute various probabilities
- (To compute probabilities we use the fact that the expectation of an indicator function $\llbracket \text{predicate} \rrbracket$ is equal to the probability $\mathbb{P}[\text{predicate}] = \mathbb{E}[\llbracket \text{predicate} \rrbracket]$
 - * The indicator function $\llbracket \text{predicate} \rrbracket$ equals 1 if the predicate is true and 0 otherwise)
- Let's compute the probability there are cakes

$$\mathbb{P}[C = 1] = \sum_{A, B, C \in \{0,1\}} \llbracket C = 1 \rrbracket \mathbb{P}[A, B, C] = \sum_{A, B \in \{0,1\}} \mathbb{P}[C = 1|A, B] \mathbb{P}[A] \mathbb{P}[B] = 0.303$$

- * See Section 3.1 for details of the calculation (this is an exercise that should really help)
- * Here we exhaustively sum over all variables
- Let us consider what happens when we observe a random variable
 - * In graphical models we often shade observed random variables



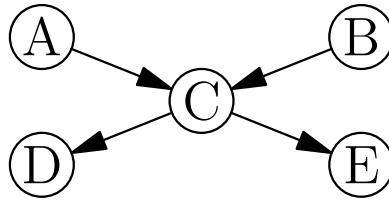
- * Let's compute quantities conditioned on an observation of C

$$\mathbb{P}[A, B|C] = \frac{\mathbb{P}[A, B, C]}{\mathbb{P}[C]}$$

- * Thus $\mathbb{P}[A, B|C = 1] = \mathbb{P}[A, B, C = 1] / \mathbb{P}[C = 1]$
- * Using this we can compute

$$\mathbb{P}[A = 1, B = 1|C = 1] = 0.066, \quad \mathbb{P}[A = 1|C = 1] = 0.630, \quad \mathbb{P}[B = 1|C = 1] = 0.317$$
- * We note that $\mathbb{P}[A = 1, B = 1|C = 1] \neq \mathbb{P}[A = 1|C = 1] \mathbb{P}[B = 1|C = 1]$
- * That is once we observe C then A and B are no longer independent
- We can extend our model further
 - * We suppose that Dave likes cakes so if there is a cake in the coffee room there is a 80% chance that I will see him eating a cake: $\mathbb{P}[D = 1|C = 1] = 0.8$
 - * Even if there are no cakes in the coffee room there is a 10% chance that Dave has bought his own cake: $\mathbb{P}[D = 1|C = 0] = 0.1$
 - * Eli also likes cakes: there is a 60% chance that I will see her eating cakes if there are cakes in the coffee room: $\mathbb{P}[E = 1|C = 1] = 0.6$

- * But she never buys herself cakes $\mathbb{P}[E = 1|C = 0] = 0$
- * We can depict the dependencies of the this large model



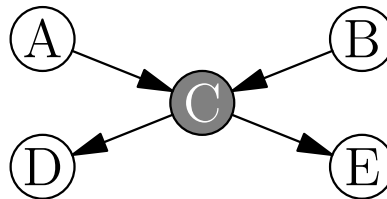
$$\begin{aligned}\mathbb{P}[A, B, C, D, E] &= \mathbb{P}[C, D, E|A, B] \mathbb{P}[B] \mathbb{P}[A] \\ &= \mathbb{P}[D|C] \mathbb{P}[E|C] \mathbb{P}[C|A, B] \mathbb{P}[B] \mathbb{P}[A]\end{aligned}$$

- where we have used the conditional independence
- note that D and E are conditionally independent of A and B given C
- That is, these probabilities will depend on events A and B , but once I know there are cakes in the coffee room it doesn't matter who put them there
- We can compute probabilities for this larger system

$$\mathbb{P}[D = 1] = 0.3121, \quad \mathbb{P}[E = 1] = 0.1818, \quad \mathbb{P}[D = 1, E = 1] = 0.14544$$

so $\mathbb{P}[D, E] \neq \mathbb{P}[D] \mathbb{P}[E]$

- D and E are not independent variables as they coupled through C
- However when we observe C



then $\mathbb{P}[D, E|C] \stackrel{(u)}{=} \mathbb{P}[D|C] \mathbb{P}[E|C]$

- E.g.

$$\mathbb{P}[D = 1|C = 1] = 0.8 \quad \mathbb{P}[E = 1|C = 1] = 0.6 \quad \mathbb{P}[D = 1, E = 1|C = 1] = 0.48$$

2.4 Latent Dirichlet Allocation

- Most probabilistic models can be represented as a graphical model
- There are times when this isn't particularly useful
- But it can just help us to understand what is going on
- We consider an example of this called **Latent Dirichlet Allocation**
 - This is sometimes known as LDA, but should not be confused with *linear discriminant analysis*
- LDA is used to model topics in a set of documents (or *corpus*)
- We want to identify a set of topics
- The topics are associated with particular words
- The documents will be associated with a small number of topics
- To model this we build a *generative model*

- This is natural to build
- Although it seems the wrong way around—we don't want to build a corpus of documents
- But Bayes's rule allows us to invert this

- Let us start with some definitions

- We consider generating a corpus of documents

$$\mathcal{C} = \{d_i | i = 1, 2, \dots, |\mathcal{C}|\}$$

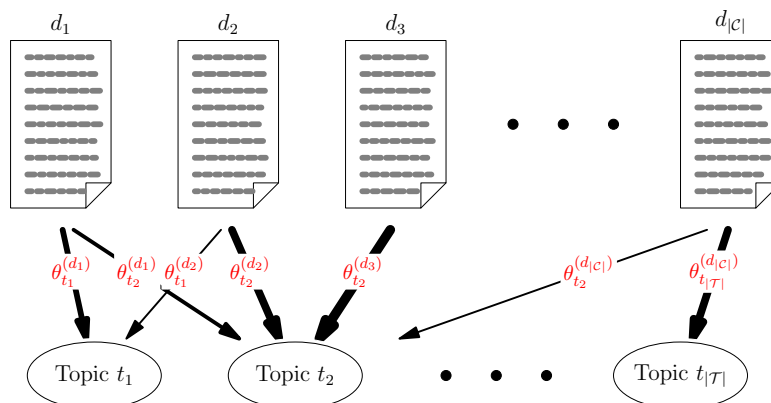
- Each document consists of a set of words

$$d = (w_1^{(d)}, w_2^{(d)}, \dots, w_{N_d}^{(d)})$$

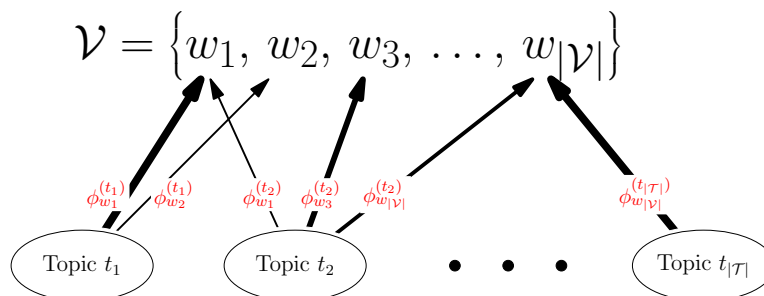
- We assume there is a set of topics

$$\mathcal{T} = \{t_1, t_2, \dots, t_{|\mathcal{T}|}\}$$

- We associate a probability, $\theta_t^{(d)}$, that a word in document d relates to a topic t



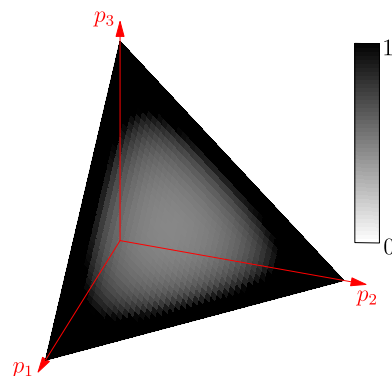
- We associate a probability $\phi_w^{(t)}$ that a word, w , is related to a topic t



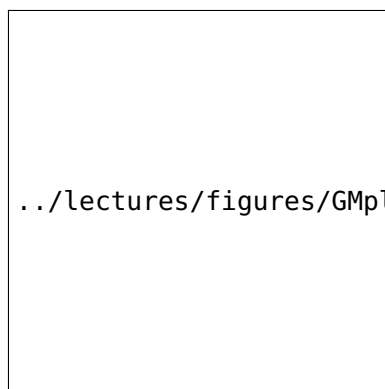
- Most documents are predominantly about a few topics and most topic have a small number of words associated to them
- We can generate probability vectors $\theta^{(d)}$ and $\phi^{(t)}$ from a Dirichlet distribution

$$\text{Dir}(\mathbf{p}|\boldsymbol{\alpha}) = \Gamma\left(\sum_i \alpha_i\right) \prod_{i=1}^n \frac{p_i^{\alpha_i-1}}{\Gamma(\alpha_i)}$$

- $\theta^{(d)} \sim \text{Dir}(\alpha \mathbf{1})$ and $\phi^{(t)} \sim \text{Dir}(\beta \mathbf{1})$
- By choosing a Dirichlet distribution with a small components, α_i , we ensure that have most of its probability density lies around the edges



- By drawing $\theta^{(d)}$ and $\phi^{(t)}$ from a Dirichlet distribution with small parameters α and β we ensure that most components are very small with a few large components
- To generate a document we choose a topic for each word and a word for each topic
- We use the categorical distribution
 - * if $\mathbf{p}$ is a vector of non-negative values that sum to 1 then $\text{Cat}(i|\mathbf{p}) = p_i$
 - * That is if $I \sim \text{Cat}(\mathbf{p})$ then I will be an integer, i with probability p_i
- Thus for word i of document d we first choose a topic $\tau_i^{(d)} \sim \text{Cat}(\theta^d)$ and then we choose a word $w_i^{(d)} \sim \text{Cat}(\phi^{\tau_i^{(d)}})$
 - * It is a slightly crazy model in that words are randomly chosen from the topics of the document with no ordering
- We could represent this by a rather ugly graphical model (see Figure 1)
- To make graphical models more manageable people have invented a graphical means of showing repeats
- For example, to illustrate that we have a probability vector $\theta^{(d)}$ drawn from a Dirichlet distribution with parameter α for each document d in our corpus $\mathcal{C}$ we can use a **plate diagram**



- Using a plate diagram we can represent the LDA as shown in Figure 2
- It takes a bit of time to decode this

../lectures/figures/LDAGM.pdf

Figure 1: Graphical Model for Latent Dirichlet Allocation

../lectures/figures/GMplateLDA.pdf

Figure 2: Graphical Model for Latent Dirichlet Allocation

- * We have a probability vector $\theta^{(d)}$ for every document in our corpus
 - this tells us the distribution of topics in the document
 - $\theta^{(d)}$ is drawn from a Dirichlet distribution with parameters $\alpha = (\alpha, \alpha, \alpha, \dots, \alpha)$
- * We have a probability $\phi^{(\tau)}$ for every topic
 - this tells us the distribution of words associated with a topic
 - $\phi^{(\tau)}$ is drawn from a Dirichlet distribution with parameters $\beta = (\beta, \beta, \beta, \dots, \beta)$
- * For each document, d , and each word, $w_i^{(d)}$ in the document we have
 - a topic $\tau_i^{(d)}$ drawn from $\theta^{(d)}$
 - the words $w_i^{(d)}$ are drawn from $\phi^{(\tau_i^{(d)})}$
 - that is it depends both on the topic $\tau_i^{(d)}$ and on the distributions of words associated with that topic
- * In practice I am usually given the documents with words (the words are observed)
- * I have shaded what is usually taken to be observed (for α and β we usually just choose these from the start—we could learn them so they would not be observed)
- The graphical model helps us write down the joint distribution
- We define matrices to denote all the variables

$$\begin{aligned}
 \mathbf{W} &= (\mathbf{w}^{(d)} | d \in \mathcal{C}) \quad \text{with} \quad \mathbf{w}^{(d)} = (w_1^{(d)}, w_2^{(d)}, \dots, w_{N_d}^{(d)}), \quad \text{and} \quad w_i^{(d)} \in \mathcal{V} \\
 \mathbf{T} &= (\tau_i^{(d)} | d \in \mathcal{C} \wedge i \in \{1, 2, \dots, N_d\}) \quad \text{with} \quad \tau_i^{(d)} \in \mathcal{T} \\
 \mathbf{\Theta} &= (\theta^{(d)} | d \in \mathcal{C}) \quad \text{with} \quad \theta^{(d)} = (\theta_t^{(d)} | t \in \mathcal{T}) \in \Lambda^{|\mathcal{T}|} \\
 \mathbf{\Phi} &= (\phi^{(t)} | t \in \mathcal{T}) \quad \text{with} \quad \phi^{(t)} = (\phi_w^{(t)} | w \in \mathcal{V}) \in \Lambda^{|\mathcal{V}|}
 \end{aligned}$$

- Then the joint distribution is given by

$$\mathbb{P}[\mathbf{W}, \mathbf{T}, \mathbf{\Theta}, \mathbf{\Phi} | \alpha, \beta] = \left(\prod_{t \in \mathcal{T}} \text{Dir}(\phi^{(t)} | \beta \mathbf{1}) \right) \left(\prod_{d \in \mathcal{C}} \text{Dir}(\theta^{(d)} | \alpha \mathbf{1}) \prod_{i=1}^{N_d} \text{Cat}(\tau_i^{(d)} | \theta^{(d)}) \text{Cat}(w_i^{(d)} | \phi^{(\tau_i^{(d)})}) \right)$$

- It is now a technical exercise to compute the quantities of interest
- For example $f(\mathbf{\Theta}, \mathbf{\Phi} | \mathbf{W}, \alpha, \beta)$ will tell us about the words associated with the topics that are present in the corpus and the topics associated with each document
- Note that we would marginalise out $\mathbf{T}$
- There are different techniques for computing these probabilities, e.g. using MCMC or variational approximations

3 Exercise

3.1 Cakes

- Write a program to compute the probability of various events concerning cakes
- To compute all the probabilities (sometimes inefficiently) we can sum over all values our variables can take
- I have done this somewhat inefficiently in the answers

4 Answers

4.1 Cakes

- I am using asymptote which I usually use for drawing diagrams, but its a language with C syntax
- Port this to a language of your choice

```
real pcGab(int a, int b, int c) { // P(C|A,B)
  real p;
  if (a==1 && b==1)
    p = 1;
  else if (a==1 || b==1)
    p = 0.95;
  else
    p = 0.05;
  if (c==1)
    return p;
  else
    return 1-p;
}
```

```
real pa(int a) { // P(A)
  return (a==1)? 0.2:0.8;
}
```

```
real pb(int b) { // P(B)
  return (b==1)? 0.1:0.9;
}
```

```
real pd(int d, int c) { // P(D|C)
  real p = (c==1)? 0.8:0.1;
  return (d==1)? p:1-p;
}
```

```
real pe(int e, int c) { // P(E|C)
  real p = (c==1)? 0.6:0;
  return (e==1)? p:1-p;
}
```

```
typedef real func(int, int, int, int, int); // define signature of general function
```

```
real expect(func f) { // compute expectations exhaustively
  real sum = 0;
  for (int a=0; a<=1; ++a) {
    for (int b=0; b<=1; ++b) {
      for (int c=0; c<=1; ++c) {
        for (int d=0; d<=1; ++d) {
          for (int e=0; e<=1; ++e) {
            sum += f(a,b,c,d,e)*pcGab(a,b,c)*pa(a)*pb(b)*pd(d,c)*pe(e,c);
          }
        }
      }
    }
  }
}
```

```

    }
  }
}
return sum;
}

/* Define functions to find expectations */
/* These are all indicator functions so I end up with probabilities */

real f(int a, int b, int c, int d, int e) {return 1;}
real fa(int a, int b, int c, int d, int e) {return a;}
real fb(int a, int b, int c, int d, int e) {return b;}
real fab(int a, int b, int c, int d, int e) {return a*b;}
real fc(int a, int b, int c, int d, int e) {return c;}
real fac(int a, int b, int c, int d, int e) {return a*c;}
real fbc(int a, int b, int c, int d, int e) {return b*c;}
real fabc(int a, int b, int c, int d, int e) {return a*b*c;}
real fd(int a, int b, int c, int d, int e) {return d;}
real fe(int a, int b, int c, int d, int e) {return e;}
real fde(int a, int b, int c, int d, int e) {return d*e;}
real fcd(int a, int b, int c, int d, int e) {return c*d;}
real fce(int a, int b, int c, int d, int e) {return c*e;}

real fcde(int a, int b, int c, int d, int e) {return c*d*e;}

write("Check joint probability is normalised: ", expect(f));
write("P(A=1) = ", expect(fa));
write("P(B=1) = ", expect(fb));
write("P(A=1)*P(B=1) = ", expect(fa)*expect(fb));
write("P(A=1,B=1) = ", expect(fab));
write("Note P(A=1,B=1) = P(A=1)*P(B=1)");
write("-");

real Pc = expect(fc);
write("P(C=1) = ", Pc);
real PaGc = expect(fac)/Pc;
real PbGc = expect(fbc)/Pc;
real PabGc = expect(fabc)/Pc;
write("P(A=1|C=1) = ", PaGc);
write("P(B=1|C=1) = ", PbGc);
write("P(A=1|C=1)*P(B=1|C=1) = ", PaGc*PbGc);
write("P(A=1,B=1|C=1) = ", PabGc);
write("Note: P(A=1,B=1|C=1) != P(A=1|C=1)*P(B=1|C=1)");
write("-");

write("P(D=1) = ", expect(fd));
write("P(E=1) = ", expect(fe));
write("P(D=1)*P(E=1) = ", expect(fd)*expect(fe));
write("P(D=1,E=1) = ", expect(fde));
write("Note: P(D=1,E=1) != P(D=1)*P(E=1)");
write("-");

```

```

write("P(D=1|C=1) = ", expect(fcd)/Pc);
write("P(E=1|C=1) = ", expect(fce)/Pc);
write("P(D=1|C=1)*P(E=1|C=1) = ", expect(fcd)/Pc*expect(fce)/Pc);
write("P(D=1,E=1|C=1) = ", expect(fcde)/Pc);
write("Note: P(D=1,E=1|C=1) = P(D=1|C=1)*P(E=1|C=1)");

```

4.2 Result from program

```

Check joint probability is normalised: 1
P(A=1) = 0.2
P(B=1) = 0.1
P(A=1)*P(B=1) = 0.02
P(A=1,B=1) = 0.02
Note P(A=1,B=1) = P(A=1)*P(B=1)
-
P(C=1) = 0.303
P(A=1|C=1) = 0.63036303630363
P(B=1|C=1) = 0.316831683168317
P(A=1|C=1)*P(B=1|C=1) = 0.19971898179917
P(A=1,B=1|C=1) = 0.066006600660066
Note: P(A=1,B=1|C=1) != P(A=1|C=1)*P(B=1|C=1)
-
P(D=1) = 0.3121
P(E=1) = 0.1818
P(D=1)*P(E=1) = 0.05673978
P(D=1,E=1) = 0.14544
Note: P(D=1,E=1) != P(D=1)*P(E=1)
-
P(D=1|C=1) = 0.8
P(E=1|C=1) = 0.6
P(D=1|C=1)*P(E=1|C=1) = 0.48
P(D=1,E=1|C=1) = 0.48
Note: P(D=1,E=1|C=1) = P(D=1|C=1)*P(E=1|C=1)

```

Advanced Machine Learning Subsidiary Notes

Lecture 25: MCMC

Adam Prügel-Bennett

February 9, 2024

1 Keywords

- Monte Carlo methods, MCMC, Variational Methods

2 Main Points

2.1 Monte Carlo

- Except in a few cases Bayesian inference is hard
 - It is easy when we have a conjugate prior giving us a simple posterior
 - But, when we want to model complicated situations our likelihood function usually doesn't have a conjugate prior
 - This means the posterior is not expressible as a simple probability distribution
 - Furthermore it is usually impossible to compute the marginal likelihood or evidence

$$\mathbb{P}[\mathcal{D}] = \sum_{\Theta} \mathbb{P}[\Theta|\mathcal{D}] \mathbb{P}[\Theta]$$

- The sum or integral over variables Θ is often too large to make this the feasible
- You could try to obtain a histogram for the posterior, but if Θ has many components the curse of dimensionality makes this difficult
- There is a simple answer which is to obtain samples from the posterior
 - We can use this to estimate expectations of any function, $g(\Theta)$, of Θ

$$\mathbb{E}_{\Theta|\mathcal{D}}[g(\Theta)] \approx \frac{1}{n} \sum_{i=1}^n g(\Theta_i)$$

- * Θ_i are random deviates drawn from the posterior $\mathbb{P}[\Theta_i|\mathcal{D}]$
- * (random deviates or random variates are values drawn from a probability distribution)
- * When the posterior distribution is well behaved (doesn't have a thick tail) then the error in the approximation fall off as $1/\sqrt{n}$
- The problem is you have to be able to sample random deviates from $P(\mathcal{D}|\Theta) f(\Theta)$
- For simple distributions there are two classic solutions to generating random deviates

1. Transformation Methods

- If we can compute the cumulative probability function

$$F_X(x) = \int_{-\infty}^x f(t) dt$$

and invert $F_X(x)$, then $X = F_X^{-1}(U)$ will be a random deviate with density f , where U is a uniform standard deviate between 0 and 1

- This only works for a few classic distributions

2. Rejection Method

- In the rejection methods we find another distribution $g_Y(y)$ that we can sample from that satisfies

$$c g_Y(x) > f_X(x)$$

- Then we generate a deviate $Y \sim g_Y$ and accept it with a probability $f_X(Y)/(c g_Y(Y))$
- The expected rejection rate is $c - 1$
- This is a more general method, but is efficient only if we have a good approximating function $g_Y(y)$ (good in the sense that c is not too large)
- Particularly for multivariate (i.e. high dimensional) random variables it is often very hard to find a good approximating function

2.2 Markov-Chain Monte Carlo (MCMC)

- Fortunately there is another method for generating random deviates which is very general
- This uses a Markov-Chain where we update an initial state of our system
- If we choose the dynamics correctly eventually we will reach a state that is correctly distributed
- Suppose we have a set of states, $\mathcal{S}$, and we want to choose a random sample from a distribution $\pi = (\pi_i | i \in \mathcal{S})$

- π_i is the probability of being in state i —we get to choose this (for Bayesian inference we will want to choose the posterior probability)

- Let M_{ij} be the transition probability from state j to state i
 - $\sum_i M_{ij} = 1$ (if we start any state, j say, we have to end up in another state)
 - $M_{ij} \geq 0$ (it is a probability)
 - Matrices of non-negative elements whose rows sum to one are known as **stochastic matrices**
- If we choose M_{ij} so that

$$M_{ij} \pi_j = M_{ji} \pi_i$$

- then after some time the probability of being in state i will be π_i (there are some mild conditions on $\mathbf{M}$ for this to be true)
- This condition is known as **detailed balance**
- Note that if we sum the detailed balance equation then

$$\sum_{j \in \mathcal{S}} M_{ij} \pi_j = \sum_{j \in \mathcal{S}} M_{ji} \pi_i = \pi_j$$

* since $\sum_j M_{ji} = 1$

- Writing this in matrix form

$$\mathbf{M}\pi = \pi$$

- * That is, π is an eigenvalue of $\mathbf{M}$ with eigenvalue 1
- Because $\mathbf{M}$ is a stochastic matrix all eigenvalues will be less than or equal to 1
- * To prove this let v be an eigenvector of $\mathbf{M}$ with eigenvalue λ

$$\sum_{j \in \mathcal{S}} M_{ij} v_j = \lambda v_i$$

taking the absolute value of both sides

$$\left| \sum_{j \in \mathcal{S}} M_{ij} v_j \right| = |\lambda v_i| = |\lambda| |v_i|$$

- * But

$$\left| \sum_{j \in \mathcal{S}} M_{ij} v_j \right| \leq \sum_{j \in \mathcal{S}} |M_{ij} v_j| = \sum_{j \in \mathcal{S}} M_{ij} |v_j|$$

where we have used $M_{ij} \geq 0$ (and essentially $|a + b| \leq |a| + |b|$)

- * Thus

$$\sum_{j \in \mathcal{S}} M_{ij} |v_j| \geq |\lambda| |v_i|$$

- * Summing both sides with respect to i

$$\begin{aligned} \sum_{i \in \mathcal{S}} \sum_{j \in \mathcal{S}} M_{ij} |v_j| &\geq |\lambda| \sum_{i \in \mathcal{S}} |v_i| \\ \sum_{j \in \mathcal{S}} \sum_{i \in \mathcal{S}} M_{ij} |v_j| &\geq |\lambda| \sum_{i \in \mathcal{S}} |v_i| \\ \sum_{j \in \mathcal{S}} |v_j| &\geq |\lambda| \sum_{i \in \mathcal{S}} |v_i| \end{aligned}$$

where we used $\sum_{i \in \mathcal{S}} M_{ij} = 1$

- * It follows that $\lambda \leq 1$
- * Let $p(t)$ be a vector whose elements $p_i(t)$ equal the probability of being in state i at time t
- * If the set of eigenvectors span the whole space, then any starting vector $p(0)$ can be written in terms of the eigenvectors

$$p(0) = \sum_i c_i v_i$$

(this is just a change of basis)

- * The probability vector after one step is given by

$$p(1) = \mathbf{M} p(0) = \mathbf{M} \sum_i c_i v_i = \sum_i c_i \mathbf{M} v_i = \sum_i c_i \lambda_i v_i$$

- * After t steps

$$p(t) = \mathbf{M}^t p(0) = \sum_i c_i \lambda_i^t v_i$$

- * If $|\lambda| < 1$ will then λ_i^t will shrink exponentially fast so that

$$\lim_{t \rightarrow \infty} p(t) = \sum_{i: \lambda_i = 1} c_i v_i$$

- * That is we converge onto the set of eigenvectors with eigenvalue 1
- * But if detailed balance is conserved then π satisfies this

- * There are some conditions to ensure that there is only one such eigenvector with eigenvalue 1
 - We have to ensure that we can get from any state to any other through some series of transitions
 - We have to prevent periodic behaviour (e.g. if our set of states were from 1 to n and we could only move from state i to state $i - 1$ or $i + 1$ then if we started on an odd state we would always be on an odd state after an even number of moves)
 - It is very easy to insure both conditions
- Now the problem is to choose the transition probabilities to satisfy detailed balance
- This is easy and can be done in different ways
- **Metropolis Algorithm**
 - A very easy way to achieve detailed balance is that starting from state i we choose a neighbouring state j
 - * We have to make sure the probability of choosing state i when in state j is the same as the probability of choosing state j when starting in state i
 - We now move to state j if
 - * $\pi_j \geq \pi_i$
 - * or we move anyway with a probability π_i/π_j
 - It is a simple exercise to show that this satisfies detailed balance
- **Metropolis-Hastings Algorithm**
 - Sometimes it is difficult to insure that choosing state i from state j is the same as choosing state j from state i
 - * As an example considering our states were non-negative integers, we move from state i to state $i - 1$ and $i + 1$ with probability $\frac{1}{2}$, but we have a problem at state 0 where we can't move to state $i - 1$
 - We can modify the Metropolis algorithm. Suppose $p(i|j)$ is the probability of choosing state i starting in state j then let $r = p(j|i) \pi_i / (p(i|j) \pi_j)$
 - * we accept the move if $r > 1$
 - * or with a probability r
 - * in practice we can just draw a random number U uniformly between 0 and 1 and accept the move if $U < r$
 - * this is the same as the Metropolis algorithm if $p(j|i) = p(i|j)$
- We can equally well apply this to (multi-dimensional) continuous variables, θ
 - If θ is our current set of parameters, we choose a new set of parameter θ' with probability $p(\theta'|\theta)$ and then accept this proposal if $r = (p(\theta|\theta') \pi(\theta')) / (p(\theta'|\theta) \pi(\theta)) \geq 0$ or with a probability r
 - We now choose $p(\theta'|\theta)$ so that with high probability θ' is close to θ , this ensures that there is a reasonable high acceptance rate
- For Bayes calculations then $\pi(\theta)$ would be our posterior $f(\theta|\mathcal{D})$
 - MCMC has the nice property that the update depends only on the ratio

$$\frac{\pi(\theta')}{\pi(\theta)} = \frac{f(\theta'|\mathcal{D})}{f(\theta|\mathcal{D})} = \frac{f(\mathcal{D}|\theta') f(\mathcal{D}|\theta')}{f(\mathcal{D}|\theta) f(\mathcal{D}|\theta)}$$

- That is, the normalising factor $f(\mathcal{D})$ that appears in Bayes' rule cancels
- This is important because this is usually incomputable (although we can use MCMC to estimate this)
- There is also often another advantage of this ratio: if we only change one variable at a time then frequently we can compute the ratio much more efficiently than the full likelihood and priors
- When this happens then it pays to choose a single variable, θ_i , at a time, choose a neighbour θ'_i from a distribution $p(\theta'_i|\theta_i)$ and decide whether to accept this update
- This one-variable-at-a-time procedure goes by the name of *Gibbs' sampling*

- **Convergence**

- A problem with MCMC is that your initial parameter, $p(0)$, is different from π so you have to wait some considerable time before your samples are from posterior distribution
- As we saw earlier the convergence rate is determined by the second largest eigenvalue of the transition matrix $\mathbf{M}$ (we never explicitly calculate $\mathbf{M}$ so we don't know what this eigenvalue is)
 - * If it is very close to 1 then your convergence can be slow
- Furthermore if your posterior distribution is broad you need lots of independent samples to accurately compute expectations
- To obtain an independent sample we have to run our MCMC a long time
- In practice you have to throw away some samples in a *burn-in* period
- You can then average over all the remaining samples you have (including repetitions where you don't accept a move)
- If you do this long enough then you should get samples that cover values of θ with a high posterior probability
- MCMC is slow because of this
- There are a lot of clever tricks to speed up convergence and decorrelation times

- Often MCMC seems daunting because of the apparent difficulties

- But don't be put off
- You only need to use the tricks for extremely complicated problems
- Usually a simple implementation works fine
- Modern computers are so fast that convergence isn't usually a problem
 - * Convergence can be a problem in pathological cases—these occur when using MCMC to model some physical systems—but for many Bayesian problems the Markov Chain is *rapidly mixing* meaning the convergence time is not excessively long

2.3 Variational Techniques

- This is a completely different approach to MCMC, but it tries to solve the same problem, but uses a different strategy
- This is again technically challenging and I don't expect you to memorise the derivation
- We saw in earlier lectures that we can cheat by seeking the parameters that maximise the posterior (the, so called, MAP solution)
- But this sacrifices all the probabilistic information

- In variational techniques we try to approximate the posterior with a simpler distribution
- That is, we approximate the prior $f(\boldsymbol{\theta}|\mathcal{D})$ with a simpler distribution $q(\boldsymbol{\theta}|\mathbf{w})$ where we get choose $\mathbf{w}$ to make q close to f
- Usually we choose q to be a factorisable distribution

$$q(\boldsymbol{\theta}|\mathbf{w}) = \prod_i q(\theta_i|w_i)$$

- These techniques were first developed in the physics community and so come with a strange language
- In the *variational approximations* we consider the **variational free energy**

$$\Phi(\mathbf{w}) = \int q(\boldsymbol{\theta}|\mathbf{w}) \log\left(\frac{q(\boldsymbol{\theta}|\mathbf{w})}{f(\boldsymbol{\theta}|\mathcal{D})}\right) d\boldsymbol{\theta}$$

- writing $f(\boldsymbol{\theta}, \mathcal{D}) = f(\boldsymbol{\theta}|\mathcal{D}) f(\mathcal{D})$ we can rearrange the variational free energy as

$$\begin{aligned}\Phi(\mathbf{w}) &= \int q(\boldsymbol{\theta}|\mathbf{w}) \left(\log\left(\frac{q(\boldsymbol{\theta}|\mathbf{w})}{f(\boldsymbol{\theta}|\mathcal{D})}\right) - \log(f(\mathcal{D})) \right) d\boldsymbol{\theta} \\ &= \text{KL}(q(\boldsymbol{\theta}|\mathbf{w}) \| f(\boldsymbol{\theta}|\mathcal{D})) - \log(f(\mathcal{D}))\end{aligned}$$

- where $\text{KL}(q(\boldsymbol{\theta}|\mathbf{w}) \| f(\boldsymbol{\theta}|\mathcal{D}))$ is the KL divergence that measures the "distance" between $q(\boldsymbol{\theta}|\mathbf{w})$ and $f(\boldsymbol{\theta}|\mathcal{D})$
- we have previously shown that the KL-divergence is non-negative and equal to zero if the two distributions are the same
- The term $\log(f(\mathcal{D}))$ is the *marginal likelihood* or *evidence* and does not depend on the parameters $\mathbf{w}$
- If we choose $\boldsymbol{\theta}$ to minimise $\Phi(\mathbf{w})$ then we minimise the KL-divergence and force $q(\boldsymbol{\theta}|\mathbf{w})$ to approximate the posterior $f(\boldsymbol{\theta}|\mathcal{D})$
- The variational free energy can also be written as

$$\begin{aligned}\Phi(\mathbf{w}) &= - \int q(\boldsymbol{\theta}|\mathbf{w}) \log(f(\boldsymbol{\theta}, \mathcal{D})) d\boldsymbol{\theta} + \int q(\boldsymbol{\theta}|\mathbf{w}) \log(q(\boldsymbol{\theta}|\mathbf{w})) d\boldsymbol{\theta} \\ &= - (L_q(\mathbf{w}) + H_q(\mathbf{w}))\end{aligned}$$

- Minimising the variational free energy is equivalent to maximising $L_q(\mathbf{w}) + H_q(\mathbf{w})$
- $L_q(\mathbf{w}) = \int q(\boldsymbol{\theta}|\mathbf{w}) \log(f(\boldsymbol{\theta}, \mathcal{D})) d\boldsymbol{\theta}$ is the expected log-likelihood of the data, where we have marginalised with respect to our approximate posterior $q(\boldsymbol{\theta}|\mathbf{w})$
- $H_q(\mathbf{w}) = - \int q(\boldsymbol{\theta}|\mathbf{w}) \log(q(\boldsymbol{\theta}|\mathbf{w})) d\boldsymbol{\theta}$ is the entropy of our approximate posterior (this measure the uncertainty in $\boldsymbol{\theta}$)
- We thus balance maximising the likelihood of the data with maximising the uncertainty in our approximate posterior
- This means we fit the data, but guard against over fitting by allowing there to be a non-zero probability wherever it does not strongly contradict the data
- In practice both $L_q(\mathbf{w})$ and $H_q(\mathbf{w})$ tend to be quite easy to compute
- We are left with an optimisation problem for the parameters, $\mathbf{w}$, but this is usually quite quick to compute

3 Exercise

3.1 Mysterious Disease

- Recall in the lecture on probabilistic inference, we defined $Z(t)$ to be the number of people that catch a disease on day t
- We assumed the growth rate is given by

$$\mathbb{P}[Z(t+1)] = \text{Poi}\left(Z(t+1) \middle| \frac{r_0}{3} (Z(t) + Z(t-1) + Z(t-2))\right)$$

– that is people of contagious for three days

- The observed number of new people with the disease, $X(t)$, on day t is

$$\mathbb{P}[X(t) = k] = \text{Binom}(k|Z(t), p) = \binom{Z(t)}{k} p^k (1-p)^{Z(t)-k}$$

- Here is $X(t)$ for the first 30 days
0,0,0,0,1,1,2,1,0,0,3,10,19,34,44,93,117,221,376,633,
1021,1643,2701,4503,7414,12091,19999,33286,54612,90283
- Estimate p and r_0 from the data
- (If you prefer simulate your own data and estimate your parameters)

4 Answers

4.1 Mysterious Disease

- We need to decide on a prior
 - A reasonable prior for p is a Beta distribution perhaps with $a = b = 1$ (this is a uniform prior)
 - For r_0 we could use a Gamma prior with $a = 2$ and $b = 1$, this has a mean of $r_0 = 2$ (it seems to be considerably greater than 1)—we need to include $r_0^{19} e^{-20 r_0}$ as our prior (the normalisation is irrelevant)
- Now we have to choose proposal distributions for the new values of p and r_0
 - You could use $p' = p + U(-0.01, 0.01)$ and $r'_0 = r_0 + U(-0.1, 0.1)$ where $U(a, b)$ is a uniform random deviate between a and b
 - This could have a problem in that p and r_0 could take illegal values (i.e. we should have $0 \leq p \leq 1$ and $r_0 \geq 0$), but for my data this is unlikely to happen
 - We could choose $p' \sim \text{Beta}(10p, 10 - 10p)$ and $r'_0 \sim r_0 \text{Gamma}(5, 5) = \text{Gamma}(5, 5.r_0)$, this ensures that in expectation p' equals p and r'_0 equals r_0 with small variations
 - * Beware that there are two common definitions of Gamma distributions

$$\text{Gamma}(z|, a, b) = \frac{b^a}{\Gamma(a)} z^{a-1} e^{-bz}$$

$$\text{Gamma}^*(z|, \alpha, \beta) = \frac{1}{\beta^\alpha \Gamma(\alpha)} z^{\alpha-1} e^{-z/\beta}$$

- * You have to check which of these your libraries use
- * In this case we would have Metropolis-Hastings to ensure we get unbiased samples

- The variables $Z(t)$ are latent variables (they will vary for each iteration) we can ignore them, but we could also estimate their mean as they tell us the actual number of cases of people with the virus
 - My data was generated with $p = 0.1$ and $r_0 = 2.5$

Advanced Machine Learning Subsidiary Notes

Lecture 26: Entropy

Adam Prügel-Bennett

February 9, 2024

1 Keywords

- To do

2 Main Points

2.1

3 Exercise

3.1

Advanced Machine Learning Subsidiary Notes

Lecture 27: Information

Adam Prügel-Bennett

February 9, 2024

1 Keywords

- To do

2 Main Points

2.1

3 Exercise

3.1